Autonomous Join-trip Matching Travel Mobile Application using Multi-Agent Technology

By

WIAD Fernando

Submitted in partial fulfilment of the requirements for the award of **BSc (Hons)**

in Computer Engineering

Department of Computer Engineering

Faculty of Computing

GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY

2024

Autonomous Join-trip Matching Travel Mobile Application using Multi-Agent Technology

By

WIAD Fernando

D/BCE/21/0004

Supervised by

Dr. Buddhitha Hettige

Submitted in partial fulfilment of the requirements for the award of BSc (Hons)

in Computer Engineering

Department of Computer Engineering

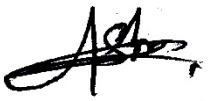
Faculty of Computing

GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY

2024

AUTHOR'S DECLARATION

I hereby affirm that the content given in this dissertation is solely my original work, and that I have not previously submitted any portion of this work for academic credit towards a degree or certification at any other institution of higher education, without duly acknowledging and citing the relevant sources.



01. 11. 2024
……. ……..

Date: Signatures of the Candidates:

WIAD Fernando

02. 11. 2024
……. …….

Date: Supervisor

Dr B Hettige

ABSTRACT

The Autonomous Join Trip Matching Mobile Application addresses the daily commuting problem in Sri Lanka by creating a robust ride sharing platform while being ethical. In this project, Multi Agent Technology is leveraged to build an intelligent system that enables efficient transportation solutions based on the interests of the users with common destination interests. The aim of the application is to reduce the waiting times and travel costs by reducing its use to collaborative trip planning and thus optimizing the urban travel experience. The application is at its core based on decentralized decision making, where intelligent agents autonomously handle the ride requests, prioritize the user's needs and adapt to the real time conditions without human intervention. This system is a key innovation because it integrates machine learning algorithms to maximize route optimization, while providing a better user experience. Furthermore, a powerful geo tracking system allows for quick communication between passengers and drivers, sending instant notifications when it's required and corrective action as needed. This project contributes to filling this research gap by applying Multi-Agent Technology into mobile applications in the area of transportation, which has only been marginally addressed. The application focuses on ethical trip matching mechanisms to not only promote efficient use of resources but also improve socio economic development in Sri Lanka. The results of the initial evaluations show significant improvements in user convenience and satisfaction, marking the application as an innovative solution in the field of mobile transport applications.

Key words: *Multi-Agent System, Join-Trip matching, Autonomous, Ride-share, Taxi-share, Urban Taxi, MAS mobile application*

ACKNOWLEDGMENT

I wish to thank deeply Dr. B. Hettige for his constant guidance and his insightful feedback that made it possible to finish this project. Helping to steer my work has been their encouragement and expertise, and their confidence to pursue ideas that are innovative. Finally, I would like to thank my family for providing unwavering support and encouragement, helping me to remain focused on my academic tasks. I thank, with gratitude, the natural patterns and structures that have inspired the conceptual foundations of this research. Adapted to the digital age here, these timeless designs are a reminder of the synergies that can occur when technology and nature meet. I want to say my sincerest thanks to all who have made this academic journey possible, directly and indirectly.

Contents

List of Tables

List of Figures

LIST OF ABBRIVIATIONS

ACL - Agent Communication Language

GUI - Graphical User Interface

HTML - Hypertext Mark-up Language

HTTP - Hyper Text Transfer Protocol

IT - Information Technology

MAS - Multi-Agent System

MVP - Minimum Viable Product

SQL - Structured Query Language

STOP - Simple Text Oriented Messaging Protocol

TCP - Transmission Control Protocol

UML - Unified Modeling Languag

CHAPTERS
Chapter 01

INTRODUCTION

1.1 Introduction

The fast growing transportation and mobility arena requires us to find the most available and environmentally friendly solutions for urban commuters to meet their needs. In places like Sri Lanka where traffic congestion, high transport cost, and the availability of public transportation are limited, there is an ever growing demand for more flexible, cheaper, and eco friendly commuting modes. The complexity of these suggested solutions indicates that innovative and affordable ways to optimize daily travel, and therefore relieve conventional transportation systems, are needed.

The focus of this project is on combining intelligent agent based systems with mobile application to develop a pioneering approach for real time taxi sharing which creates dynamic environment where autonomous trip matching emerges using decentralized approach. The solution relies on Multi Agent Technology, using autonomous decision making agents that collaboratively and seamlessly search through, connect and matching passengers and drivers with common travel interests. By this approach, the project presents an efficient, trustful way of trip sharing between users with similar destinations and thus decreasing individual transportation costs and decreasing travel wait times. This project creates connected taxi routes with intelligent agents to boost route efficiency, using the flexibility and adaptability of the routes as well as user suggestions, dynamically and in real time, to best match shared destinations.

With ethics, practicality and accessibility in mind, the design of the "Join Go" application is made. The application goes further than just sharing rides; it features state of the art geo tracking and machine learning algorithms to compute optimal routes in real time, forecast shared travel patterns, and adapt to passenger needs. Such intelligent route optimization is being carried out by Join Go to introduce a new mode of transport to Sri Lanka, reducing environmental impact and enabling the socio-economic challenges, by providing a lower cost alternative to individual taxi services.

This project bridges a unique research gap by combining two traditionally separate fields through the implementation of novel Multi-Agent Technology tailored for a mobile application infrastructure. Using the application’s decentralized agent framework, an efficient, scalable model for collaborative transport planning is developed that responds to the changes in mobility needs that characterize cities.

1.1.1 Multi-Agent Technology in Join Go

In the Join Go project, Multi-Agent Technology is an important contribution to system efficiency and passenger and driver travel experience optimization. It deploys a decentralized approach to create each individual identity for each agent in order to work autonomously and efficiently. This structure reduces database traffic since agents can function without huge amounts of temporary data in the system.

When the agents' tasks are done, the agents are removed from the container systematically, saving the system resources and increasing overall performance. Not only does this methodology make operations run smoother but also makes for an ethical trip matching process, making sure passengers and drivers get paired up correctly with regards to interests and routes. Finally, this application with Multi-Agent Technology greatly helps the Join Go platform increase the operational efficiency by providing the user with a seamless and responsive user experience and ensuring effective resource management.

1.2 Research Problem

The global demand for efficient and affordable transportation solutions is a huge challenge, especially in countries like Sri Lanka where traffic congestion, high transportation costs and lack of accessible public transport all contribute to making commuters lives hell. The problems are exacerbated by the requirement to route travel and minimize waiting time, especially for passengers that depend on shared taxis for affordable and predictable transport. Without effective solutions, any individual commuter must pay more for transportation, spend more time travelling, while cities experience more congestion and environmental strain.

Despite these challenges, however, adoption of ride sharing and shared mobility applications is slow, in particular of real time, ethical trip matching that enables both convenience and fair cost distribution among passengers. A major gap exists in the current lack of technology that supports decentralized, intelligent trip matching. This gap is critical to closing a practical, low cost and scalable solution to support shared travel arrangements, particularly for users who have common travel destinations and are able to benefit from cooperative ride sharing.

In our work, we introduce 'Join Go', a mobile application that uses Multi Agent Technology to autonomously match trip requests from passengers with drivers travelling along the same route. The project aims to reduce commuting costs and improve efficiency through the use of decentralized, intelligent agent-based systems for ethical and practical ride sharing. In this work, we endeavor to close the gap between existing limited shared ride solutions and a future autonomous adaptable system

capable of optimizing transport in real time to enable commuters with an easy and sustainable travel option.

1.3 Research Background and Motivation

The global demand for efficient and affordable transportation solutions is a huge challenge, especially in countries like Sri Lanka where traffic congestion, high transportation costs and lack of accessible public transport all contribute to making commuters lives hell. The problems are exacerbated by the requirement to route travel and minimize waiting time, especially for passengers that depend on shared taxis for affordable and predictable transport. Without effective solutions, any individual commuter must pay more for transportation, spend more time travelling, while cities experience more congestion and environmental strain.

Despite these challenges, however, adoption of ride sharing and shared mobility applications is slow, in particular of real time, ethical trip matching that enables both convenience and fair cost distribution among passengers. A major gap exists in the current lack of technology that supports decentralized, intelligent trip matching. This gap is critical to closing a practical, low cost and scalable solution to support shared travel arrangements, particularly for users who have common travel destinations and are able to benefit from cooperative ride sharing.

In our work, we introduce 'Join Go', a mobile application that uses Multi Agent Technology to autonomously match trip requests from passengers with drivers travelling along the same route. The project aims to reduce commuting costs and improve efficiency through the use of decentralized, intelligent agent-based systems for ethical and practical ride sharing. In this work, we endeavor to close the gap between existing limited shared ride solutions and a future autonomous adaptable system capable of optimizing transport in real time to enable commuters with an easy and sustainable travel option.

1.4 Overall Aim

This project aims at developing and implementing an innovative mobile application, Join Go, based on Multi-Agent Technology to make ride sharing for daily commuters and long distance travelers in Sri Lanka easy, ethical and cost effective. The target of this application is to make shared taxi services more accessible and convenient, from that addressing the problem of long waiting times, high peak hour prices and lack of taxis during rush hours. The core objective is to develop a scalable system to

intelligently match passengers with similar travel routes in real time, thereby reducing individual travel cost and maximizing use of taxi resource.

To achieve this, the research interweaves intelligent agent-based communication, decentralized decision making and real time geolocation tracking, such that users benefit from a flexible and interactive platform. This project will address these goals and through this transform the ride sharing landscape in Sri Lanka with a practical and sustainable solution for both the commuter and taxi service operational efficiency. In the end, Join Go aims to offer a better option for Sri Lankans to commute, which is more equitable, accessible and sustainable in the country's transportation sector, and ultimately contributes to overall socio-economic improvement in the sector.

1.5 Objectives

In order to achieve the main purpose of the Join Go project, it is decided to achieve the following specific objectives. These targets cover the analysis, development, and evaluation phases necessary to create an efficient, ethical, and sustainable ride-sharing mobile application tailored for Sri Lankan commuters:

1. Reviewing existing ride sharing and multi agent technologies in order to close gaps and seize opportunities for taxi sharing applications.

2. Find the most appropriate technologies for implementing decentralized multi agent communication within a mobile application framework.

3. We design and develop the Join Go mobile application, which leverages Multi-Agent Technology, Spring Boot, React Native and real time geo tracking to facilitate ethical trip matching and cost sharing for both short and long distance trips.

4. System performance will be evaluated in terms of user experience, cost reduction, and the performance of agent based decision making in different traffic conditions (rush hours).

5. See to the practical usability and scalability of Join Go for the Sri Lankan market, to ensure that it can serve the everyday commuters as much as the long distance travelers.

The goals for this systematic development are to create an application that helps improve commuter convenience while addressing the special transportation problems of Sri Lanka.

1.6 Scope of the Project

This project focuses on the development and deployment of the "Join Go" mobile application and the use of Multi-Agent Technology for ethical trip matching with passengers and drivers for maximum travel experience. The first goal is to develop a flexible system that is able to efficiently manage ride sharing and travel coordination among many users, thereby increasing the efficiency of the transportation services in urban areas.

For example, this system is comprised of the creation of decentralized individual identities for each agent, and they are able to operate autonomously. Since agents run their tasks without having to hold too much temporary data in the system, database traffic is minimized. When the agents are done with the actions they are deleted from the container freeing resources and improving overall system efficiency.

The project will cover the entire development cycle of the “Join Go” application, from design to construction, to testing and evaluation of the system. Real time geo tracking will be adopted, incorporating automatic alerts and communication between collaborative passengers and drivers for corrective actions. The aim of the project is to achieve these objectives to greatly improve the accessibility and affordability of transportation services, responding to the increasing need for convenient and cost effective travel in situations where taxis are not available.

Ultimately, we want to create a new taxi sharing experience in Sri Lanka, where taxis would be shared more sustainably and in a user friendly manner, contributing to broader efforts to improve travel experience while tackling socio-economic issues in the region..

1.7 Layout of the Thesis

Chapter Number	Titel	Content
Chapter 1	**Introduction**	This chapter presents an overview of the research project introducing the background context, motivation, and research questions of the study. The paper outlines the challenges, the overarching aim of the project, specific objectives, hypotheses, and presents the innovative solution of the ‘Join Go’ mobile application which implements MultiAgent

		Technology for ethical ride sharing and improved travel experiences.
Chapter 2	**Literature Review**	This chapter reviews extensively existing literature regarding the functionalities, benefits, and limitations of ride sharing applications. It critically reviews previous work on Multi Agent systems and their applications in transportation and presents a framework that allows us to understand the current state of the technology.
Chapter 3	**Approach**	The chapter presents the methodologies developed to tackle the identified transportation issues. The "Join Go" application design is informed by the assumptions, inputs, outputs, processes, and user interactions of the system features and requirements as presented in this report.
Chapter 4	**Analysis and Design**	The analytical and design phases of the project are addressed in this chapter. It presents the theoretical frameworks and models that drive the architecture of the system, and the design specifications that guide the system's implementation.
Chapter 5	**Technology Adapted System Development**	This chapter provides detail of the specific technologies and tools used in the development of the "Join Go" application. It describes how we undertook the development and explains how it was coded and implemented.

Chapter 6	**Testing and Implementation**	In this chapter we outline the protocols of testing the application for its functionality and reliability. The system is discussed and the methodologies used in ensuring that the system meets its performance criteria and the user requirements.
Chapter 7	**System Evaluation**	This chapter reports the results of the evaluations of the developed system. It presents the results of the testing and evaluates how the application can improve ride sharing experience and shorten the wait time.
Chapter 8	**Conclusion and Further Work**	This concluding chapter offers a summary of the findings from the research and discusses possible avenues for enhancing and further developing the "Join Go" application. It suggests areas for future work that would expand its capabilities and further increase its impact on transportation in Sri Lanka.

Table 1: Context of Thesis

1.8 Summary

The first chapter starts off by setting up the foundation and the framework of the project. It is essential in understanding the proposed system. The introduction establishes a context in which the project's objectives along with other important parts of the research become easier to understand or comprehend. The purpose of this section is to provide an overview of the project's key concepts followed by a structured presentation to help the reader understand the project.

Chapter 02

LITERATURE REVIEW

2.1 Introduction

This chapter leads to an exhaustive study of present methodologies and strategies in the field of transportation management with particular attention to the issues and solutions of the ride-sharing systems. The analysis is then expanded to the deployment of Multi-Agent Technology for better trip matching processes to optimize the travel experience for passengers as well as drivers. In the light of the growing ineffectiveness of urban transport systems during peak hours, this literature review attempts to identify approaches that can effectively reduce waiting times and operational costs.

In the context of Sri Lanka and in particular at KDU Kotelawala, long waiting times for short trips and the soaring cost of taxi services during rush hour has necessitated the need to understand all available solutions. The review seeks to identify what's strong and weak in current systems; how emerging technologies can fill the gaps. In this post we will discuss studies and practices that paved the way for more efficient ride sharing mechanisms and discuss the economic implications of using such systems by the everyday users.

We will incorporate relevant metrics, e.g., empirical measures of the popular times for taxi services, rush hour dynamics around KDU Kotelawala, Sri Lanka to illustrate the impact of rush hour dynamics on transportation efficiency. That data will then be used to analyze how Multi Agent Systems can help mitigate the effects of high demand in the proposed ride sharing solution to increase the effectiveness and user experience of the overall solution.

Day	Morning Rush (7 AM - 9 AM)	Evening Rush (5 PM - 7 PM)
Monday	High Demand	High Demand
Tuesday	Moderate Demand	High Demand
Wednesday	High Demand	High Demand
Thursday	Moderate Demand	High Demand
Friday	High Demand	Very High Demand
Saturday	Low Demand	Moderate Demand

Day	Morning Rush (7 AM - 9 AM)	Evening Rush (5 PM - 7 PM)
Sunday	Low Demand	Low Demand

Table 2:Example Table: Popular Times for Taxi Services at KDU Kotelawala, Sri Lanka

We capture peak demand times for taxi services in this table, this gives us a basic understanding of the operational context of our project. We can use our Multi-Agent System to tailor to passenger demand patterns and maximize the opportunities to reduce costs and improve the user travel experience during busy periods. The following sections will discuss in details some specific technologies, methods, and case studies that guide the design and development of the proposed solution.

2.2 Existing Multi-Agent Systems for Transportation Applications

However, the dynamic and complex nature of urban mobility has made the integration of MAS in transportation applications an area of research that has received a lot of attention. Many existing systems use MAS for efficiency improvement, user experience improvement, or to address transportation challenges. In this section, we look at notable systems, their operational framework and how they compare to conventional mobile ride applications.

2.2.1 Existing Multi-Agent Systems

1.1 **Coordination and Routing Systems**

- **Smart Mobility Solutions:** MAS is used in systems like IBM's Intelligent Transportation Systems to manage real time traffic and to do dynamic route optimization. These solutions solve these problems by coordinating multiple agents (vehicles, traffic signals, and mobile users) to smooth traffic flow and reduce congestion [1].
- **Carpooling Platforms**: MAS is used by apps such as BlaBlaCar to connect drivers with passengers that share the same travel route, thereby optimizing vehicle occupancy and lowering travel cost [2].

1.2 **Autonomous Vehicles**

- **Self-Driving Technology**: Companies like Waymo and Tesla are incorporating MAS in their self-driving systems, allowing vehicles to communicate with each other and the surrounding environment. This

technology aims to enhance safety, reduce accidents, and streamline navigation [3].

1.3 **Dynamic Ride-Sharing Services**

- **Agent-Based Ride Matching**: Kleinman and others are starting to explore MAS principles for dynamic ride matching services like Lyft and Uber, where many drivers and passengers are dynamically paired together on the fly, based on proximity, destination, and rider preferences [4].

2.2.2 Comparison with Conventional Mobile Ride Applications

While effective at solving the problem, conventional mobile ride applications often have limitations that can limit user experience and operational efficiency. Key characteristics of these applications include:

- **Centralized Control**: Traditional ride-hailing services rely on a centralized database to manage driver and passenger interactions. This can lead to delays in matching rides and increased wait times [5].
- **Static Matching Algorithms**: Basically, most systems today use basic algorithms that didn't adapt to changing conditions and end up being inefficient, especially at peak hours [6].
- **Limited User Customization**: Often conventional applications offer a one size fits all approach, without the option of personalizing to the user's preferences, such as shared ride based on common interests or routes. [7].
- **Higher Operational Costs**: Centralized systems are more costly to operate for the reason that they require many server resources and managing large volumes of data [8].

2.2.3 Advantages of Join Go Over Conventional Systems

"Join Go" introduces several advantages that enhance its effectiveness as a transportation application, particularly in the context of ride-sharing:

1. **Decentralized Decision-Making**: By using Multi-Agent Technology, the decision making for each single agent (passenger or driver) can be made independently, and in real time, without the help of a central authority. This increases responsiveness and reduces delays and increase system efficiency [9].
2. **Dynamic and Ethical Trip-Matching**: Ethical trip matching mechanisms are implemented in the system to route passengers and drivers based on common destinations and interests with a view to optimizing routes and considering social aspects and user satisfaction [10].
3. **Improved Resource Management**: By discarding agents after their tasks are finished, Join Go minimizes resource consumption, allowing the application to perform at its best without extensive server infrastructure [11].

4. **Real-Time Geo-Tracking**: Not only does integration of real time geo tracking improve user experience, but also better and quicker means of communication between drivers and passengers, which in turn smooths coordination and facilitates faster response times [12].
5. **Scalability and Flexibility**: Real time geo tracking integration not only improves user experience but also assists in better and faster communication between drivers and the decentralized architecture of Join Go is also scalable and will handle additional users and requests without affecting performance user experience. [13]
6. **Focus on User Experience**: Join Go improves the existing situation by focusing on users' preferences and offering personalized ride options which, in turn, enhance overall driver and passenger satisfaction [14].

2.2.4 User Willingness to Adopt "Join Go" and Shortcomings of Existing Applications

- User Willingness to Use "Join Go"

The adoption of the "Join Go" application can be attributed to several factors that align with contemporary commuter needs and preferences:

1. **Desire for Efficiency**: While urban populations grow, commuters are looking for ways to cut down travel time and help make their commutes more efficient. Go is riding this demand by offering trip matching and real time geo tracking features that cater directly to this demand and a streamlined commuting experience during rush hours. [1].
2. **Cost-Effectiveness**: Affordability has become a big consideration for many commuters as economic pressures rise. Go's shared ride model not only cuts individual travel costs, but also allows passengers to share fares with those traveling in the same direction, making it a good option for cost sensitive users. [2] [3].
3. **Enhanced User Experience**: Features that are user centric like personalized ride options and ethical trip matching through shared interests and common destinations are what Go focuses on. This approach speaks to users that are looking for more than just an efficiently transported solution; they want a travel experience that works with their social and personal preferences. [4].
4. **Safety and Trust**: With safety being the highest priority in the current era, Join Go offers a secure platform for users to travel in a secured way with verified drivers and other passengers. User ratings and ride tracking make users more willing to adopt the application and build trust from users. [5] [6].
5. **Sustainability Concerns**: More and more commuters are looking to find solutions that foster environmental awareness. Go's model encourages carpooling and shared rides, thus reducing the number of vehicles on the road, and reducing carbon footprints. [7].
6. **Technology Integration**: Smartphones and mobile technology are making it easier for users to embrace apps that make their lives simple. Join Go's intuitive

interface and easy to use functionality match today's tech savvy commuters' lifestyle, making it a convenient choice for transportation. [8].

1. Shortcomings of Existing Applications

While existing ride-hailing and car-sharing applications have made strides in improving urban mobility, several shortcomings hinder their effectiveness, particularly during rush hour commuting:

1. **Centralized Algorithms**: Centralized algorithms for ride matching should be avoided in many traditional applications, and this could result in inefficiencies. These systems become saturated with high demand during peak hours and lead to inefficient matches, longer wait times, etc. [9].
2. **Limited Flexibility**: The problem is that the existing applications has no choice like the way to adjust for changing traffic conditions or for changing user needs. For example, if the destination of a passenger changes, the system might fail to reassign the rides in an efficient manner causing frustration and delays. [10].
3. **Higher Costs During Peak Times**: The surge pricing models implemented by popular ride-hailing services may discourage users during peak times. This tariff strategy instantly creates higher than expected prices, forcing commuters to find alternative and cheaper ways to travel [11] [12].
4. **Inadequate User Customization**: Traditionally, applications lack characteristics of personalizing the experience; therefore, users are unable to choose ride alternatives which meet their social choices or transit patterns. Without a certain level of customization, however, the ride experience can leave something to be desire. [13].
5. **Safety and Trust Issues**: Existing ride-sharing applications still face safety and trust issues among their user bases. News of safety incidents have created skepticism around the safety of these platforms, causing many potential users to delay their adoption. [14] [15].
6. **Environmental Impact**: Advertising ride-sharing is a rare provision in an app market actively promoting more vehicles and infrastructure. Hell, the preference for single rides still contribute to urban gas pollutants as well as waste. [16].
7. **Inconsistent Service Quality**: Users often experience varying levels of service quality, which can be frustrating. Factors such as driver availability, vehicle conditions, and responsiveness can significantly affect the overall user experience [17].

By taking into account the user's willingness to use Join Go and the deficiencies of the existing applications, this study reveals the necessity of new solutions in urban transportation. Join Go addresses the critical pain points of rush hour commuting — efficiency, cost, safety and user experience — and positions itself as a viable alternative

to commuters' needs today. In addition to this, the application focuses on ethical trip matching and sustainability which will encourage the wider acceptance and use of the application.

2.3 Multi-Agent Systems for Decentralized Matchmaking

A MAS is a collection of intelligent agents that individually work in an autonomous way but work together to achieve the same goals. Each agent is autonomous to some degree as they can sense local conditions, decide, interact with other agents and the environment. Particularly in environments with for dynamic situations, such as transportation systems where conditions can change rapidly and need adaptive answers, this decentralized structure can be beyond useful. In these contexts, multi agent systems present a new approach to improve the efficiency and effectiveness of transportation services by allowing agents to quickly respond to user requests and to optimize resources allocation.

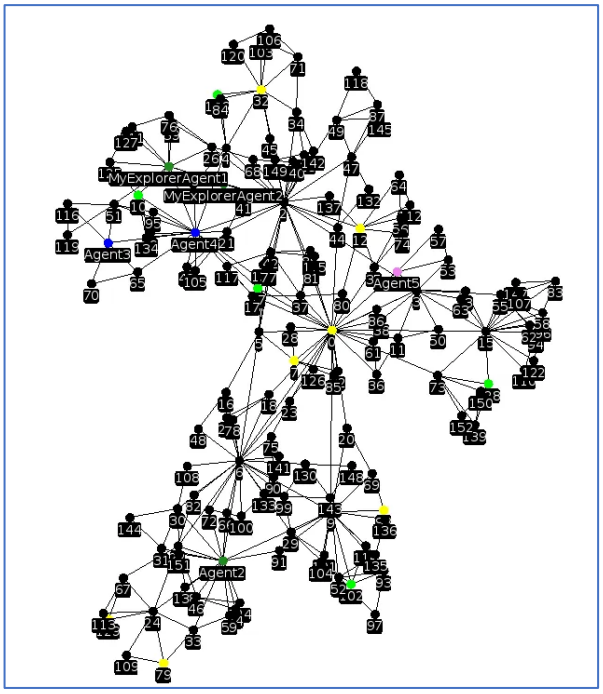


Figure 1: Inter-Connection of Agents in MAS (web-like spread)

Autonomous or direct matching in transport is the method of informing passengers of the availability of drivers or vehicles or services for hire without their being on a common pool or database or controller (figure 1). This is very beneficial especially for ride-sharing services, most probably because quick and effective symbiosis is critical. Due to the fact that a decentralized system calls for the division of load across several agents, requests are being appropriately processed individually and these systems offer

vast enhancements in response time as well as making it satisfactory for the users. The shift from the centralization of many of the managerial roles to decentralized operation enables the development of new strategies in transportation that are more viable for the ever-evolving urban environment.

The one in decentralized matchmaking is flexibility in system resource usage is one of the primary benefits of the suggested system. In a conventional client–server design approach, a singular database pool processes all the requests which may cause congestion during its time of high demand. On the other hand, decentralized structures allow local agents to have immediate information on traffic availability, passengers' demand, and peak hours. Since agents act autonomously, they are able to make decisions faster and provide faster service to passengers, while at the same time ensuring drivers' routes are the most efficient. This localized decision-making implies efficient resource use since only what is needed as observed within a local center is procured, without worrying about cross center waste.

Another great benefit from decentralized matchmaking is the tremendous effect on the reduction of the network data-base traffic. As in Complicated centralized system when number of request raises a particular data center gets overloaded and the quality of service begins to deteriorate. This is where decentralized matchmaking is pivotal in eradicating such challenge since it spread the requests among the agents in way that it can accommodate the transactions at one given time. This increases the capacity of the system to address the different need of clients as it expand the number of clients the systems have the potential of handling in case clients demands for the system increases. Decentralized systems thus correct the shortcoming of the initial idea of having a centralized network where the systems are linked because they easily accommodate fluctuating user traffic requiring rides, most especially during the day. This is particularly important in environments where the decision-making processes need to be fast and agents are able to make the decisions themselves since the system is decentralized. Every agent can act according to the change of conditions, thus being able to respond quickly to the new rides' request or some unexpected situation. For instance, if a passenger cancels a ride, the matching agent that the system uses can contiguously re-evaluate the number of drivers and redistribute them accordingly thus making the system flexible. This capability does not only help the operations to run more efficiently but also makes the customers happy as they will be served as soon as possible.

Another main strength of decentralized systems is the scalability. The same can be said about these systems: as the need in the transportation services rises, such systems can adapt to a higher traffic load without significant changes to its architecture. New agents can be added to the network depending on the growing user demands improving the strength of the system. These aspects of scalability are especially important in cities where the demand for transport can vary greatly over time. As such, the decentralized system of matcher agents means that agents are free to work independently and together, making it possible for the system to adapt to any changes in the circumstances or user needs as necessary.

Teamwork is a common feature of decentralised matching since the agents involved should work together. When a ride is ordered, agents who work in the application can talk to each other to find the most suitable driver according to distance and passenger preferences. This way of working is not only logical in terms of the usage of resources, but also contributes to the organization of shared transportation which is more effective and cheaper. For example, if two passengers ordered a taxi to go to the same place, the agents ensure the two share a taxi which saves the time of both the passengers and the driver.

Also, there is an opportunity to improve user private and secure experience because of the decentralized system where data is stored on several agents instead of one location. This structure reduces the probability of a breach of the data since the agents are isolated; the loss of one cannot affect the others. Users receive more control over their data as well as they will trust the application. Adding to the above mentioned feature, flexible and strong encryption and security features also prevent user's data from being hacked.

Another advantage of using multi-agent systems is that ethical factors are integrated into the existing model to match clients. If desired, agents can compare potential matches based on Social factors in order to ensure a travel experience where there is cooperation. Decentralized matchmaking can also be motivated by users with assistance through the use of shared interests or common goals to the destination. Besides, this ethical approach improves the satisfaction of users of transport services and unites passengers and drivers into a single transport organism.

Maintaining service quality is a vital requirement for decentralized systems to dynamically adapt to real time changes (figure 1). Agents' monitor the environment and can adjust operations on recurring basis according to present conditions. For instance, if a big event results in a dramatic spike of requests in a certain area, agents can re-route drivers, prioritize requests, etc. and maintain the system's efficiency regardless of pressure. In urban settings where traffic patterns and user needs change rapidly, this adaptability is critical.

In the end, the multi agent technology integration to decentralized matchmaking improves the user experience significantly. Reduced wait times, more reliable service and the convenience of seamless coordination of agents benefit passengers. This enables drivers to improve their earnings, their work workflow and better optimize their routes. This holistic approach to transportation not only helps enhance users' satisfaction level, as well promoting wider adoption of ride-sharing services, this can further allow for the advancement and long-term growth and sustainability of urban mobility solutions overall.

2.4 Summary

In the literature review chapter, the existing systems are reviewed as the systems relevant to integration of MAS in autonomous decision making for shared taxi applications. We argue that MAS is successful in decentralized matchmaking processes, can capitalize on better use of resources and reduced database congestion. The various implementations of MAS for transport contexts are reviewed, and the findings emphasize that MAS systems can respond dynamically to real time demands, a critical problem in the traditional centralized systems. These results lead to the conclusion that to provide a flexible and optimized transport service in a changing urban mobility landscape, MAS will play an important role. Finally, this multi agent framework addresses the existing barriers and presents a sustainable and user centric transportation system.

Chapter 03

APPROACH

3.1 Introduction

In this chapter, essential components that contribute towards addressing the ride sharing problem during rush hour are integrated to form the proposed system. This thesis will explore the interactions between the passengers, drivers, and intelligent agents that enable the matchmaking process.

Ride requests, destinations and user preferences will be inputs while matched ride options, estimated costs and travel times will be outputs. In the chapter, we focus on the processes by which the system can operate in an efficient way, and specifically on how Multi-Agent Technology helps with decentralized decision making, reduces the database traffic, and optimizes resource utilization.

When it comes to integration of MAS with mobile applications, there is a common scarcity of resources that effectively illustrate successful implementations. There are reasons for this scarcity: agent based systems are complex, real time data processing is required, and seamless user interaction is difficult. That is, this project addresses these complexities to address the gap in the current literature and practical applications by demonstrating how MAS can work successfully with mobile platforms to enhance a more responsive and efficient ride sharing experience.

3.2 Methodology

3.2.1 Agile Methodology

The “Join go taxi” app project makes use of Agile methodology to increase flexibility and responsiveness to change requirements in the development of the project. The project is divided into iterative sprints for development team to work on particular

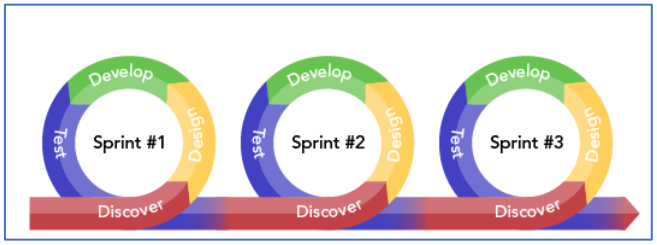


Figure 2: Sprint Runs for Agile Methodology

feature like user registration, ride matching, new type of ride, new feature and project adjustments. The iteration process allows it for team to adjust to the needs of the user and the challenges that are showing up in real time, to ensure every increment of the app incorporates the features and functionality in harmonious cooperation with other components. For new addition of function, this was an Individual project and simple steps for a sprint in the following figure were used. The project is divided into iterative sprints, allowing the development team to focus on specific features, such as user registration, ride matching, new type of ride feature which adding new features and adjustments in the project. This iterative process enables the team to adapt to user. needs and emerging challenges in real time, ensuring that each increment of the application meets the features and functionality working cooperatively with other components. As this was an Individual project simple steps for a sprint as in the following figure were employed in case of new feature of addition of function. Spans segments covering overall view plus included sprints for which sights of development of the project..

A sophisticated MAS architecture is used in this project to optimally match passengers and drivers in real time, especially during peak travel periods. Architectural approach to solve the fundamental urban transportation problem of demand for taxi services exceeding supply, resulting in long wait times and dissatisfied customers. The MAS uses advanced technologies like Java and JADE so that agents can communicate robustly inside the MAS through Agent ACL messages. This internal dialogue is required to enable smooth interactions and collaborative decision making among agents, that have been painstakingly crafted to play specific roles in the overall system.

Operational communications are powered by a Spring Boot backend that ties in with the MAS in an intricate fashion. This integration uses both TCP sockets and HTTP protocols, the former for communicating at runtime, and the latter for static code generation. The most common requests are handled by TCP Sockets for low latency communication, which is a must have for real time interactions. But HTTP is designed for less frequent operations; it allows data that does not need an immediate response to TCP connections. By dividing communication methods strategically, the system is able to better balance load in a way that lets it adapt to traffic variations without sacrificing performance.

When the system starts up, it automatically creates as few agents as possible (in this case three agents for TCP socket communication with the Spring Boot backend). A minimalistic approach is critical when handling system resources, without falling prey to the shortcomings of too much agent creation, which might cause performance bottlenecks. In addition, two HTTP agents are assigned to process requests, so that the system responds even if the number of user requests varies in volume. The system efficiency is improved as well as the user experience as the latency in responses is reduced.

The MAS generates specific agent types upon receiving operational requests, these agent types being tailored to the specific requirements of each request. JoinPassenger

Agent is one of the critical components which should identify other passengers in a configurable radius which is heading to the same destination. This means that users can share rides, saving a ton of money and shortening wait times. This agent is configurable from the backend for the duration of time it will remain active in the system, which is good because the needs and operational context of the users change. This adaptability guarantees passengers are matched suitably, and thus the total travel experience is better for them.

The ScheduleJoinPassenger Agent complements the JoinPassenger Agent by providing same functionalities for scheduled rides. This agent lets the users pre arrange shared rides by specifying desired pickup times and destinations directly in the user interface. The scheduling feature further eases the lives of passengers, some of whom may be forced to plan their travel in advance. This agent's deletion time is user defined, and is used to align the agent with the passenger's travel plans and to streamline the ride-sharing process based on the passenger's specific needs.

Another important role the LongDistancePassenger Agent has to play in the system is for users booking longer taxi rides with distances usually over 10 kilometres. This agent is able to recognize that conventional taxi services may be unable to handle such requests in the most efficient way, and intelligently groups long trips into manageable pieces. It connects different drivers for each segment of the journey to enable the passengers to make longer trips without delays. This innovative approach simultaneously meets the needs of the long distance travel passenger demand while maximizing the utilization of the resources in the taxi network.

The Driver Agent (a.k.a. taxi drivers) that actively seek trip requests are equally important. Such an agent filters the eligible opportunities based on the passenger's requests, sends viable options to the front end user interface while keeping the drivers to real time informed about the potential rides. As long as the driver is online and available, the driver agent continues to run within the container and updates its status continuously. This dynamic presence allows drivers to load manage their workload while informing the passengers of their option, thus improving service quality.

MySQL is used to help manage the database within the project; with the application's requirements, it is an efficient choice. The database encompasses four primary entities: There are Passenger that stores user profile information, JoinTrip Request that records basic details of the trip request made by the Passenger such as start and end locations, travel time, vehicle type and the type of trip; Driver Status that keeps track of driver's availability and profile details and Driver Match that keeps record of trip request for which there are driver candidates with their IDs. This structured approach to data management allows for effective filtering and transmission through the MAS using Spring Boot, and real time notifications via the STOMP protocol. This will make sure all the components of the system are synchronized at the end and will undoubtedly improve the user experience.

The STOMP is a key feature of our MAS that allows us to integrate push notification management to the front end application. Firstly, STOMP makes it possible for the

server to push notifications and updates directly to users, without them having to reload their app. In the Spring Boot backend, STOMP is used to send messages when an event happens — for instance when a new driver is matched to a passenger request or when an update is sent on the status of a ride. This push mechanism aids user engagement as well as keeping passengers and drivers up to date with the most important updates in real time.

STOMP requires creating a WebSocket connection between the front end and the backend. When there is a connection, we can send messages in a structured format, usually JSON wrapped around defining the nature of the update, IDs (if applicable) and any details required. Suppose a driver is matched to a passenger; then a STOMP message is broadcasted, which contains the driver's ID, estimated time of arrival, and other relevant information. This real time notification capability makes the ride hailing experience smoother and more intuitive.

Efficient information exchange is an integral feature of our MAS, and is provided by our internal communication mechanism that uses ACL messages. The conversation ID is assigned to each message so that interactions between agents can be tracked and controlled. This structured communication method permits agents to sort and categorize conversations, which leads to informed decision making in this operational context. Additionally, the template format for sending broadcasts makes it easy for agents to share messages immediately with the information they received. That level of communication efficiency is critical in sustaining a responsive, agile system that is flexible to real time demands.

Most of the interaction with the application is by the user creating user profiles that include basic travel details like pickup locations, preferred travel time, destinations and any special requirements. It personalizes services to meet the needs of the users and increases user satisfaction. Drivers can create profiles to detail their availability status and can choose from suggested trip options based on filtered requests available in the application on the driver's side. The dual engagement model ensures active participation from passengers and drivers alike in the system promoting a dynamic and responsive ride sharing environment.

MAS agents also have some behaviours in the application like one shot, cyclic, ticker, and stepper behaviours. Such behaviors are essential for maximizing agents' performance in particular use cases so that we can have less redundancy in code and have code reuse. Not only does this simplify development, it also makes the system easier to maintain, and the future can be added with future user needs and technology advancement.

Our methodology uses this comprehensive multi-agent approach not only to reduce redundancy and improve the overall efficiency of the system but also as a robust framework for managing the complexity of urban transportation. This gives the MAS a decentralized nature, so that agents are free to operate without burdening the central database with excess traffic. In particular, during high demand periods such as rush hour, quick decision making and response is key to user satisfaction.

This methodology integrates a Spring Boot backend with multi agent technology to yield a dynamic, efficient and user friendly system. We seek to provide a complete solution to the issues experienced in urban environments by urban commuters through the use of strengths provided by the intelligent agent behaviours and database management as well as real time push notifications through STOMP. In addition to improving the user experience, this design aims to maximize resource utilization in the transport network, and hence contribute to more sustainable and efficient transportation. This innovative approach brings to life the ride sharing experience in an efficient, cost-effective and user centric way.

3.3 Approach

3.3.1 Functional Requirements

- User Registration and Authentication: Allow users to create secure accounts for accessing app features.

- Compatible Trip Matching on the Go: Connect passengers with nearby drivers based on location and destination.

- Passenger and Driver Management

- Ethical Passenger Trip Matching: Ensure fair trip allocations based on user preferences.

- Appropriate Trip Suggestions: Offer trip suggestions based on user behavior and preferences.

- Delivery of Featured Trips:

    - Normal Taxi: Standard ride service for individuals.
    - Join/Shared Trip Taxi: Shared rides with other passengers.
    - Scheduled Shared Taxi: Pre-scheduled shared rides.
    - Long-Distance Taxi: Service for longer trips with segmented routes.

- Payment Management

- Feedback and Rating System

- Push Notifications

- Real-Time Geo-Tracking Updates

- Trip History Tracking.

3.3.2 Nonfunctional Requirements

- Data Entry and Management
- Maintenance and Upkeep
- Regulatory Compliance
- Maintenance and Diagnostics
- Security
- Scalability
- User-friendly Interface
- Integration with Third-Party Services
- Reliability
- Performance Monitoring.
- Accessibility
- Localization

3.4 Input

3.4.1 Passenger Inputs

- First Name
- Last Name
- Address Lines
- Gender
- Phone Number
- Email
- NIC (National Identity Card)
- Selected Type of Ride (from the four options: Normal Taxi, Join/Shared Trip Taxi, Scheduled Shared Taxi, Long-Distance Taxi)
- Expected Vehicle Type
- Pickup Location

- Destination Location

3.4.2 Driver Inputs

- First Name
- Last Name
- Address Lines
- Gender
- Phone Number
- Email
- NIC (National Identity Card)
- Vehicle Type
- Home/Residence Town
- Taxi Availability Status (update when online or available for rides)

3.5 System Requirements

3.5.1 Software Requirements

- Android OS platform with Google Play Services
- or
- IOS

3.5.2 Hardware Requirements

(min) or higher

- Processor (CPU): Quad-core CPU, 1.4 GHz or higher
- Memory (RAM): 2 GB
- Storage: Minimum 16 GB
- Display: At least 720p resolution (1280x720 pixels) with multi-touch capability.
- Graphics (GPU): OpenGL ES 2.0 support
- Sensors: Basic sensors like accelerometer and gyroscope for full app compatibility.
- Connectivity**:** Wi-Fi and Bluetooth, with optional support for GPS if location-based services are needed.

3.6 Output

The “Join Go Taxi App” provides a full range of outputs aimed at improving the ride sharing experience for both passengers and drivers focusing on efficiency and convenience. The intelligent trip-matching system is the key to its operation: it matches passengers with nearby drivers based on their location and destination preferences, and thus creates the optimal ride sharing schemes. A result of this process is that it lowers wait times and encourages cost sharing of passengers going in the same direction.

With a user-friendly interface, passersby can choose from different ride options ranging from normal taxis, join/shared trips, scheduled shared rides and long distance trips to best fit their travel needs. The application acts as a streamlining of ride request management to drivers, notifying them of eligible trip opportunities based on their availability and preferences. Real time updating of the taxi availability status enables drivers to keep their taxi availability status updated in real time thus improving the efficiency of operation and also regarding to the passengers request.

Built to increase ride sharing dynamics, optimize transportation efficiency and promote ethical travel practices, the “Join Go Taxi App” is a compelling proposition for modern commuters. The comprehensive outputs of the project help create a more connected, efficient and sustainable urban transportation ecosystem.

3.7 Summary

In this chapter, we go into a detailed look at the "Join Go Taxi App" approach, discussing the inputs, outputs, functional requirements, and unique features that specifically address the needs of both passenger and driver users. The system design allows users to make personal profiles, request or manage trip details, and receive real time trip updates to provide a streamlined and cost-effective ride sharing experience. The approach promotes an ethical, responsive, and dynamic transportation system through a focus on efficient user interaction and the utilization of multi agent technology for automated trip matching. This approach is based on a foundational assumption of efficiency and user convenience, and the strategic design seeks to maximize efficiency in taxi sharing and minimize overall travel costs. Finally, the chapter highlights the innovative part of the project, that is the use of decentralized agents to process trip data independently, which makes the application scalable and efficient in responding to various user needs in the shared transportation sector.

Chapter 04

DESIGN AND IMPLIMENTATION

4.1 Introduction

The design and implementation of the project is described in this chapter and the strategies and technical frameworks used to fulfill the research objectives and application requirements are outlined. The first phase of a design process is an in depth review of the project's functional and non-functional requirements to start with ensuring that each design decision supports the application's objective of fast, real time trip matching followed by seamless user interaction and secure handling of data. This section covers the complete system architecture, user interface as well as the backend framework with the integration of multi agent technology and communication protocol needed for real time interaction between passenger and driver modules. The system architecture and data flow are represented by various technical approaches such as UML diagrams and other design models, giving a clear idea of the multi layered system. In addition, the methodology also employs both analytical and conceptual techniques for capturing both technical details and user centric perspectives for a robust and scalable application architecture. For this reason, this chapter presents a structured basis for the following implementation and testing phases, in which the design principles are transformed into a mobile application with the necessary functionality for the purposes of the transportation sharing project.

4.2 Analysis

This project was divided into two parts: analysis stage, where the critical data was gathered to define the specific requirements that would be used as a responsive and user centered ride sharing application. Insights into existing mobile transportation solutions were gathered along with feedback from regular commuters and drivers, enriching the key information. It enabled us to understand the distinct needs of the passenger and driver in a ride sharing context including the need for real time location tracking, efficient trip matching, and adaptive functionality for disparate types of rides. The insights were directly used to structure the multi agent system which is able to support flexible, automated decision making balancing individual user needs and preferences while maximizing trip efficiency. Non functional requirements such as system security,

response time and scalability were also considered to make sure the application can cope with dynamic user interactions and maintain a high-performance level.

A detailed analysis of data management needs informed the design approach, where MySQL was chosen as it provides the required balance between efficiency, reliability, and ease of use for the application's database requirements. It ensures that the app can handle secure and fast data transactions even at peak load times since the application needs real-time interactions between agent and user. By supporting WebSocket communication and multi-agent architecture, seamless communication can be designed between backend, agent-based processing and frontend layers, granting users a unified experience. This analysis resulted in a comprehensive system design mapping the functional needs with user experience requirements providing a solid ground for serving up a scalable, secure and efficient app activity to operate an autonomous ride-sharing app to modern standards.

4.2.1 Data Gathering Protocol

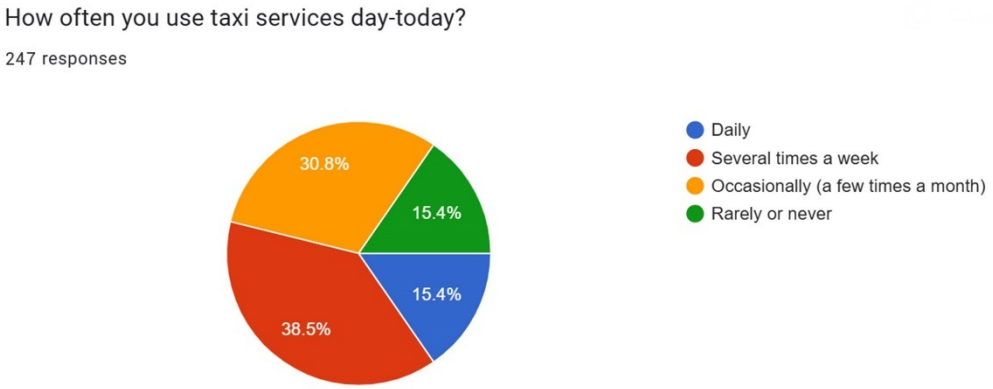
Method of Data Collection	Collected Areas
Interviews	• Comparison between current ride-sharing systems and the proposed Join Go app. • Identification of limitations and inefficiencies in existing ride-sharing models. • Insights on user-friendliness and essential app features.
Literature Reviews	o Analysis of existing ride-sharing and multi-agent technologies. o Examination of limitations in real-time trip matching and resource management in current systems. o Study of current transportation system drawbacks.

Method of Data Collection	Collected Areas
	○ Exploration of novel multi-agent approaches for enhanced matching applications.
Questionnaires	○ Market space for such requirement for possible stakeholders. ○ Challenges users adopting with existing ride-sharing applications. ○ User expectations for improved, ethical trip-matching. ○ Suggestions and solution ideas that informed the proposed system design.

Table 3: Data Gathering Protocol for project.

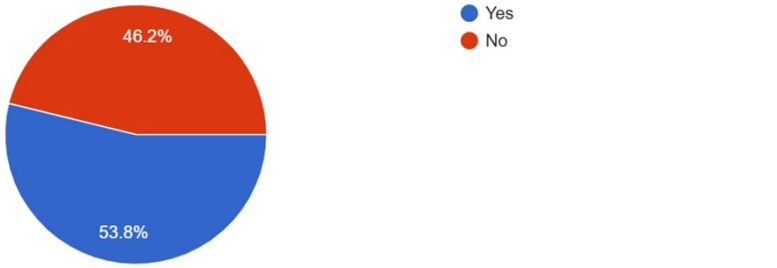
4.2.1.1 *Responses and Key Insights form Questionnaire Survey*

The responses I received from the questionnaire survey I distributed to friends, peers and professional contacts mainly people working in the corporate environment were mostly beneficial in providing me an indication of the whether there is a demand or relevance for this market. My main goal was to better understand attitudes and beliefs about the proposed service delivery model, assess perceived value as well as interest and reaction to this type of offering.



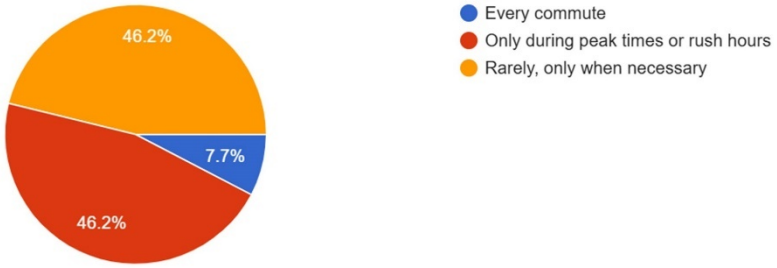
Are u a Daily Commuter?

247 responses



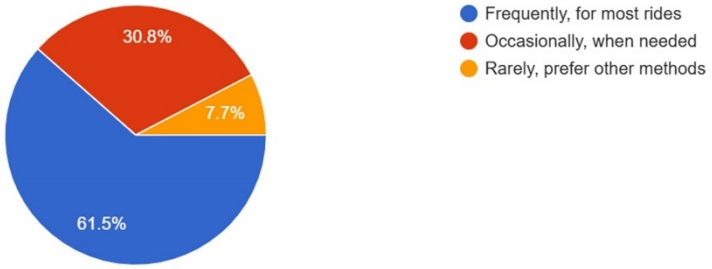
How often you look for a taxi service in commute?

247 responses



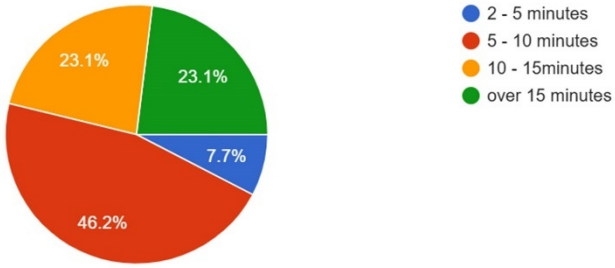
How often you use mobile app for taxi service?

247 responses



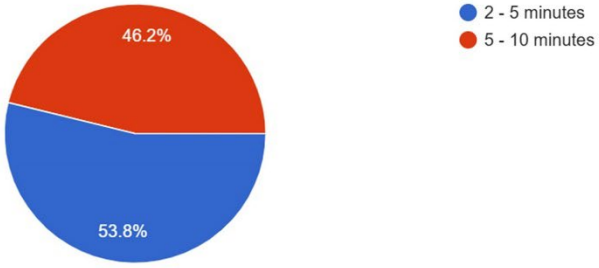
Average waiting time for taxi in Rush hours:

247 responses



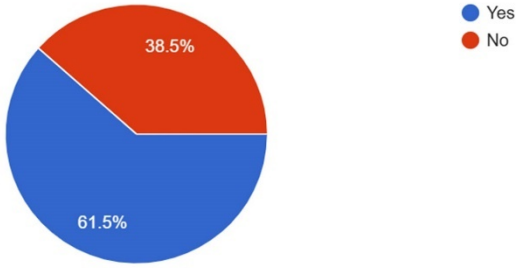
Average waiting time for taxi:

247 responses



Have experienced in sharing taxi

247 responses



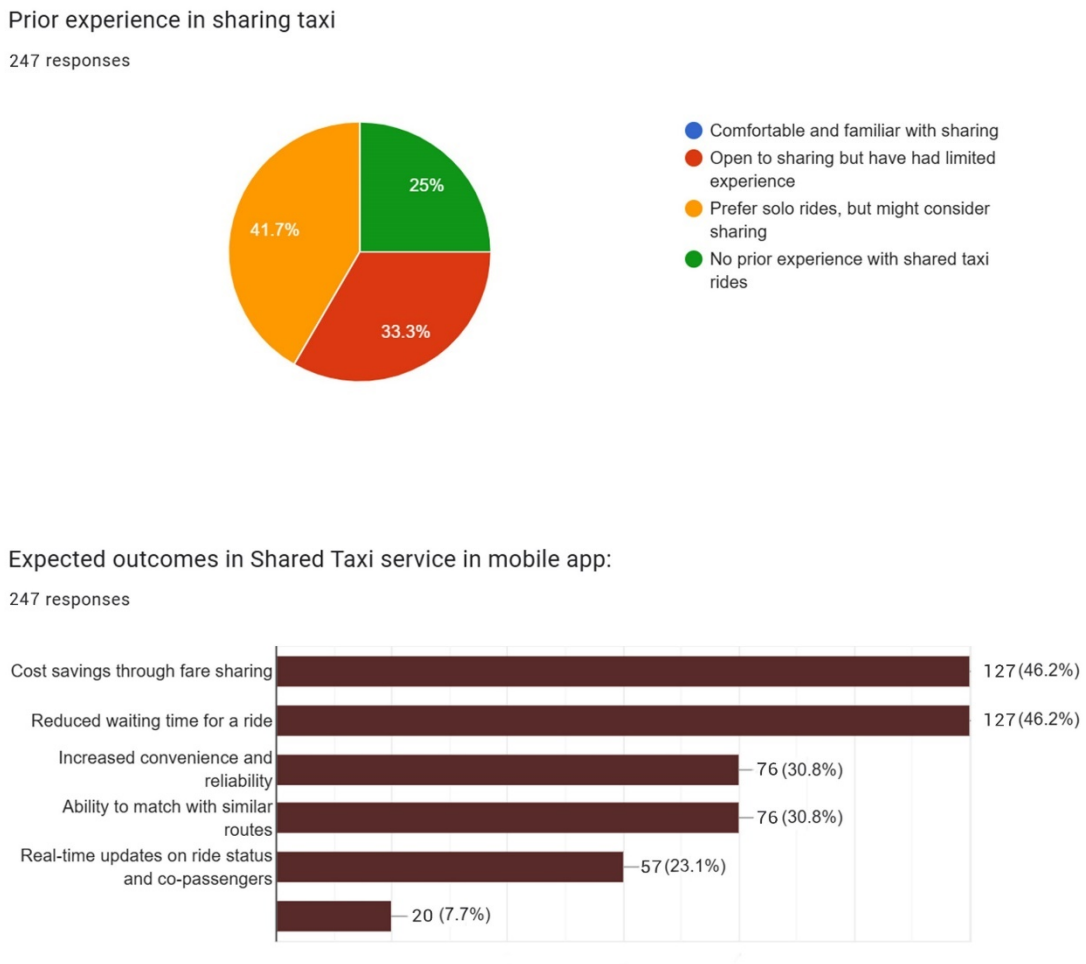


Figure 3:Questionnaire Survey Response graphs

Insights from the questionnaire indicates that most of the people preferred using mobile applications for availing taxi services as it is more easy, and convenient. A lot of respondents, particularly regular commuters, told us that they really feel the lack of taxis at peak hours, when the waiting times are totally disproportionate compared with

regular hours—an insight that we also got by actually going out and pitching it, so to speak, and also through the surveys. Perhaps most stark is how uncomfortable respondents are with the shared taxis in existing systems, with many expressing a willingness to accept shared rides if done in a reputable and trustworthy way (so during peak periods) (figure 3). From the responses, survey respondents cited reduced wait times, cost savings and proper trip-matching with the right co-passenger as the most awaited benefits of this service.

A few respondents sent thoughtful recommendations that enriched the latter stage development verticals of dev. The relative scarcity of detailed comments from busy professionals speaks to the success of the survey in targeting its audience. All in all, the feedback provides invaluable advice in the sense that it draws upon many of the types of improvements directly from the very practical ideas that the questionnaire dedicated respondents put forth, to drive the project forward.

4.3 Comprehensive System Architecture of Proposed System

The Go project is composed of several phases: the design and architecture phase includes a comprehensive review of the overall system architecture of Join Go project including the frameworks, the database, the user interface design and agent communication. It describes the system architecture using several UML methods use case diagrams, sequence diagrams, entity + relationship diagrams, and class diagrams to help convey what the system does and how the components will interact with one another. In the design, various types of agents including joinPassenger, scheduleJoinPassenger, longDistancePassenger, and taxiDriver agents are defined, along with their corresponding behavior types such as one-shot, cyclic, and ticker to ensure proper communication and behavior execution in the multi-agent system. We will also explain how these agents interact with each other to deliver real-time transportation services that improve user experience through ethical trip-matching and seamless coordination.

4.3.1 Multi-Agent System Architecture

I believe that the most productive way to explain the architecture being employed in this MAS for this project is to simply use the steps from initiation and example code as proof and a tool to understand the implementation.

The MAS executes on commands and calls from Spring Boot framework which provides seamless connection between MAS and an agent-based architecture. At the

start of the application, agents are established to be available for prompting as soon as the Spring Boot server is up and running. This architecture enables fast reactions from the system to incoming requests.

In addition to the core agents, the architecture allows for generating different agent types when required by calls from the Spring Boot application. The adaptiveness is essential for realistic scenarios in ensuring that the system is effective and applies to real-time. Architecture of Agents. After that the architecture of agents consist of three major types of agents. TCP Listener Agent, HTTP Listener Agent, and HTTP Sender Agent. The TCP Listener Agent listens to incoming TCP requests, while the HTTP Listener Agent listens to HTTP requests from the Spring Boot server. On the other hand, the HTTP Sender Agent makes communication from JADE to Spring Boot seamless by exchanging messages and commands through Spring Boot. This setup of agents not only ensures organized communication but also enhances the overall performance of the MAS in the taxi application.

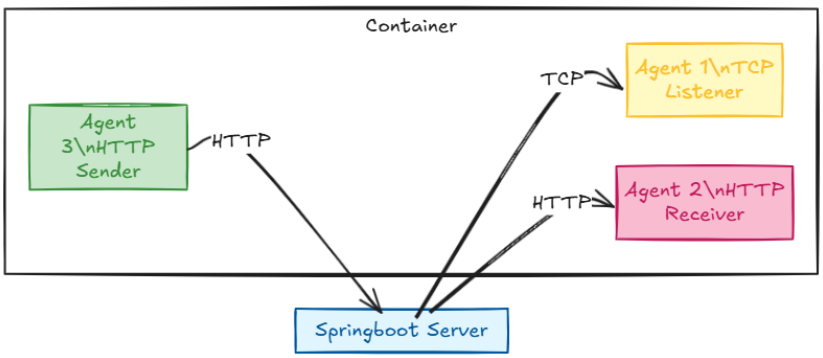


Figure 4: Initiation of MAS container to Establish Communication of server

The following figures illustrates the code for the main method, which initializes JADE and creates an agent of the PassengerTCPListner type, responsible for generating additional agents in response to requests from Spring Boot.

```java
import jade.core.Agent;

public class Main extends Agent {
    public static void main(String[] args) {
        // Create and start the passenger and taxi agents
        jade.core.Runtime rt = jade.core.Runtime.instance();
        jade.wrapper.AgentContainer container = rt.createMainContainer(new jade.core.ProfileImpl());
        try {


            container.createNewAgent("Jade2SBAgent", JadetoSB.class.getName(), null).start();
            container.createNewAgent("PortListenerAgent", "PassengerTCPListner", null).start();
            container.createNewAgent("HTTPControllerAgent", JadeHTTPControllerAgent.class.getName(), null).start();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Figure 5: Main Method, initiates Jade Server

```java
    private void createPassengerAgent(JoinRequestDTO passenger) {
        try {
            // Get the current container
            AgentContainer container = getContainerController();

            Object[] args = new Object[] { passenger };
            if(passenger.getTripType() == 2) {
                AgentController passengerAgent = container.createNewAgent(passenger.getJoinReqId(), PassengerAgent.class.getName(), args);
                passengerAgent.start();
            }else if(passenger.getTripType() == 4){
                AgentController schedulepassengerAgent = container.createNewAgent(passenger.getJoinReqId(), SchedulPasngerAgent.class.getName(), args);
                schedulepassengerAgent.start();
            }else if(passenger.getTripType() == 3) {
                AgentController longridepassengerAgent = container.createNewAgent(passenger.getJoinReqId(), LongRidePassengerAgent.class.getName(), args);
                longridepassengerAgent.start();

            }
            //System.out.println("Created agent: " +passenger.getJoinReqId());
        } catch (StaleProxyException e) {
            e.printStackTrace();
        }
```

Figure 6: Method in PassengerTCPListner that creating other agents dynamically

Through commands from Spring Boot, the PassengerTCPListener Agent dynamically creates agents within the container, initiating inter-agent communication via ACL messages based on each agent's designated methods and functions. Following figure 6

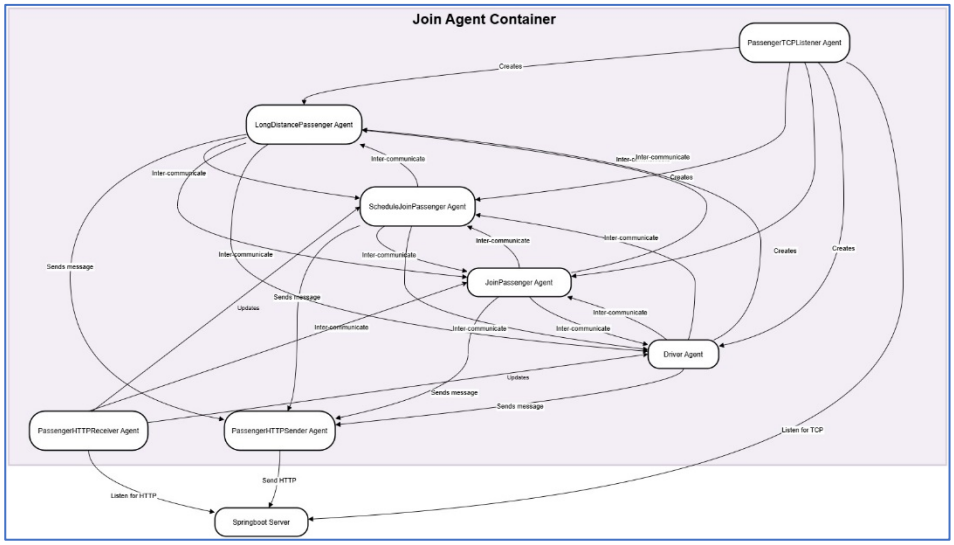


Figure 7: Simple diagram of communication in MAS Container when single instance of an agent is created in each type.

depicts the communication in the MAS for the simplest form it could have by only having one agent from each type and then can assume the populate the inter-communication when there are multiple instances of those types of agents dynamically created in the container.

Now let us discuss individual agents in the MAS for both passengers and drivers and how these agents are built to process various topics of the communication. Every agent possesses certain JADE behaviors particular to its type, and that is why it is very easy to modify them for other needs if necessary. For the purpose of structured and contextualised messaging all the agent communications employ ACL message template. This template has a conversation ID that helps sort the topic of each communication thread, thus making messages meaningful and easy to sort through. With these message templates, the system can effectively coordinate and control interactions between agents, while making each of the conversations relevant to the goal of the given task. This structured approach makes the co-ordination optimal and the interactions of the agents clear in the application.

4.3.1.1 *"JoinPassenger" Agent-*

Let's now explore the role of the `JoinPassenger` agent in this MAS, with a focus on its operation and eventual termination:

A passenger looking to share a taxi ride is represented by the JoinPassenger agent. The PassengerTCPListener agent provides the specific data for each passenger, such as location and destination details, from which this agent is created dynamically. From the Spring Boot server this data is needed to initialize the JoinPassenger agent's attributes, such as JoinRequestDTO (containing passenger data) and other lists for managing previously contacted and compatible match information.

Key Attributes

- **`passengerData`**: An instance of `JoinRequestDTO`, containing essential information about the passenger's ride request, including their destination, start coordinates, and schedule time.
- **`closeradius`**: A static radius defining the proximity threshold within which another passenger's ride request is considered compatible.
- **`destPlaceIdCheckednequlList`**: A list to track the place IDs that the agent has already checked for matching requests to avoid redundant communication.
- **`joinCompaitbleList`**: A list that stores compatible passenger requests for potential matches, used for periodic data transfer to the backend.
- **`messagedAgents`**: This list records other agent names that have already received broadcast messages, allowing the agent to efficiently manage communication.

The agent's operations unfold in distinct phases, from setup to behavior execution and eventual termination:

- **Setup Phase**:

At this point the agent registers itself with the directory facilitator (DF) in order for its services to be discovered. It also validates initialization arguments and if data is present, populates JoinRequestDTO attributes. Then multiple behaviors are added to the agent that perform specific tasks including broadcast location, detect matching requests, and send notifications.

- **Behavior Execution**:
    - *"destinationBroadcast" and "sendorinfotoMatch": Within these cyclic behaviors we manage communication with other agents, broadcasting the passenger's destination and responses for match filtering. The agent categorizes and manages different communication topics using conversation IDs in ACLMessage (such as placeId, newReqNotice, and fromPassengerJoinList). They enable the agent to identify passengers that have similar destinations and update its list of matches.*
    - *"agentdeleteTimer": This is a crucial timed behavior that monitors the agent's runtime. The agent has a TIME_LIMIT and in case of this limit if it expired, the agentdeteTimer checks whether there are any passengers in the list of joinCompaitbleList that have been matched. Then it compiles these matches into a ResJoinMachListDTO and sends it to Jade2SBAgent that does further server level processing. The agent then does its termination and runs doDelete() to complete it's role.*
- **Termination**:

The agentdeleteTimer behavior makes sure the JoinPassenger agent stops efficiently. When broadcast phase completes or TIME_LIMIT expires the agent cleans up and checks for any final matches to transmit and if found dispatches them to Jade2SBAgent. This makes it able to complete the cycle of communication and resource management, doing so by calling doDelete(), terminating self and releasing resources.

```
class sendorinfotoMatch extends CyclicBehaviour{


    @Override
    public void action(){


        MessageTemplate placeIdBroadcastTemplate = MessageTemplate.MatchConversationId("placeId");
        ACLMessage placebroadtMsg = receive(placeIdBroadcastTemplate);

        MessageTemplate joinpassinfoTemplate = MessageTemplate.MatchConversationId("joinpassenger");
        ACLMessage joinpassdataMsg = receive(joinpassinfoTemplate);



        if (placebroadtMsg != null) {
            if( placebroadtMsg.getContent().equals(passengerData.getDesplace_id()) && !isindestPlaceIdCheckednequlList(placebroad


                try {
                    ACLMessage response = placebroadtMsg.createReply();
                    response.setConversationId("joinpassenger");
                    response.setPerformative(ACLMessage.INFORM);
                    response.setContent("infrotoMatch");
                    response.setContentObject(passengerData);
                    send(response);
                    destPlaceIdCheckednequlList.add(placebroadtMsg.getSender().getLocalName());
                }catch (IOException e) {

                    e.printStackTrace();
                }

            }

        }
```

Figure 8:"sendorinfotoMatch" method in PassengerTCPListener Agent class

```
class destinationBroadcast extends  CyclicBehaviour{


    @Override
    public void action(){


        MessageTemplate newReqNoticeTemplate = MessageTemplate.MatchConversationId("newReqNotice");
        ACLMessage newReqNoticeMsg = receive(newReqNoticeTemplate);


        if (newReqNoticeMsg != null) {
            String mypalceId = passengerData.getDesplace_id();

            ACLMessage response = newReqNoticeMsg.createReply();
            response.setPerformative(ACLMessage.INFORM);
            response.setConversationId("placeId");
            response.setContent(mypalceId);
            send(response);



        }else {
            block();
        }

    }

}
```

Figure 9:"destinationBroadcast" method in JoinPassenger Agent class

The JoinPassenger agent is essentially a series of actions that find matching passengers and coordinate ride sharing requests. Figure 7 and 8 show some sample code snapshots of sample code and other code content is included in APPENDIX C. Structured in its communication using ACL messaging with categorized conversation IDs, it gives a very organized flow of information. From setup to autonomous termination, the agent's lifecycle details its key role in enabling efficient and timely ride sharing while the MAS is able to operate with little manual oversight.

4.3.1.2 *"ScheduleJoinPassenger" Agent*

In this work, the SchedulPasngerAgent is a passenger oriented agent that coordinates with other agents representing scheduled passengers to help facilitate shared ride scheduling. It has several key attributes initialized with values sent from the PassengerTCPListener agent when the agent is dynamically created. These values are pulled from the Spring Boot server, in particular for each scheduled ride request. Using these data points, the SchedulPasngerAgent performs matching processes and guarantees the passenger updates on potential shared ride matches.

The same set of attributes of the JoinPassenger and moreover schedule time for the trip are present in this agent class.

Several key behaviors of the SchedulPasngerAgent perform its operations for specific aspects of the agent's operation in managing a scheduled passenger request.

- Agent Registration and Setup:
  - In the `setup()` method, the agent registers itself with the Directory Facilitator (DF) using a `schedulepassenger` service type, making it discoverable to other agents looking for scheduled passengers. If initialization arguments are provided, `passengerData` is populated with this `JoinRequestDTO`, which includes scheduled deletion times and request details fetched from the Spring Boot server.

- **Behavior Execution**:
  - Scheduled Deletion: The agentscheduledelteTime behavior, which is an extension of WakerBehaviour, is triggered at a certain time determined by passengerData.getDelelteTime(). When the scheduled time is reached, the agent terminates, logging the termination and calls doDelete(). This results in the agent not being allocated system resources outside of when it is actively relevant to a passenger's scheduled ride request, after the ride has completed.

  - Matching Timer: Inside the time window (TIME_LIMIT) there is the agentdeteTimer behavior, which is a TickerBehaviour. It periodically checks the joinCompaitbleList, if matches are found, it compiles them into a ResJoinMachListDTO object. It then forwards match data to the

backend and sends this object to the Jade2SBAgent. The list is cleared and the agent is prepared to take a match if it is available after transmitting data.

- Destination and New Arrival Broadcasts: DestinationBroadcast and broadcastNewarrival are behaviors that broadcast the passenger's destination to other agents in the schedulepassenger category. The destination broadcast lets the agent announce its location and receive responses from agents with matching route or compatible passenger destination. New arrival broadcasts are used to notify other agents of any new request so that communication between the scheduled passenger agents is kept efficient.

- Matching Process: Within the sendorinfotoMatch behavior, when receiving destination broadcasts (placeId) from the agent, the agent will match with compatible passenger agents. A request is considered compatible if the distance between the current agent's location and a potential match is less than the closeradius. Secondly, the agent processes the joinCompaitbleList and updates it, then sends the list to Jade2SBAgent, when five matches are added, it sends the joinCompaitbleList. It results in a dynamic matching and communication system which responds to potential shared rides within the specified radius..

- **Termination**:

  SchedulPasngerAgent scheduling feature is important for coordinating and matching rides. The agent integrates a timed deletion behavior (agentscheduledelteTime) and terminates autonomously when the scheduled ride time elapses. This is the feature that enables MAS to save resources, maintaining only the agents that are active for real time ride requests.

```
class agentscheduledelteTime extends WakerBehaviour{

    public agentscheduledelteTime(Agent a, Date deleteTime) {
        super(a, deleteTime);
        // TODO Auto-generated constructor stub
    }
    @Override
    protected void onWake() {
        try {
            System.out.println("Scheduled time reached: "+ passengerData.getDeleleTime()+" --Deleting agent... @ "+ getLocalName());

            doDelete(); // Terminate the agent
        }catch(Exception e) {
            e.printStackTrace();
        }
    }
```

Figure 10:"agentscheduledelteTime" mehtod in ScheduleJoinPassenger Agent class

```
class firstnewreqplaceidone extends OneShotBehaviour{

    @Override
    public void action() {

        DFAgentDescription template = new DFAgentDescription();
        ServiceDescription sdnew = new ServiceDescription();
        sdnew.setType("schedulepassenger");
        template.addServices(sdnew);

        try {
            DFAgentDescription[] result = DFService.search(this.getAgent(), template);
            if (result.length > 0) {
                for (DFAgentDescription dfAgent : result) {
                    //AID agentAID = dfAgent.getName();
                    if( !messagedAgents.contains(dfAgent.getName().getLocalName()) ) {

                        String mypalceId = passengerData.getDesplace_id();
                        ACLMessage msgplaceinfo = new ACLMessage(ACLMessage.REQUEST);
                        msgplaceinfo.addReceiver(dfAgent.getName());
                        msgplaceinfo.setConversationId("placeId");
                        msgplaceinfo.setContent(mypalceId);
                        send(msgplaceinfo);

                    }

                }
            }
        } catch (FIPAException fe) {
            fe.printStackTrace();
        }

    }
}
```

Figure 11:"firstnewreqplaceidone" methos in ScheduleJoinPassenger Agent class

The SchedulPasngerAgent is a time managed, efficient passenger agent to do the match process for scheduled ride request in a limited time horizon. In figure 9 and 10, we provide some sample code snapshots and in APPENDIX C, we have included other Code content. This approach ensures that prompt agent deletion and low data retention are supported so that MAS's resource management goals are served, and the passenger's scheduling preferences are satisfied, both of which are critical to effective ride scheduling and shared travel planning in the MAS environment.

4.3.1.3 *"LongDistancePassenger" Agent*

This is a particular agent about the LongRidePassengerAgent class that deals with long distance trips by breaking the route into segments and sending taxi requests in manageable segments. In this way, it effectively aggregates a number of local drivers together to complete the entire trip, rather than using a single driver for a long distance trip. Values received from the PassengerTCPListener agent dynamically generate this agent and populate this agent's attributes. They are communicated back from the Spring Boot server.

In addition to pullulating attributes as JoinPassenger as a class for the agent, this agent also has points of coordination for the long trip from Google maps API call and decoded at the front end..

- Agent Registration and Setup:

  Upon initialization, the `LongRidePassengerAgent` registers itself in the JADE Directory Facilitator (DF) with the service type "longtrippassenger." The `JoinRequestDTO` data, which includes passenger details, long route coordinates, and segment distance, is provided as an argument during the agent's setup. This data is extracted from `passengerData` and stored in the following main attributes:

- `fullLongRoute`: List of coordinates marking the full route.
- `segmentdistance`: Maximum distance each segment can span.

  This setup is crucial for segmenting the trip and managing segmented route broadcasts.

-

- **Behavior Execution**:

  - Route Segmentation (segmentRouteBehave): This OneShotBehaviour calculates the distinct segments of the complete long route. By segment distance, the route is divided, so that the agent can handle smaller segments one by one. The longRouteSegments is a list of longrouteSegmentDTO that represent each segment as one. After being segmented, each segment is broadcasted to available drivers.

  - Initial Driver Broadcast (firsttoDriverBroadcast): This OneShotBehaviour looks for agents with the 'taxidriver' service type registered in DF and for each segment in longRouteSegments, it sends a taxi request (TaxiRequestDTO). For each segment, the agent sends an ACLMessage to a driver, encapsulating relevant segment data (start and end coordinates). This is what allows local drivers to fill in gaps on the journey, so that they can carry out the whole journey collectively.

- New Driver Broadcast (newtoDriverBroadcast): This listens for updates on new taxis available (conversationId "newTaxiAvailable".) When a broadcast is processed it creates a new taxi request for any remaining segment in longRouteSegments, allowing for dynamic responsiveness to available drivers along the route.

- **Termination**:

  This TickerBehavior monitors the amount of time that has elapsed from the time the agent was developed. Since it is advisable to manage resources, if the agent runs beyond its given TIME_LIMIT (15,000ms), the doDelete() method is invoked to self-terminate the agent.

```
class segmentRouteBehave extends OneShotBehaviour{

    private List<Coordinate> fullroute;
    private int segdistance;

    public segmentRouteBehave(List<Coordinate> dafullroute, int distance) {
        this.fullroute= dafullroute;
        this.segdistance = distance;

    }
    @Override
    public void action() {


        List<longrouteSegmentDTO> segments = new ArrayList<>();

        if (this.fullroute == null || this.fullroute.size() < 2) {
            return;
        }

        Coordinate segmentStart = this.fullroute.get(0);
        double accumulatedDistance = 0.0;
int count = 1;
        for (int i = 1; i < this.fullroute.size(); i++) {
            Coordinate currentPoint = this.fullroute.get(i);
            double distance = calculateDistance(segmentStart, currentPoint);

            accumulatedDistance += distance;

            if (accumulatedDistance >= this.segdistance) {
                // If accumulated distance meets or exceeds segment length, create a segment
                segments.add(new longrouteSegmentDTO( passengerData.getJoinReqId() ,count, segmentStart, currentPoint));

                // Reset for next segment
                segmentStart = currentPoint;
                accumulatedDistance = 0.0;
                count ++;
            }
        }
```

Figure 12:"segmentRouteBehave" method in LongDistancePassenger Agent class

```
class newtoDriverBroadcast extends CyclicBehaviour{

    @Override
    public void action() {

        MessageTemplate taxiBroadcastTemplate = MessageTemplate.MatchConversationId("newTaxiAvailable");
        ACLMessage taxiBroadcastMsg = receive(taxiBroadcastTemplate);

        if(taxiBroadcastMsg != null) {

            if(!longRouteSegments.isEmpty()) {
            try {
                TaxiRequestDTO toTaxi= new TaxiRequestDTO();
                toTaxi.vehicletype = passengerData.getReqVehicletype();
                toTaxi.JoinReqid = passengerData.getJoinReqId();


                ACLMessage response = taxiBroadcastMsg.createReply();
                response.setConversationId("passengerTripCall");
                response.setPerformative(ACLMessage.INFORM);

                for(longrouteSegmentDTO asegment: longRouteSegments) {
                      toTaxi.taxiReqid = asegment.getTripReqId()+ asegment.getSegNo();
                      toTaxi.startLat = (float) asegment.getStart().getLatitude();
                      toTaxi.startLon = (float) asegment.getStart().getLongitude();
                      toTaxi.destLat = (float) asegment.getEnd().getLatitude();
                      toTaxi.destLon = (float) asegment.getEnd().getLongitude();

                response.setContentObject((Serializable) toTaxi);
                send(response);
                }

            }catch (IOException e) {

                e.printStackTrace();
            }

        }else {
```

Figure 13:"newtoDriverBroadcast" method in LongDistancePassenger Agent class

The LongRidePassengerAgent class enables long distance passenger trips by breaking the journey into small route segments, and coordinating with numerous drivers. On initialization it registers itself as a "longtrippassenger" in JADE's Directory Facilitator and gets its data (route details and segment distance) from the PassengerTCPListener agent. The figure 11 and 12 contain some sample code snapshots and the APPENDIX C contains other Code. This is achieved with behaviors such as segmentRouteBehave, which segments the full route, and firsttoDriverBroadcast and newtoDriverBroadcast, each of which broadcasts each segment to available drivers. This enables the agent to schedule several local drivers in series to travel the entire journey. The agent has a self deletion timer (agentdeteTimer) which will terminate the agent after a set time if no matches are found, thereby maximizing efficient use of the resources. This approach provides for the cost effective, flexible and reliable solution to long passenger trips by dynamically allocating drivers on the route.

4.3.1.4 *"TaxiDriver" Agent*

The TaxiDriverAgent class is a multi-functional agent working in a JADE environment, which implements dynamic trip matching, reporting of status, and communicating with passengers and other system agents. This agent is a taxi driver with a status data structure (TaxiStatusDTO) and location in which is populated at runtime by the PassengerTCPListener agent and is populated with data from the Spring Boot server. Core functions of the agent are to match the trip requests to the driver location and preferences, broadcast availability and adjust registration status based on real time taxi mode and occupancy status.

Key Features and Functional Description:

- Agent Registration and Availability Broadcasts:

    - agent initialization handled in setup() method; set attributes with TaxiStatusDTO values from PassengerTCPListener. The agent registers itself with JADE's Directory Facilitator (DF) as a 'taxidriver' service on startup, thus making themselves discoverable by other agents. Agent registers itself with JADE's Directory Facilitator (DF) as a "taxidriver" service, making it discoverable by other agents. The registration with DF() method manages this registration and also triggers "newTaxiAvailableBroadcast" behavior. It broadcasts about the agent's availability to potential passengers and informs the passengers of new taxi options available nearby.

      .

- Trip Matching and Route Filtering:

We have come up with an innovative feature in this taxi app that filters rides based on the drivers desired destinations, home or a preferred store and match them with passengers going to similar destinations. This approach works out to satisfy both passenger and driver need and thus we get a "two birds, one stone" solution. If drivers choose this option from the front end, the app will prioritize these preferred routes for matching. In the absence of a preferred destination, the system reverts to ordinary trip matching, in which the driver's current location is the only criterion considered, yielding flexibility and efficiency throughout the ride sharing experience.

    - The "lookingfoTripCalls" behavior is to evaluate incoming trip requests based on proximity and passenger destinations.. The agent uses the haversine function for distance calculations, to see if a passenger's starting point is close enough (closeradius) and if the passenger's destination also matches the driver's preferred point (e.g. specific home or store area). The "two birds with one stone" solution allows the driver to serve a passenger request and at the same time get closer to a desired destination, (figure 14) hence offering driver satisfaction and passenger service.If driver's heading location is set, a further filtering check is run to ensure that the passenger's destination is also within home-close-radius (a larger radius to account for the driver's errors), but in this case a wider radius to accommodate the driver's benefit. defined proximity (closeradius) and if the passenger's destination aligns with the driver's own preferred endpoint (e.g., a specific home or store area). This "two birds with one stone" approach allows the driver to fulfill a passenger request while conveniently heading toward a desired destination, enhancing both driver satisfaction and passenger service.

- When the driver's heading location is set, an additional filtering check verifies if the passenger's destination aligns with this preference, especially within home-close-radius, a wider radius to accommodate slight deviations for the driver's benefit. The agent sends an acceptance message to the Spring Boot server through Jade2SBAgent if criteria are met, indicating a successful match.

- Dynamic Driver Status and Availability Management:

  - This is DriverStatisUpdates: it cyclically monitors incoming status updates for the driver.. It watches for changes in key fields, such as heading location, current location, taxi status (occupied or available) and service availability. It is this behavior that is critical to keeping the system up to date with the driver's real time availability. If the TaxiStatusDTO indicates that the driver is no longer available or not in taxi mode (taxi status 0), then the agent deregisters from the DF so it will not be matched against any incoming trips during that time. Taxi status (indicating whether occupied or available), and service availability. This behavior is critical for keeping the system up-to-date with the driver's real-time availability.
  - If the TaxiStatusDTO signals that the driver is no longer available or not in taxi mode (taxi status 0), the agent deregisters from the DF to prevent trip matching during this time. However, when the driver becomes available again (i.e., taxi status 1), the agent re-registers, recreating visibility in the network and enabling trip matching.

```
class lookingfoTripCalls extends CyclicBehaviour{


    @Override
    public void action() {

        MessageTemplate tripCallTemplate = MessageTemplate.MatchConversationId("passengerTripCall");
        ACLMessage tripCallMsg = receive(tripCallTemplate);

        try {

            if(tripCallMsg !=null) {
                TaxiRequestDTO taxiCall = (TaxiRequestDTO) tripCallMsg.getContentObject();
                double distance = haversine(driverData.getCurrentLat(), driverData.getCurrentLon(), taxiCall.startLat, taxiCall.startLon);
                if(distance <= closeradius && taxiCall.vehicletype == driverData.getVehicletype()) {

                    //A latitude of 0 and a longitude of 0 (0, 0) points to a location in the Gulf of Guinea, off the coast of West Africa, which is often
                    if(driverData.getHeadingLat() == 0 && driverData.getHeadingLon() == 0) {
                        ACLMessage msg = new ACLMessage(ACLMessage.REQUEST);

                        msg.setConversationId("fromJadetoSB");
                        msg.setContent("DriverAboutMatch");

                        ResDriverMatch driveMatchList = new ResDriverMatch();
                        driveMatchList.setTaxiStatust(driverData);
                        driveMatchList.setTaxiRequest(taxiCall);

                        msg.setContentObject((Serializable) driveMatchList);
                        msg.addReceiver(new AID("Jade2SBAgent", AID.ISLOCALNAME));
                        send(msg);
                    }else {
                        double enddistance = haversine(driverData.getHeadingLat(), driverData.getHeadingLon(), taxiCall.destLat, taxiCall.destLon);
                        if(enddistance <= homecloseradius) {
                            ACLMessage msg = new ACLMessage(ACLMessage.REQUEST);

                            msg.setConversationId("fromJadetoSB");
                            msg.setContent("DriverAboutMatch");

                            ResDriverMatch driveMatchList = new ResDriverMatch();
```

Figure 14:"lookingfoTripCalls" method in TaxiDriver Agetn class

```
class DriverStatisUpdates extends CyclicBehaviour{

    @Override
    public void action() {


        MessageTemplate stausTemplate = MessageTemplate.MatchConversationId("taxistatus");
        ACLMessage taxistatusMsg = receive(stausTemplate);


        try {
            if(taxistatusMsg !=null) {


                TaxiStatusDTO driverStatUpdate =(TaxiStatusDTO) taxistatusMsg.getContentObject();
                var headdingchange = (driverData.getHeadingLat() != driverStatUpdate.getHeadingLat() || driverData.getHeadingLon() !=  driverStatUpdate.getHe
                if(headdingchange) {
                    driverData.setHeadingLat(driverStatUpdate.getHeadingLat());
                    driverData.setHeadingLon(driverStatUpdate.getHeadingLon());
                }

                var currentlocationchange = (driverData.getCurrentLat() != driverStatUpdate.getCurrentLat() || driverData.getCurrentLon() !=  driverStatUpdat
                if(currentlocationchange) {
                    driverData.setCurrentLat(driverStatUpdate.getCurrentLat());
                    driverData.setCurrentLon(driverStatUpdate.getCurrentLon());
                }

                var taxiStasuschange = (driverData.getTaxiStatus() != driverStatUpdate.getTaxiStatus());
                if(taxiStasuschange) {
                    driverData.setTaxiStatus(driverStatUpdate.getTaxiStatus());
                    if(driverData.getTaxiStatus() == 0) {

                        DFService.deregister(myAgent);

                    }else if(driverData.getTaxiStatus() ==1) {
                        registerWithDF();
                        //this.notify();
                    }
```

Figure 15: "DriverStatisUpdates" method in TaxiDriver Agent class

This implementation is of a sophisticated multi agent system where the TaxiDriverAgent is able to adapt to dynamic passenger requests and driver preferences, performing status-based DF registration and deregistration to efficiently use resource and update in real-time. For example, figure 13 and 14 contain some sample code snapshots and other Code content is included in the APPENDIX C. The model not only allows for a seamless trip assignment but optimizes the driver's journey to fit his/her personal route preferences for overall system effectiveness.

4.3.2 MAS Connectivity to the mobile application:

Our mobile application and the Spring Boot server are connected primarily through a Multi Agent System (MAS), using the JADE (Java Agent Development Framework). With this strategic architecture, business logic layer communication is efficient through the use of fixed TCP sockets for real time requests and Hypertext Transfer Protocol (HTTP) for auxiliary interactions. We establish this robust communication framework to enable easy data exchange to improve the user experience as a whole.

The fact that one to one communication is one of the standout features of our MAS implementation is unique. JADE allows individual agents to interact directly with specific users or other agents, in an efficient and personalized way. For instance, when a passenger requests a ride, the agent of that passenger can communicate directly with the driver agent. Real time update, trip negotiation, and tailored responses are all possible due to this direct engagement, increasing user satisfaction and improving an operational flow. From a server-side resource optimization, JADE can be incorporated into the MAS with substantial advantages. This hugely reduces the processing load on the Spring Boot server, allowing the MAS to take control of the essential logic and decision-making processes. This means the server can concentrate on RESTful CRUD

operations, while the MAS takes care of state management, intelligent decision-making algorithms and so on. This division of labor is especially important during times of high demand; it means that the server will continue to be responsive and effective.

Additionally, JADE's agent-oriented architecture allows us to control communication and resource allocation among agents quite efficiently. It keeps each agent independent from each other and each agent makes decision using predefined criteria and interaction with user without continuous involvement of server. The autonomy which results from this can decrease the server-side consumption of resources, since the Spring Boot server can only handle database interactions instead of every operational request itself. Therefore, we can reap server resources and reduce latency, while also improving the overall system performance.

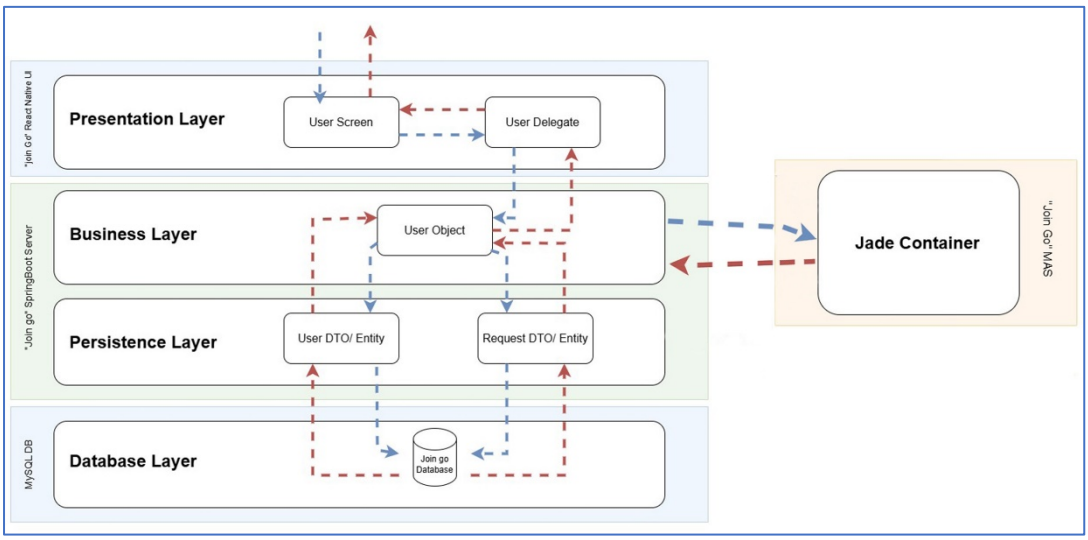


Figure 16:MAS connectivity to the mobile application Architecture.

The integration of JADE within our MAS architecture is in summary, not only improving our mobile application's communication dynamic but also very beneficial for server-side operations. We optimize resource usage and lower the processing load on the Spring Boot server to form a more effective and robust system. The user experience is further enriched by the one-to-one communication capabilities, which allow for direct and effective communication between drivers and passengers, while the server keeps its focus on delivering robust and reliable data services. The MAS and Spring Boot server are ultimately in synergy and result in an optimized and responsive application that will adapt to the changing needs of its users.

4.3.3 Presentation Layer

The presentation layer is the most user interface to the system, allowing users to work successfully with the application. It is made to show information clearly and intuitively

so that all the functionalities are accessible to the user. In our proposed system, which is based on the Join Go taxi app project, the presentation layer is very essential in providing cause for the interactions between the passengers and drivers and the overall experience is seamless.

Mobile application built with React Native is part of the presentation layer of the Join Go project. The way this application is designed, is tailored to meet the needs of both passengers and drivers using user friendly interfaces that enable passengers and drivers to easily navigate and interact with the application. They have various screens: registration, ride requests, trip matching, and real time notices, all of which are built with user experience, accessibility in mind.

For example, passengers can easily input data and select buttons to register, search for rides and view their ride history. They also get real time notifications about their ride status, driver assignment and payment updates. The interface is visually pleasing with Tailwind CSS for styling to keep the layout responsive and consistent across any device.

4.3.4 Business Layer

The business layer of the Join Go application is the heart of what the application does, controlling data processing and providing the means for the presentation and data access layers to interact. The integration of Spring boot server with MAS using JADE framework improves operational efficiency to make decision by decentralized means. Dynamic trip matching and real time communication between passengers and drivers are autonomously performed by intelligent agents which significantly reduce server processing loads. For example, let's say that an agent has to respond to the ride request quickly and can immediately see which driver is nearby based on multiple parameters to ensure a timely and satisfactory match.

4.3.4.1 *Trip Matching Overview*

- Data on User's Current and Future Location –

In application any user has to fill both their home location as well as the location to which they are traveling. This data serves as the basis on which the identification of the possible sharing rides is built thus enabling the system to know the routes for each user and also to look for possibilities of overlaps.

- Calculation of routes-

This mobile application or device retrieves the Google Maps Directions API service to provide route information for each user from their current location to their final destination. This route information includes details such as possible turns, estimated times of arrival and optimal route taken focusing on accuracy of matching trips.

- Check Proximity-

The system checks the starting points of the users to establish their possible proximity. This is matching based on proximity fitness criteria reduces the potential matches by defining the users who can get on the same vehicle from the same point, thereby increasing the speed of the process and decreasing the waiting times.

- Overlap of Destination-

The system also determines any potential route overlaps for the users in consideration. This measure they undertake is to check how much the active routes' endpoints deviate from each other. This step is very important in cases where two or more users have to travel a long distance together.

- User Matching-

As soon as the riders fulfill the proximity and the route destination requirements, the system offers the ride to them as a sharing candidate. Both users are alert when such an offer is made and they are encouraged to facilitate the match. This alert system ensures that the users who have been matched in the system know that they can share the trip, which helps the application in the primary objective of helping in cost reduction of far distance travelling as well as enhancing efficient shared travel.

Additionally, the business layer optimizes the usage of resources by delegating the complex business logic operations to JADE agents and Spring Boot server takes care of the RESTful CRUD operation. This modular architecture also brings in the great advantage of easy scalability and flexibility; new features can be added without reworking much. The business layer provides an efficient communication facility enabled through TCP sockets and HTTP, not only improving the user experience but also helps the Join Go application to adjust in the changing market demand and offer a smooth ride sharing experience to passengers and drivers.

4.3.5 Persistence Layer

In the Join Go application, the persistence layer is a major interface between the mobile application and a database, providing a means for persisting, retrieving, and managing data. This layer is in charge of ensuring the database is consistent and uses it to maintain the integrity of the data against given data operations imperatives. The persistence layer is provided by Spring Data JPA and it abstracts away many types of database interaction, like saving new user registration, updating ride request, getting historical trip data, etc.

The main job of the persistence layer is to implement CRUD (Create, Read, Update, Delete) operations via repository interfaces, so the application can easily talk to the database. Implements things such as transaction management and entity validation to

ensure data gets consistent across all transactions without arising any problems like duplicate entry or data corruption. And it makes the complexities of database interactions abstracted away from the business layer so developers don't have to worry about SQL queries and all the other data management headaches when focusing on core application logic.

The persistence layer not only performs standard data operations but provide additional benefits to optimize the performance of database by doing efficient query execution and indexing strategies. Using caching mechanisms and connection pooling, it speeds up the whole application, making sure that the users get immediate feedback when either interacting with the system. This robust design not only assures data integrity but also eases the application's capacity to scale up in number of users who use it.

4.3.6 Database Layer

A foundational component of the database layer in the Join Go application is a layer that uses Spring Data JPA to define the data model and relationships using a code first approach. If developers annotate Java classes as entities, then a robust structure for managing data can be created through the application code, making for a more streamlined development process and more maintainability. This method also helps the data model and application objects to map in a clear state with respect to different entities in database tables.

The relational database management system MySQL keeps the data intact and the relationships between these entities like users, rides and vehicles. Foreign keys and constraints in MySQL simplify the interactions of related data making complex queries and efficient data retrieval easy. Furthermore, standard procedures and sequences incorporated in the database allow them to faster execute repetitive tasks, e.g., data validation, calculations, making the database less prone to human error. This setup ensures that scalability while the application keeps growing and adding new functionalities – it will always have a consistent and organized database structure. The Join Go application uses Spring Data JPA and MySQL, with standard procedures, and a reliable database layer that will support its operational needs.

4.4 Entity Relation Diagram of System

An Entity Relationship (E-R) diagram is a visual tool that shows the relation between different entities in a system. By understanding the relation and network amongst other pieces of data or information, we can add values for each data set.

The E-R diagram consists of three key components:

- Attribute: The properties and traits of the entity that are defining characteristics.

- Entity: A distinct type of objects such as individuals, entities, locations, abstract concepts etc. represent distinct objects in system.
- Relationship: This element visualizes how two entities are connected, or how they relate to each other.

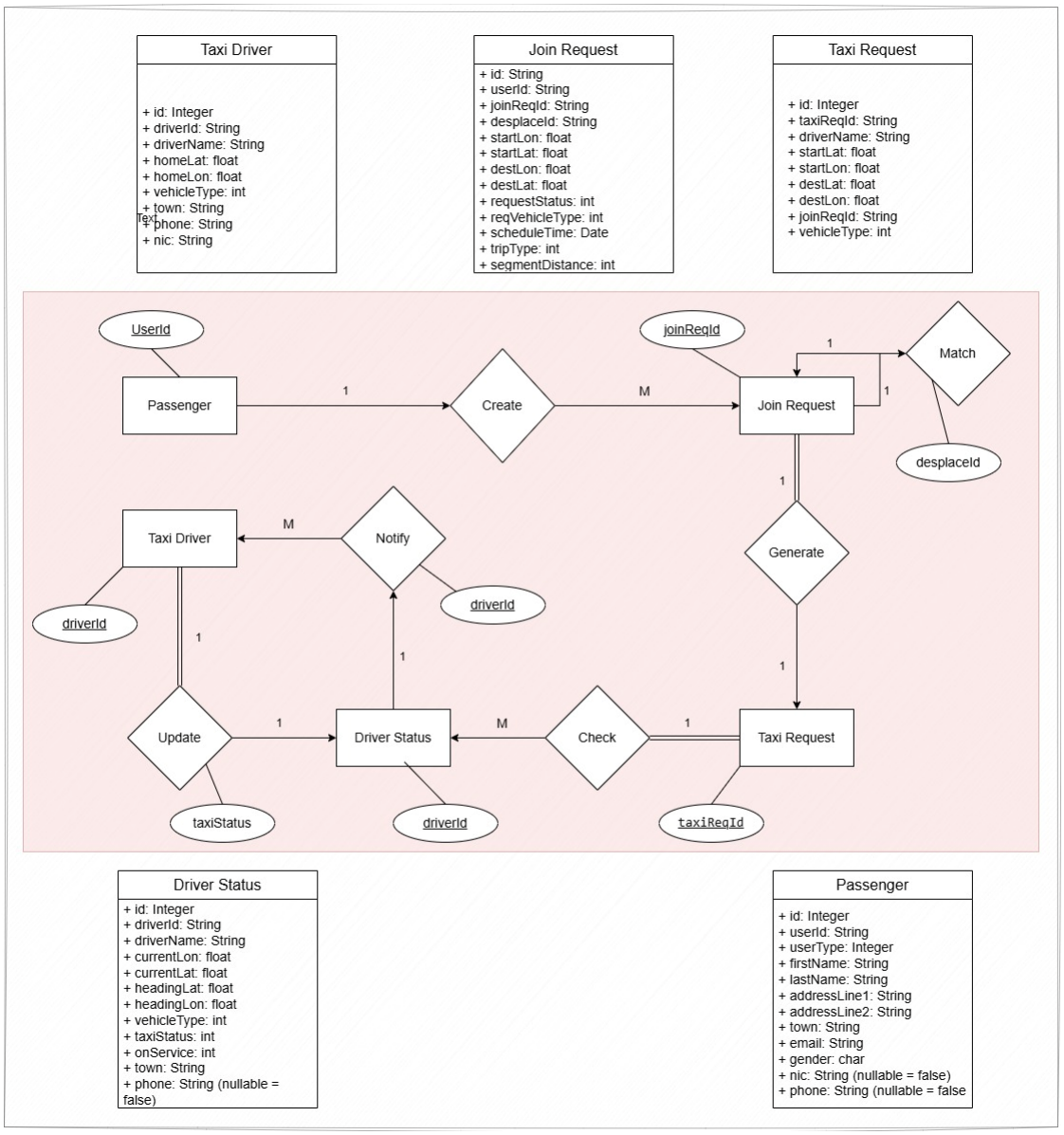


Figure 17:Entity-Relationship Model for the System

4.5 Use-Case Diagram for Application

A use-case diagram provides a structured overview of the system's core functions and interactions, serving as a fundamental tool to understand how the application operates and fulfills user needs. For the Join Go project, this diagram visually represents the primary tasks and relationships that link users — such as drivers and passengers — to the essential features of the system, as well as connections to supporting services like data handling and multi-agent operations.

Not only does this diagram clarify who each actor is and what actions they are able to perform within the system, it also allows the development process to synchronize with the project’s goals. The use case diagram then details the interactions of ride request, trip matching and profile management to ensure that a well coordinated and efficient system is designed. It then provides the basis for the development of an optimized, user

friendly application that fulfills the project requirements, namely, streamlined trip matching, effective communication, and improved driver passenger services.

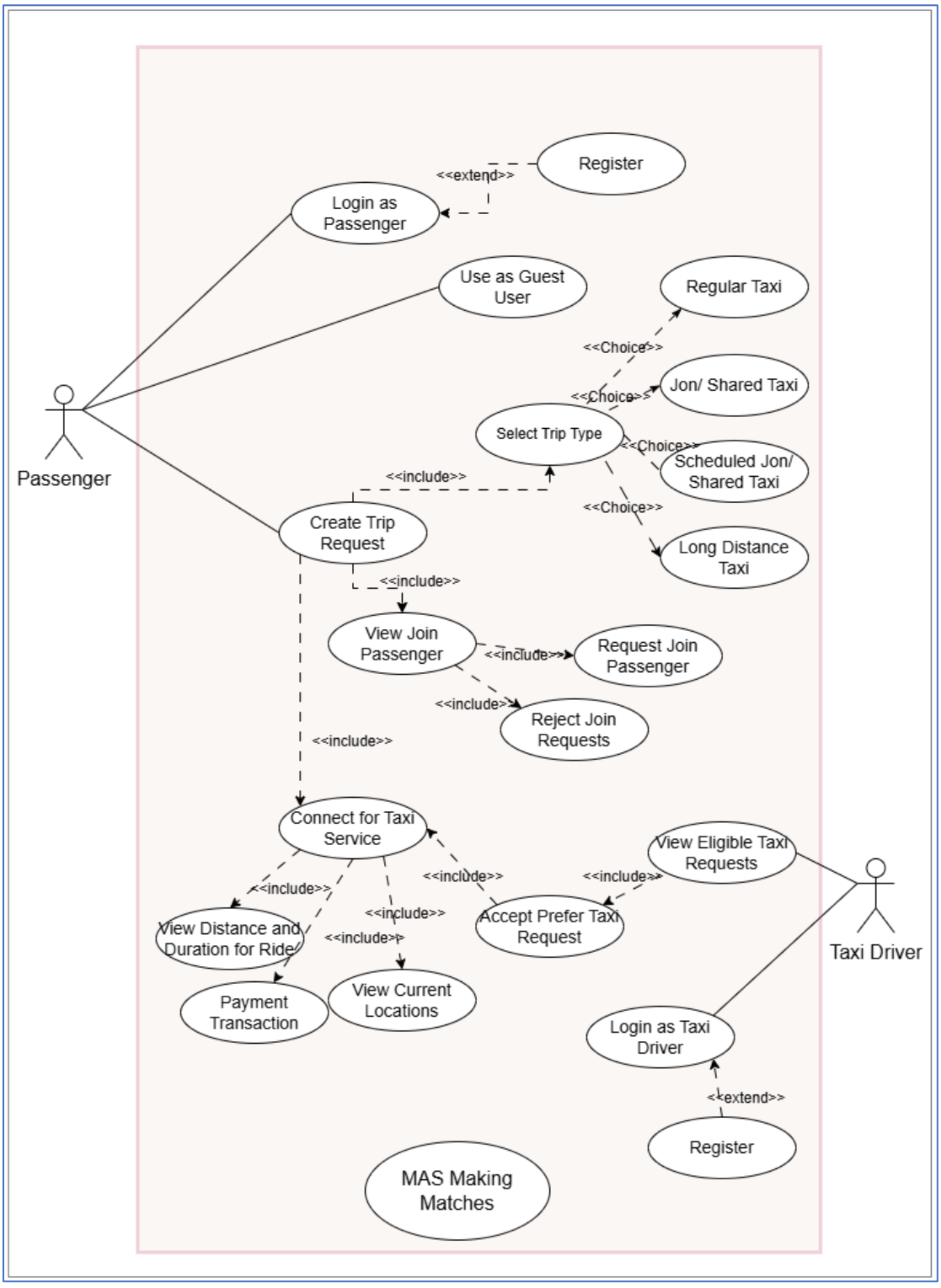


Figure 18:Use-Case Diagram for the Application

4.6 Activity Diagram

An activity diagram is a must have for us to define the sequence and flow of actions within the Join Go application and clarify the step-by-step steps on core functionalities. This diagram captures the workflow from ride requests through trip matching to final trip completion, reflecting the journey each user: what passenger or driver experiences within the app.

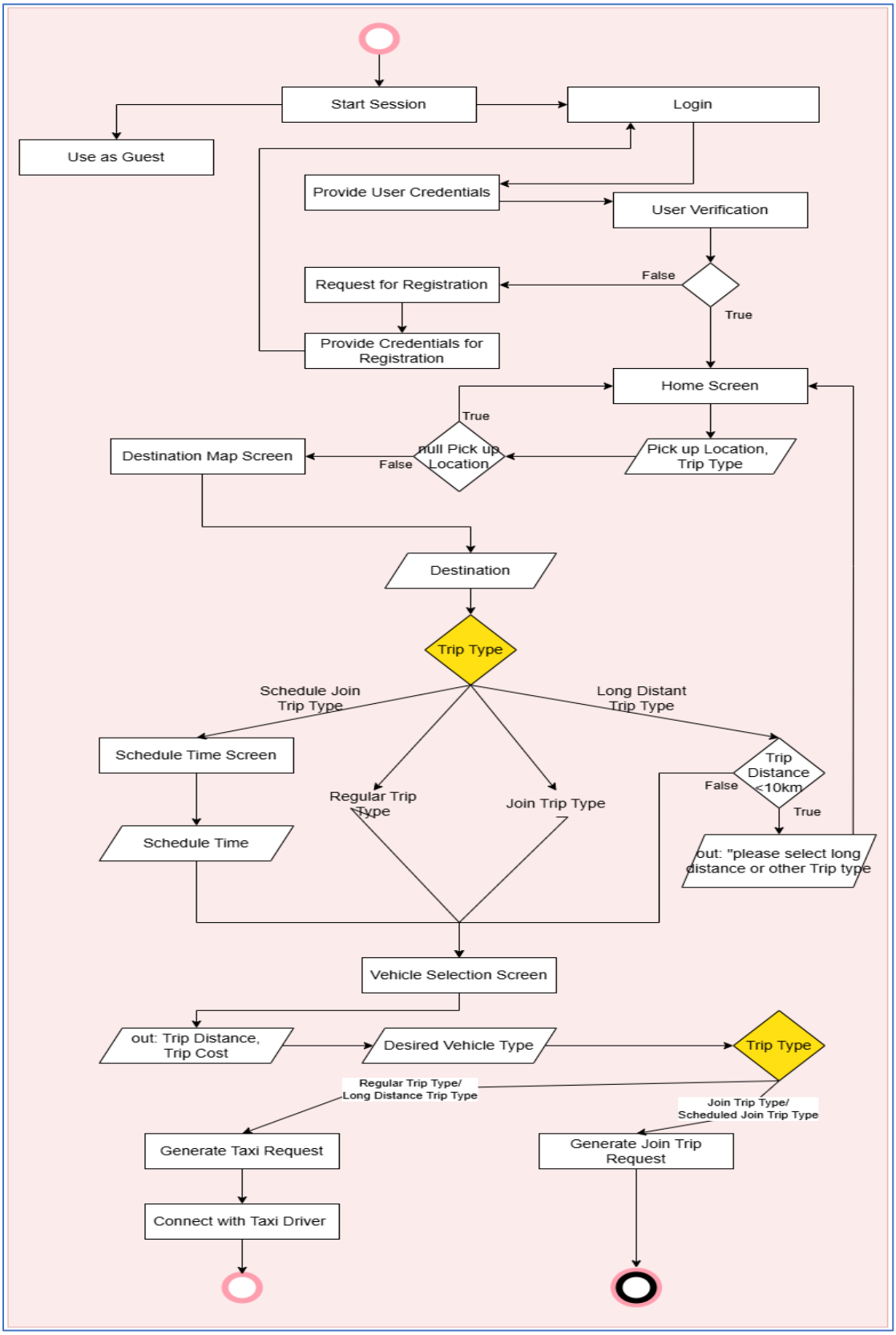


Figure 19: Booking a Trip activity for Passenger

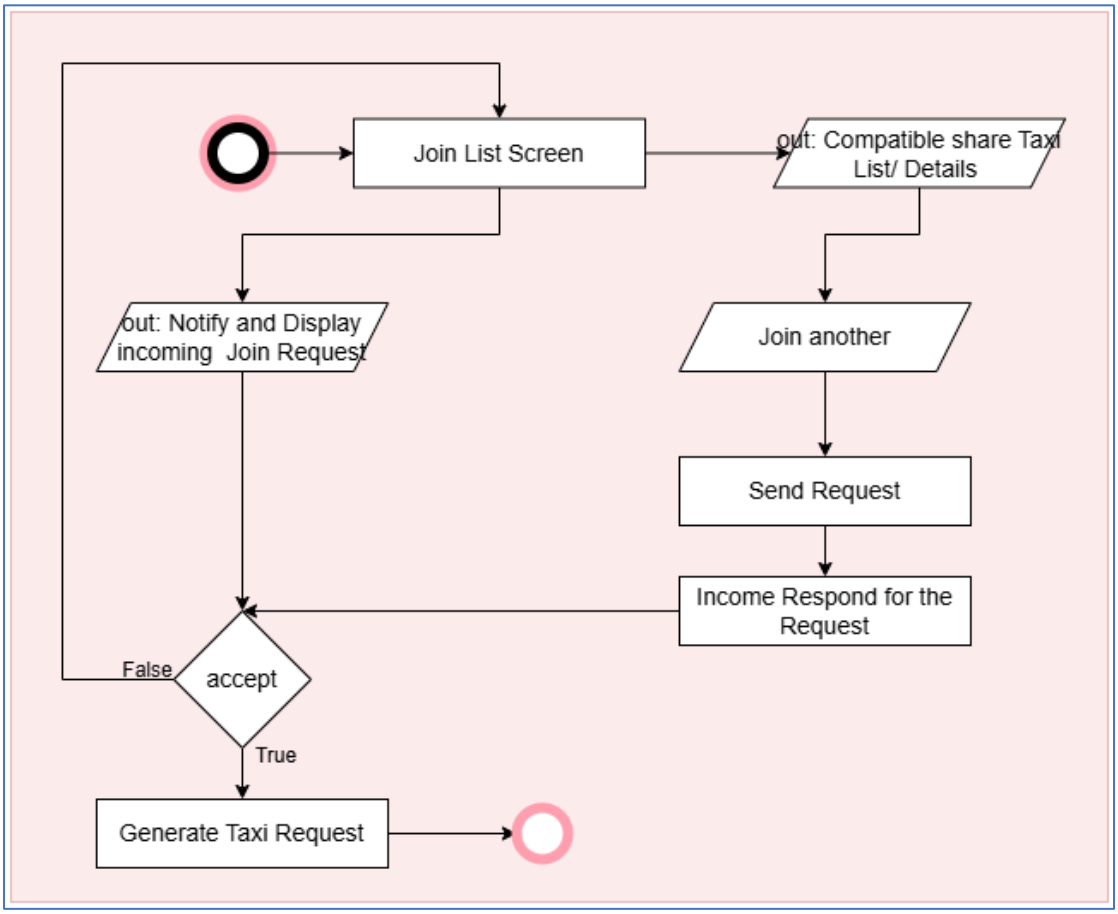


Figure 20:Join/ Share Taxi with another activity for Passenger

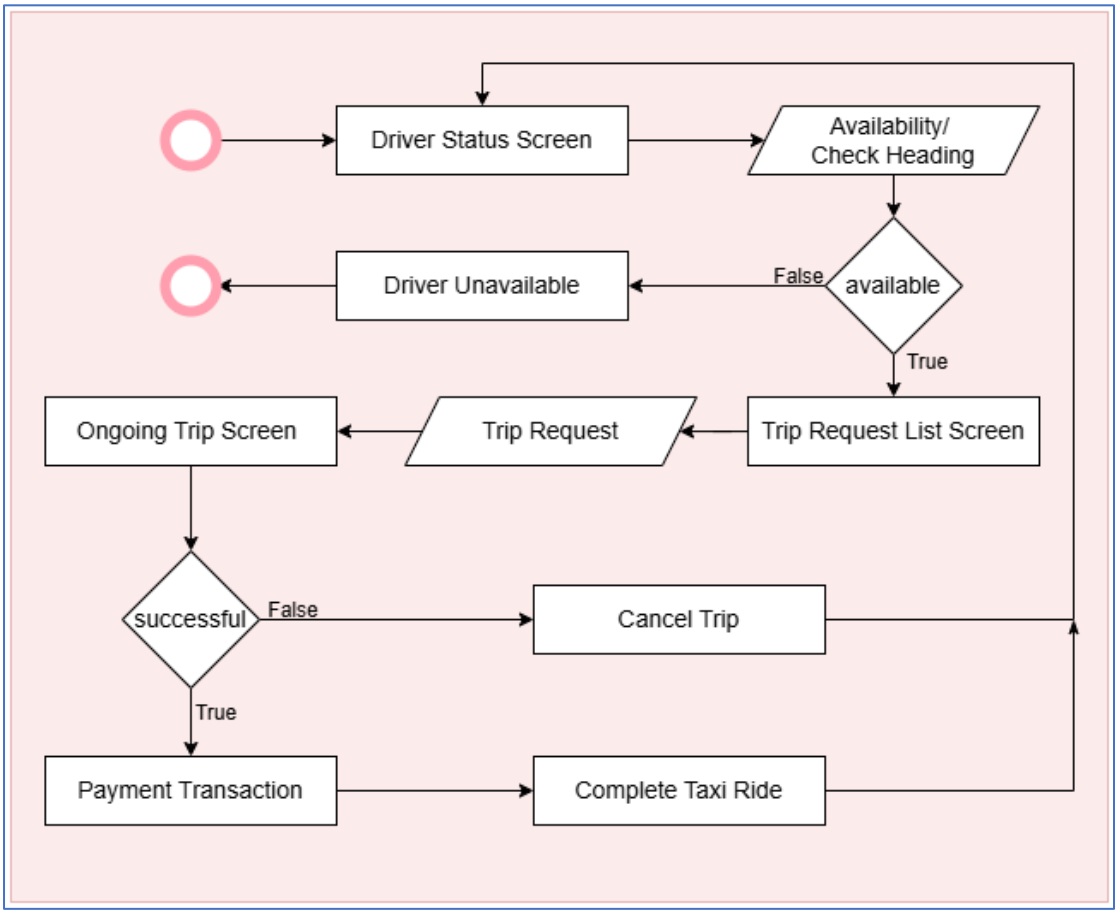


Figure 21:Taxi Ride activity for Driver.

4.7 Development of User Interfaces

Application UI is constructed to give a smooth and user-friendly experience for the user in terms of requesting rides, choosing options, and tracking routes in real time. The UI has an intuitive navigation and clear labels that drastically reduces the learning curve and maximize user satisfaction as well as engagement on app features.

4.7.1 User Interfaces for Passenger User

I. *Login interface for login to system:*

This interface (figure 21) enables users to access the application based on their preferred type: whether they are new registrants, existing logged-in users, or guest users.

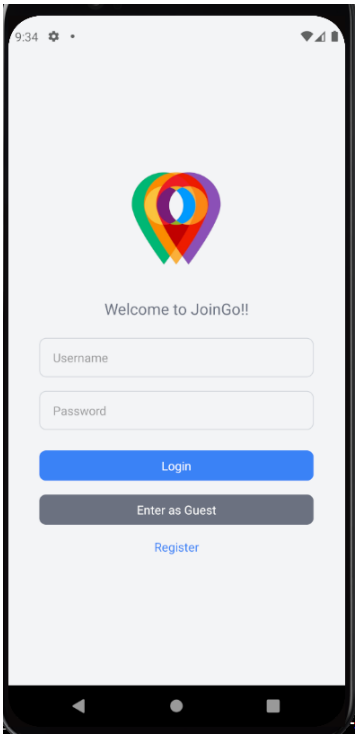


Figure 22Login interface for login to system

II. User Register interface:

The user registration interface is designed to collect user details for creating a user profile within the system, facilitating the authentication of the user in subsequent activities. Interface is set with validations for empty fields (figure 22)

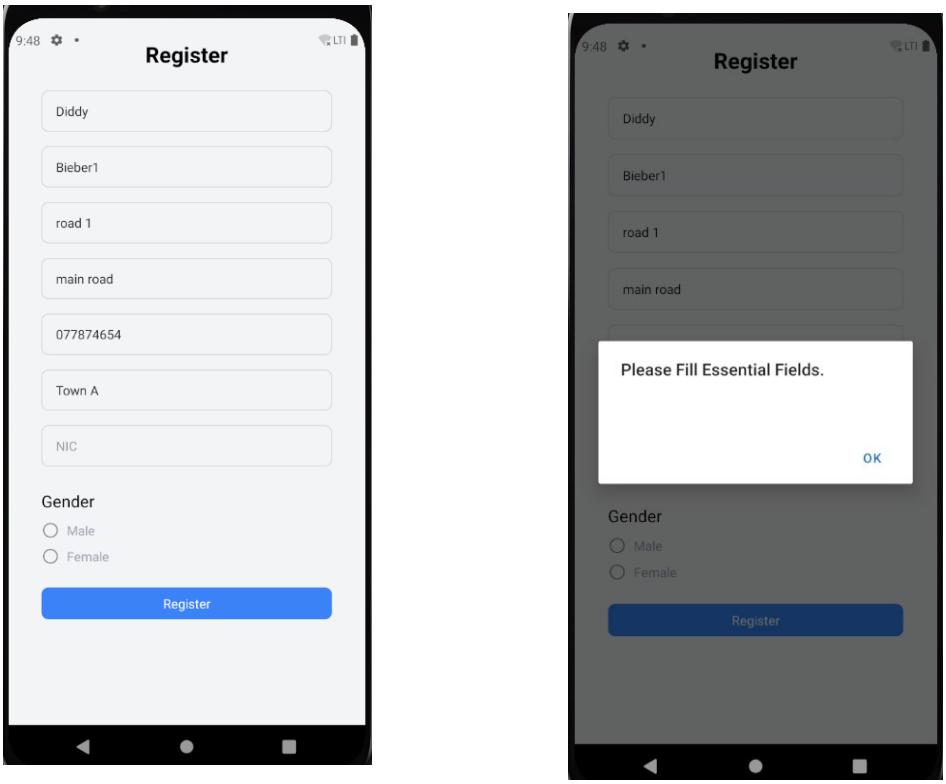


Figure 23:User Register interface

III. Ride Type Selection Interface:

The Ride Type Selection Interface (figure 23) collects the user input of the pickup location and the type of taxi service desired by the user, the type of taxi service that best suits his or her needs. The validation measures included in this interface guarantee that all required fields are filled and thus errors in the next application function.

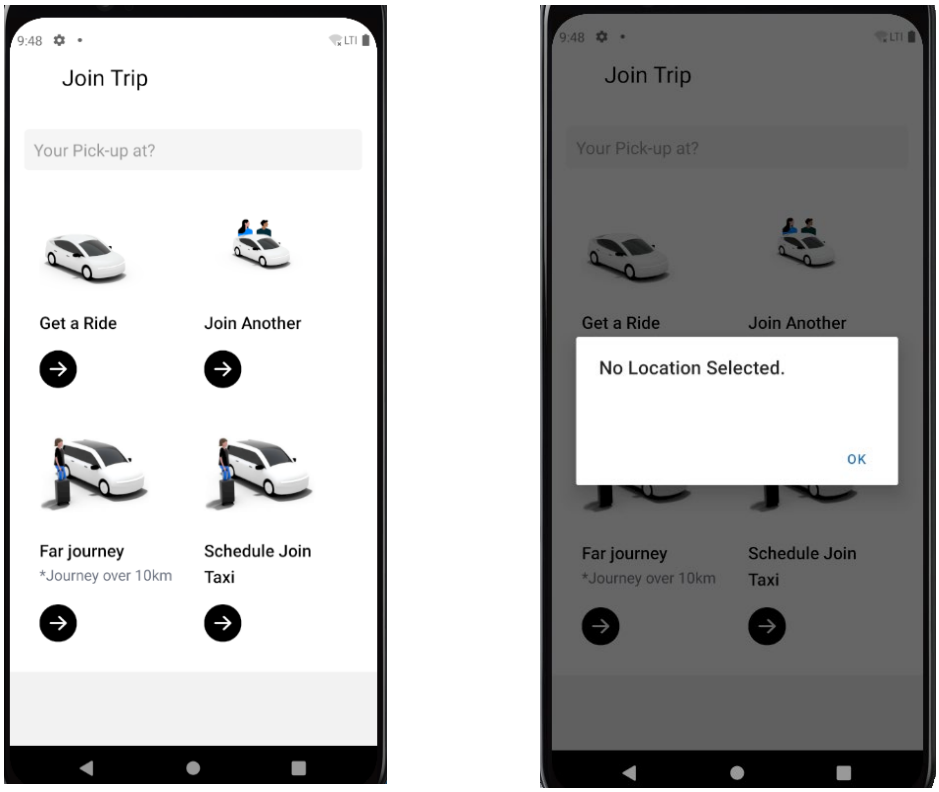


Figure 24:Ride Type Selection Interface

IV. Join Trip Destination Select Map Interface:

The Join Trip Destination Select Map Screen provides trip destination and displays the route from pickup location to destination via map (figure 24). The interface of this includes an auto-suggestion which will auto fill user input to make the process more streamlined.

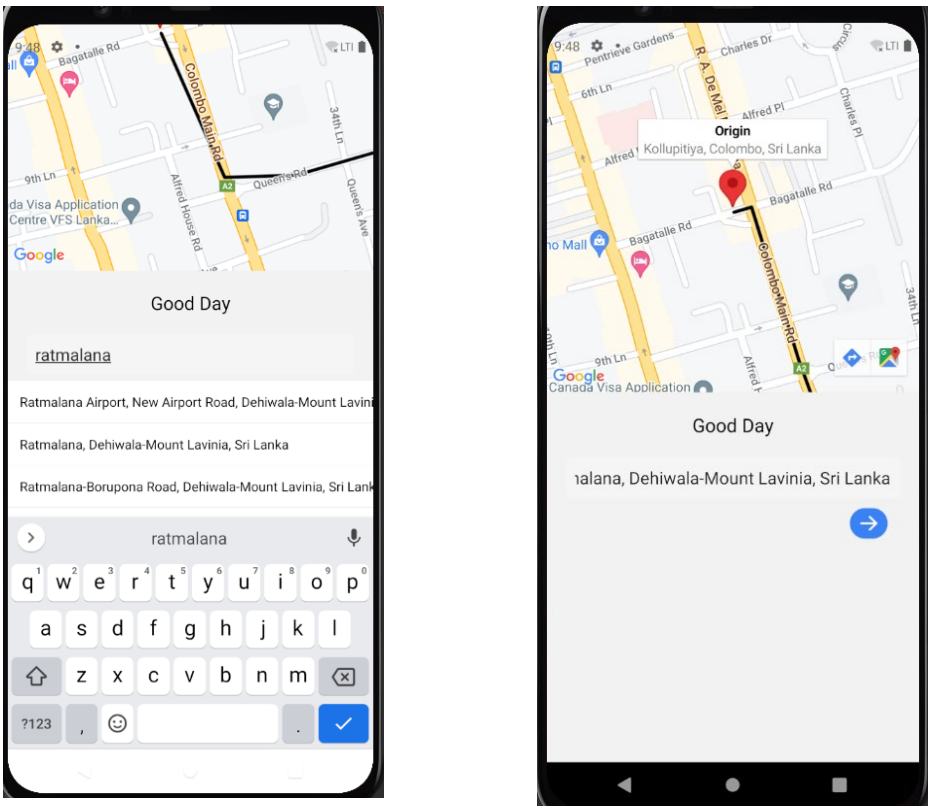


Figure 25:Join Trip Destination Select Map Screen

V. *Long Trip Taxi Destination Select Map Interface:*

Specifically for long distance taxi requests, the Long Trip Taxi Destination Select Map Screen provides a filtered view of trips above a minimum distance policy of 10 km (figure 25). It guarantees users can effortlessly choose and request appropriate long distance journey to make the service more efficient and effective.

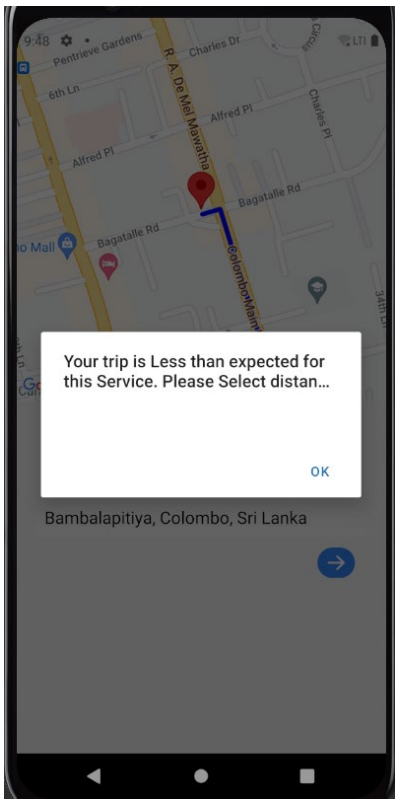


Figure 26:Long Trip Taxi Destination Select Map Screen

VI. Schedule Time for Taxi Interface:

The Schedule Time for Taxi Screen is tailored for users to set a scheduled time for their selected taxi type (Figure 26). This interface allows users to input their desired pickup time, ensuring that their transportation needs are effectively met and enhancing the overall user experience in planning their trips. After Placing the taxi order, final results are viewed for the scheduled taxi.

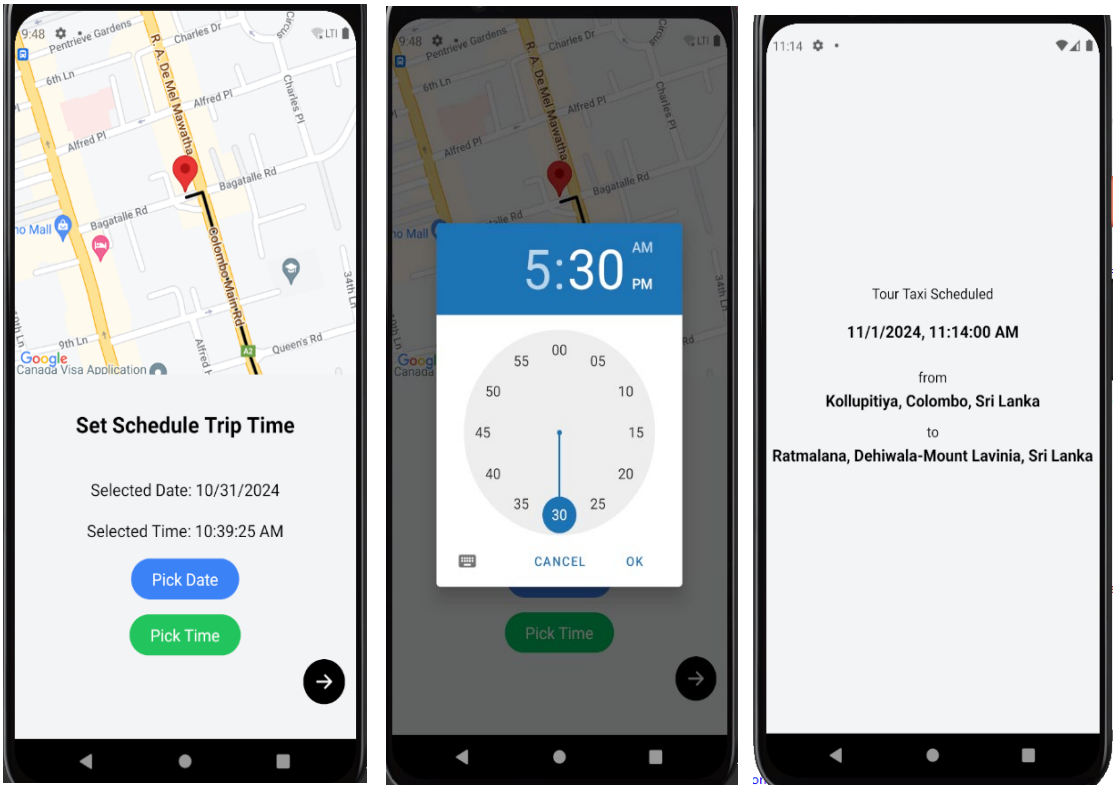


Figure 27:Schedule Time for Taxi Screen

VII. *Taxi Vehicle selection Interface:*

Passengers can select their preferred ride vehicle type on the Taxi Vehicle Selection Screen. It shows important information, including the estimated distance and the taxi service (figure 27). This interface allows users to make a decision before they request a taxi through the application which makes the user experience overall better.

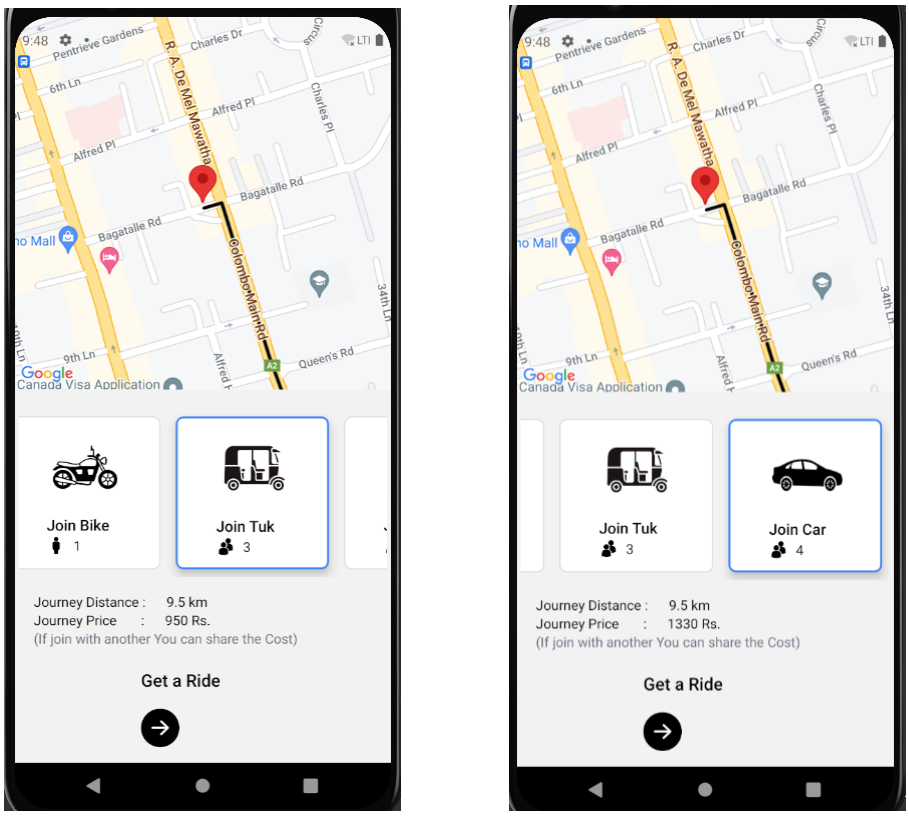


Figure 28:Taxi Vehicle selection Screen

VIII. Join List Interface:

Shared taxis passengers who opted for the join or scheduled trip option are showcased on the Join List Screen. Available passengers seeking co-passengers for shared rides are presented, so users can click to learn more about other people traveling to similar destinations with pickup locations near-by (figure 28). After this, users can make requests or inform potential co passengers to arrange taxi sharing seamlessly.

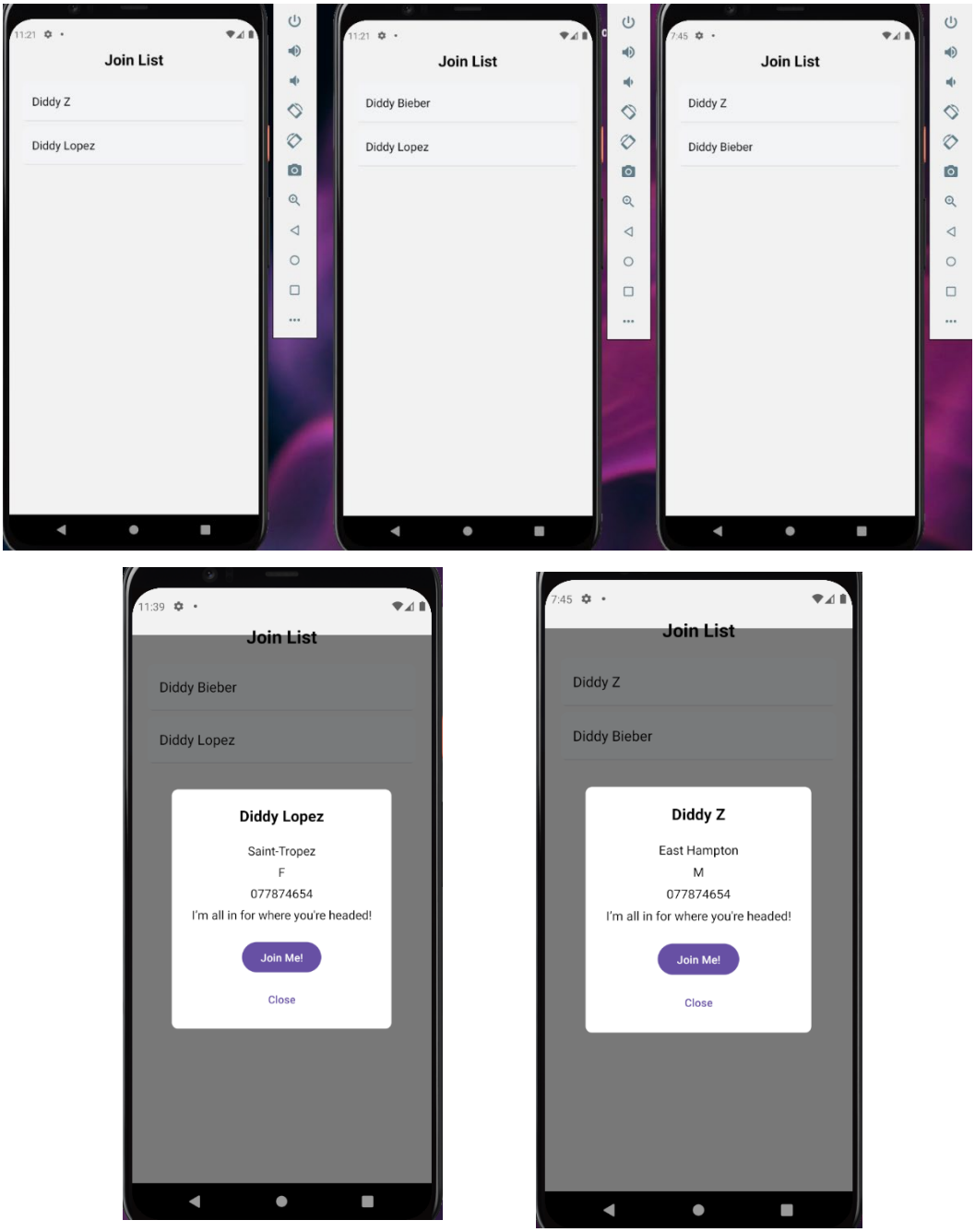


Figure 29:Join List Screen

4.7.2 User Interfaces for Driver User

I. *Login Interface to login to system:*

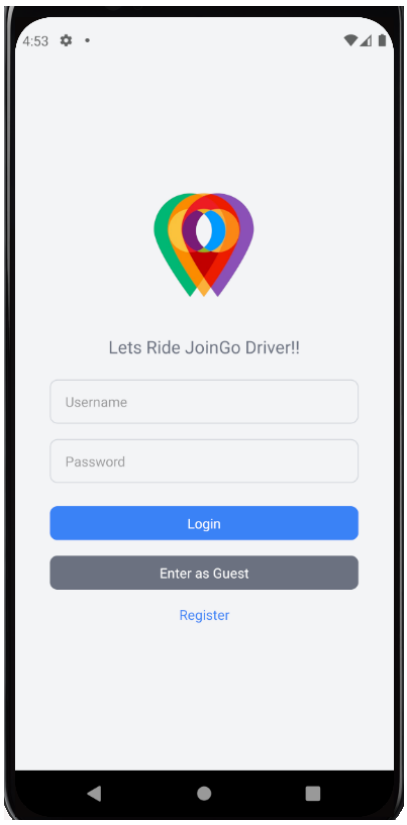


Figure 30:Login Interface for driver system

II. *Register Interface for application:*

The application has a specially designed Register Interface for the taxi drivers. This interface also collects the driver's hometown information as well as creating a user profile for authorization. The data in this is fundamental to a feature which helps drivers respond to passenger requests by finding trips heading towards their home location.

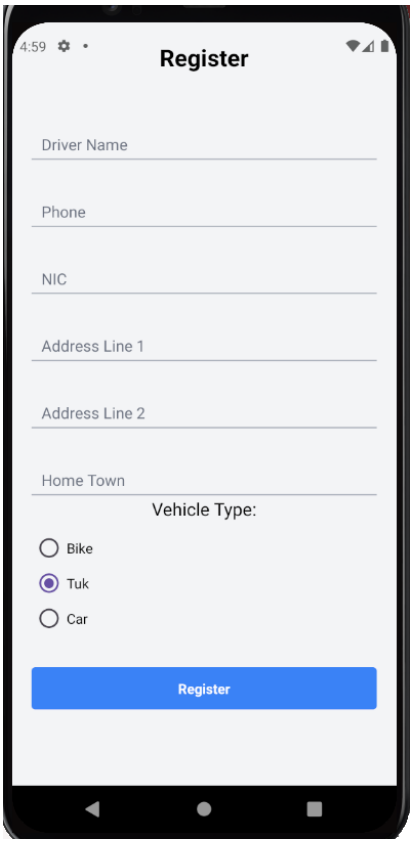


Figure 31:Register Interface for application

III. *Taxi Service start Interface:*

The Taxi Service Start Interface allows taxi drivers to pick up what they need and start the taxi service. This interface also allows drivers to close the service when they want to go offline, so that they will no longer be open for passenger requests via the mobile application.

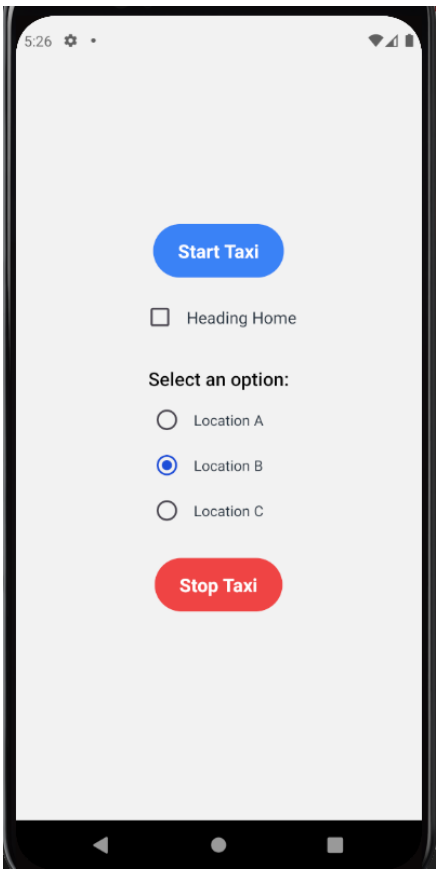


Figure 32:Taxi Service start Interface

IV. *Taxi Request List Interface:*

Taxi request list interface Taxi request list interface is the organized and filtered list of taxi options for the taxi driver based on passenger's request. Each of which can then be tapped by drivers to see more details and choose or accept rides for passenger pickups. Once accepted, the system updates the list in real-time as ride requests are processed on a first-come, first-served basis — meaning that the first driver who accepts the ride request may get that ride dispatch, and no other driver can take it after that. This feature highlights the competitive environment of the application, as drivers need to respond promptly to secure available opportunities.

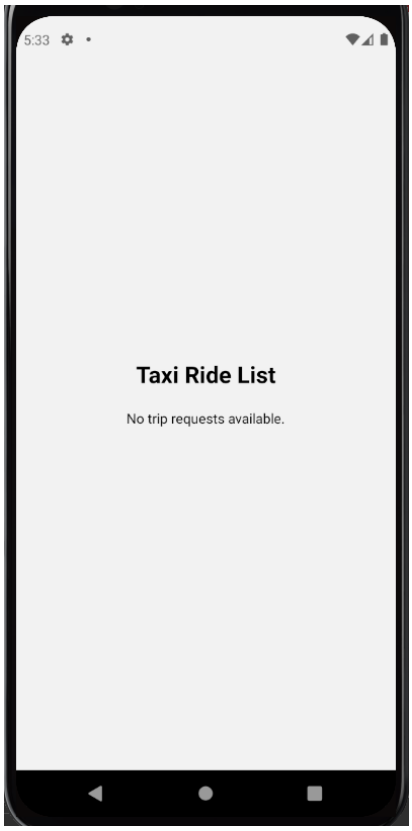


Figure 33:Taxi Request List Interface

4.8 Summary

This chapter explained about taxi application project, its design framework, architectural design of taxi application project, and working components of the project. As discussed, the presentation layer that enables end-users to connect with intuitive UIs for passengers and drivers. The business layer was explained, where application logic is being processed, with the use of MAS and JADE that promote operations with efficiency, avoid delay usage of server resources and increase the performance as a whole.

We also looked at the persistence layer which handles the communication between the mobile app and the database, ensuring data is secure, consistent, and CRUD operations are supported using Spring JPA and MySQL. To help with understanding how users were expected to interact with the system and how processes worked, we presented use-case diagrams and activity diagrams respectively to represent system behavior and workflows.

The overall project will incorporate various system design considerations and techniques with the latest technologies in order to develop a smart taxi service to deliver strong performance and user-friendly experience to the users in Sri Lanka. This strong foundation has been established during the implementation phase with further development and scalability in mind allowing the application to grow and evolve in alignment with a future ever evolving transportation eco system.

.

Chapter 05

Technology

5.1 Introduction

In this chapter, we introduce and review the core technologies chosen for the development and optimization of our project's requirements. The objectives, scalability, and functionality each technology was chosen for were specific. Spring Boot and JADE are chosen at the frontend, Spring Boot for its shared Java foundation that helped reduce development curve and eliminate potential compatibility issues at the same time, and JADE for its already existing integration with the backend logic. The featured integration also brings to bear Spring Boot for CRUD work, database interaction and efficient RESTful communication as well as provide the project's MAS requirements through JADE for dynamic trip matching and request processing.

Spring JPA and MySQL are used to make user info, trip details, and so on persistent. Spring Boot backend connects these databases and persistence tools that will keep the data stored and retrieved efficiently. MySQL's ability to maintain data integrity and provide for complex queries required to match rides and drivers in real time is a result of its structured data management capabilities.

On the front end, React Native and Expo are used to create a cross platform mobile interface, alongside the backend technologies, and the front end is performed through tests on the mobile devices using Android Studio to emulate the mobile devices. Together, these tools make deployment easy and allow maintaining user interfaces that talk to backend processes, resulting in a smooth and efficient user experience. I used Git-hub Desktop for keeping the track on development in time stamps of the development. The system then achieves a high level of optimization in the handling of complex data interactions and automated trip coordination in order to meet the real time demands and scalability needed by the project with these combined technologies (figure 33).

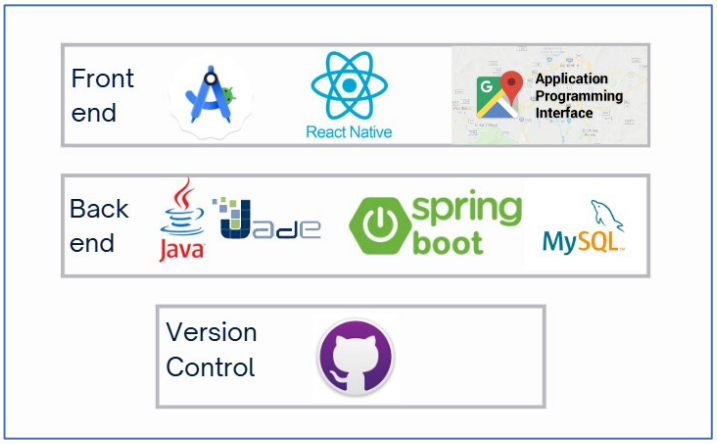


Figure 34:Outline Technologies used in Project

5.2 Research Gap for the project Technologies

In exploring the research gap for the technologies used in this project, a significant challenge emerged: MAS functionality integrated within a mobile application architecture. At present there is very little documentation and practical resources for connecting autonomous agents to mobile applications in a way that supports real time responses and resource sharing. Node.js is frequently utilized as the backend for mobile applications due to its speed and lightness, however, it lacks the structural depth and transaction management needed for complex MAS driven operations, as in this project.

Due to this gap, we opted for Spring Boot as the backend rather than Node.js as the backend, as it offered a more flexible way to handle the requirements of MAS. This however did come with a challenge, the lack of examples of Spring Boot driven mobile applications especially with MAS, having examples of such is scant. As a result, this project was conceived as a novel approach to combining these technologies, where JADE agents run on Spring Boot to solve trip matching, user requests and resource allocation in real time. By using these frameworks in combination, we are able to overcome limitations of typical mobile backends and provide an architecture that supports dynamic, efficient, and autonomous interactions specific to the functional needs of the application.

5.2.1 Outcome of the Research

This research outcome shows that each technology is serving a specific optimized purpose in the application's architecture, presentation layer, business layer, persistence layer, database layer; (figure 34) for each component to work in perfect harmony. The technology placement and interconnections are planned judiciously to form a single system meeting all functional requirements. The application can provide reliable and efficient services to users if the application uses the Strengths of each tool.

RESTful HTTP is used by React Native to communicate with Spring Boot backend services, as a fundamental communication link between the mobile front end and the backend services. The app uses this connection to make user requests and responses dynamically, making the user experience on the mobile interface smooth. Spring Boot is the core backend framework that handles interactions between app and MAS on HTTP and TCP protocols. With this setup, real time data transfer and decision making are possible to enable the role that MAS plays in matching passengers with the most suitable drivers based on location and other requirements.

Also, because of Spring Boot we can connect to MySQL through Spring JPA and use SQL queries which are fluent and easy to use, which also facilitates storing and retrieving data as well as ensuring database consistency. This structure is essential to supporting the app's functionality; we need user profiles, ride requests, and matching records. They're optimized for performance, each connection, a responsive, reliable system that improves user experience while allowing for all operational requirements.

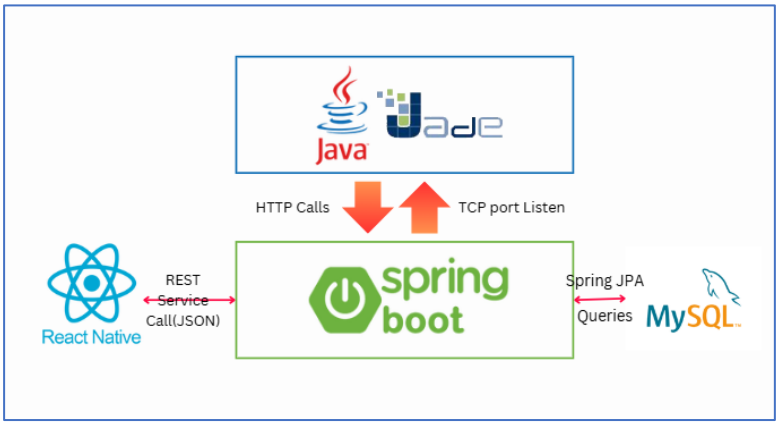


Figure 35:Result of research Communication among Technologies in different layers of the architecture

5.3 Tools and Technologies Used

5.3.1 React native framework (Front-End)

The frontend development of this project is based on the cornerstone framework of React Native, which provides cross platform functionality by allowing one codebase to render the applications on both iOS and Android. The framework is based on JavaScript and React, the latter's component-based architecture that makes it as efficient in developing responsive, user-friendly mobile interfaces. A big reason for selecting React Native over Flutter or Xamarin is its capability to produce native like applications with excellent performance.

Being an open-source async cross platform mobile development, React Native offers a good ecosystem of its own library along with a strong community support, which lends it ability to build most of the UI components as well as integrate other library like Redux

to have a precise state management. For this project, React Native is the best tool to handle real time data interactions and updates that an application should have while being a dynamic taxi service application. Also, the fact that React Native can communicate directly with native modules through bridges, thereby making it seamless to have location services, etc. as a requirement of the project means that the project requirements are achieved with high efficiency.

5.3.1.1 *React Hooks*

React Hooks is an important part of React (and therefore React Native) which allows developers to use state and other React functionalities without writing class components. Locating component's local state management and side effects is done through hooks like useState and useEffect, which is very important in the context of dynamic data interaction in the application. Using Hooks for this project allows for more of a functional programming style of writing code which is cleaner and easier to maintain, especially in a mobile environment where performance is key.

Along with that, callbacks are supported by Hooks for defining functions which can be executed after some operations like screen navigation or data fetching or user interaction. The application is more flexible than before because such flexibility adds to the responsiveness of the application, thereby keeping the smooth experience of the user experience. Not just does it reduce the codebase yet it follows current best practices to develop a responsive and user friendly taxi service application which is flexible to scale and integrate new features.

5.3.2 JavaScript Language (Front-End)

In this project, JavaScript is a fundamental part of the front end of this project as the main programming language that React Native utilizes to make the application use a dynamic and event driven structure. Its synchronous nature makes it easy to interact with APIs, and to handle real time data, which is necessary for a live taxi service application. The versatility of JavaScript helps the project to work with user interactions, update the interfaces responsively and without any issues with React Native's component-based architecture.

On top of that, the ecosystem of JavaScript is very wide, and also has a lot of tools and libraries such as Redux, which, in fact, greatly improve state management. Inclusion of a large-amount of libraries and modules makes the language flexible and extendable, and it works well with all of them, especially when you need to build user center components with responsive feedback and continuous connectivity, like live location tracking. This was done for it's maturity, reactivity, strong alignment with React Native framework and result was optimization of performance and seamless flow of development for your mobile app.

5.3.2.1 *Redux Library*

Real time is imperative for changing user status reports, trip details, and live location tracking, and so redux is used in this project to be an efficient state management tool for maintaining data consistency. Redux centralizes the application's state and helps the project manage frequent data changes across components without sacrificing performance due to the dynamic nature of the values throughout the project. For this project, updates, and API calls, and most importantly, asynchronous actions. Not only does its robust state container provide a reliable, responsive user experience but also can be used to greatly improve debugging and testing; ensuring seamless functionality of this dynamic taxi service application.

5.3.2.2 *Tailwind CSS*

Tailwind CSS is utility first framework means that it provides you with all the styling utilities that you can use in your JSX and you no longer need to create CSS files or even Style Sheet. The ease of use here makes it easier to develop as you can accept or reject changes quickly and see the styling in relation to the structure of the component. Tailwind's utility classes are simple and legible, which helps keep the process of maintaining consistency within the application light on its feet and light on overhead compared to standard CSS methods. Using Tailwind CSS, the project gains rapid development capabilities, a clear design system, the ability to make responsive, visually appealing interfaces without the added complexity of managing large sets of stylesheets.

5.3.2.3 *Google Maps API*

To improve the functionality on location based, the project integrates the Google Maps API. Features like location auto suggestions when user is searching for pickup or destination points makes for a seamless experience when writing in pickup or destination points. The API also provides route generation for the application to display the best paths for both drivers and passengers. It allows users to plan and manage trips efficiently by computing accurate distance between locations thus making informed decisions about their journey. The reason for choosing Google Maps API over other APIs is of course the capability, the documentation and performance reliability, so it is the best choice for the geographical requirements of the application when they are dynamic.

5.3.3 Visual Studio Code (Front-End)

The main development environment for the project is Visual Studio Code (VS Code) by Microsoft.Inc, a very flexible and powerful front end development environment. This light weight nature performs the complexity of react native applications without any lag. VS Code has a big library of plugins that helps with the development

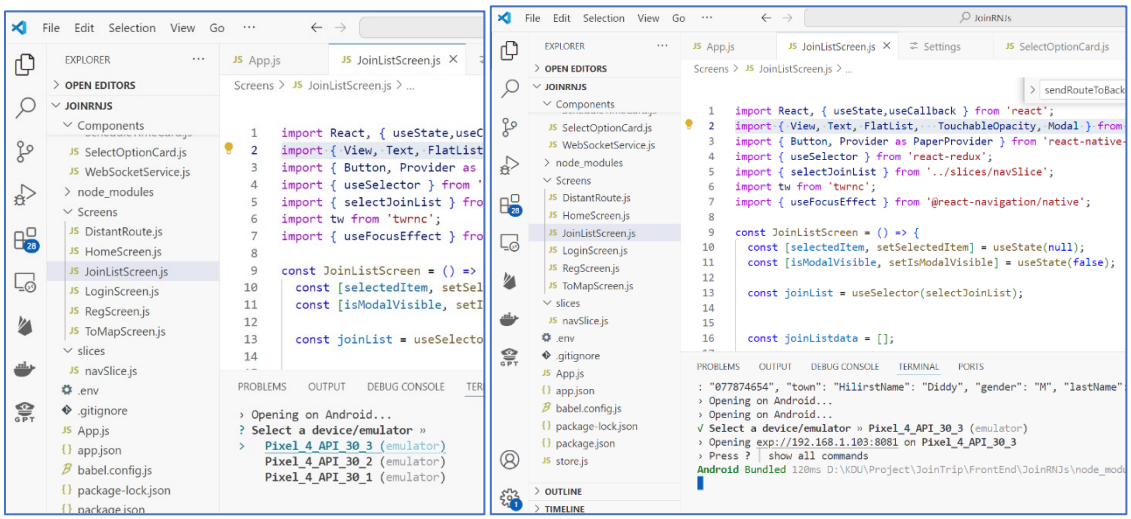


Figure 37: Multiple Instances of mobile Emulators ran on VS Code

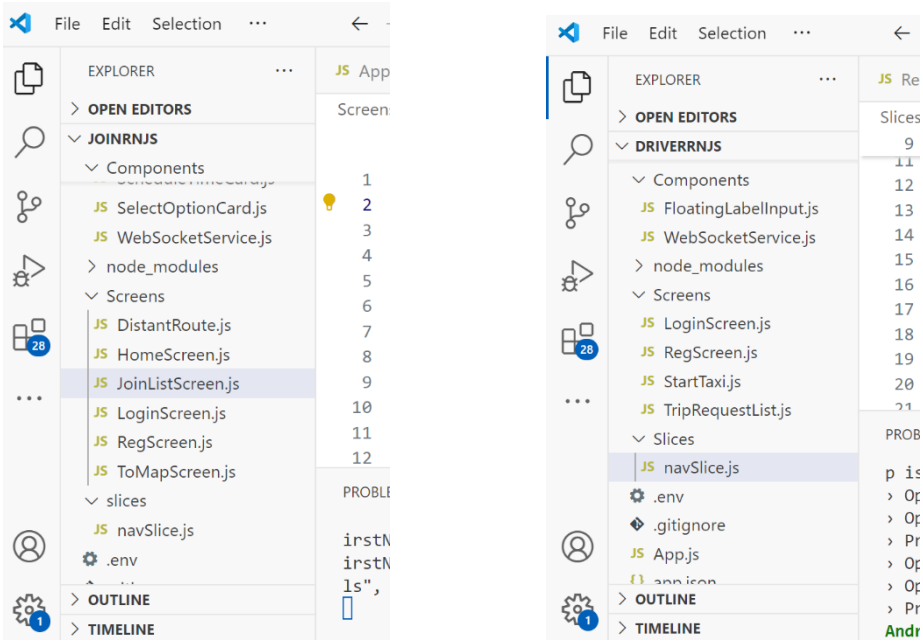


Figure 36:Both Front-end developments done using VS Code

experience by offering language support, syntax highlighting, and code snippets that compliment JavaScript and React Native, making the coding process a lot smoothly. Another great feature is IntelliSense, that smart completes on variables of that type and functions, thus boosting productivity immensely. This project demands the flexibility of VS Code, making it a better choice as an integrated development environment (IDE)

to this project than other different tools because developers can simply build, test, and debug the application with high coding standards.

5.3.4 Spring Boot Framework (Back-End)

The Spring Boot framework plays an integral role in the back end development of the project both from a reliability and robustness point of view it was chosen for its ability to build enterprise level applications. With its convention over configuration approach, development becomes easier with rapid application development and best practices are followed. Spring Boot's extensive ecosystem includes many built in features like dependency injection and embedded servers, all of which help improve modularity and simplify deployment (figure 35). In addition, its community is also very strong and the documentation is thorough — if you are stuck or struggling to optimize your app, there's excellent resources and comprehensive documentation from many other members of the community. Spring Boot is clear and functional thus accelerating development and supporting easy integration with other technologies like MySQL and Jade MAS to meet the requirements of the project. This combination of features makes Spring Boot a strong choice over other back-end frameworks, so that the application is both scalable and maintainable as it grows to meet demands in the future.

5.3.4.1 *Annotations in spring boot*

Annotations in Spring Boot are very important to improve the framework reliability and usability. It gives us a declarative way to configure many components in the application and developers are able to define the behavior and properties in the code. For instance, with @Autowired we don't need to do anything to provide required dependency, this reduces the boilerplate code and helps read the code. @Transactional annotation helps in the management of transactions where a group of operations may be executed with or without success by either completely executing all of them or by rolling back all of them if any of them fails but that is specially used in database interactions to maintain data integrity.

With these annotations, developers can avoid the complex XML configuration for things like request mapping with @RequestMapping, and transaction management without having to write complicated code. In addition to speeding up development, this streamlined approach also reduces the danger of errors in the sense that components work together. With its annotation-based approach, Spring Boot offers higher productivity and a more maintainable codebase than other frameworks that may require heavy configuration or lack a natural setup so it can be a good choice in building robust application in the project.

5.3.4.2 *Rest Controller*

To achieve the architecture of the project, the project relies on the integration of REST Controllers in Spring Boot to develop RESTful web services that allow smooth communication between the frontend and backend components. By using the @RestController annotation, developers can make HTTP requests and responses easy and respond data in JSON format. In addition to helping build an API in a simple way, this streamlined process also promotes a stateless architecture that facilitates scalability and performance by allowing the app to handle different requests separately. Additionally, Spring Boot's REST Controllers include built in error handling and response management so that the application can gracefully respond to an exception and give meaningful feedback to the user. The REST Controller in Spring Boot provides a clear and efficient solution compared to the traditional frameworks that may have complex configurations, or do not have robust API support, which is precisely what the project needs.

5.3.4.3 *Spring JPA (Code First)*

Spring JPA (Java Persistence API) partially fulfill its project's backend architecture by defining the easier way of interaction and managing data. By taking a code first approach, developers define their data models using Java classes that Spring JPA automatically maps to database tables at the start of the server (figure 38). Which allows developers to define their data models using Java classes, which Spring JPA then maps to database tables automatically in the initiation of the server (figure 38). This method makes setup simpler because the database schema is generated automatically from these class definitions. On the other hand, Spring JPA simplifies data manipulation with it's built in methods like save() to add new records and findBy<PropertyName>() to query existing data. Not only does this make implementation easy, it also makes development fast and developers can concentrate more on business logic and not boilerplate code. Its such features make Spring JPA very suitable for the project's requirements since it needs rapid iterations and efficient data management.

```
2024-10-31T11:07:24.182+05:30  INFO 60052 --- [JoinGoREST] [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       :
HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
Hibernate:
    alter table tbl_driver_status
       add column town varchar(255)
Hibernate:
    alter table tbl_taxi_driver
       add column nic varchar(255)
Hibernate:
    alter table tbl_taxi_driver
       add column phone varchar(255)
Hibernate:
    alter table tbl_taxi_driver
       add column town varchar(255)
2024-10-31T11:07:24.471+05:30  INFO 60052 --- [JoinGoREST] [           main] j.LocalContainerEntityManagerFactoryBean :
Initialized JPA EntityManagerFactory for persistence unit 'default'
2024-10-31T11:07:24.855+05:30  INFO 60052 --- [JoinGoREST] [           main] o.s.d.j.r.query.QueryEnhancerFactory     :
Hibernate is in classpath; If applicable, HQL parser will be used.
2024-10-31T11:07:25.091+05:30  WARN 60052 --- [JoinGoREST] [           main] JpaBaseConfiguration$JpaWebConfiguration :
spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Expli
```

Figure 38:Spring Boot JPA initiation and handling of Database

5.3.5 Jade MAS Framework (Back-End)

The back-end architecture of the project is dependent on the MAS framework (which provides the icon feature) to deliver the JADE (Java Agent Development Framework) framework that allows the development of intelligent agents that are able to make autonomous decisions. Because of its design — particularly for applications that need coordination and resource sharing between multiple agents (figure 40) — it is especially well suited for the project's aims of efficient trip matching and real time communication between passengers and drivers. Extending JADE to Spring Boot backend and mobile application is easy due to JADE's inherent flexibility that incorporates easy integration with existing systems. It's this capability that allows the application to be responsive to the user, and also to adapt to changing user needs. Additionally, the support that JADE provides for agent communication protocols makes the application more powerful, allowing agents to work together to deliver a much more streamlined user experience. Using JADE, the project not only satisfies its functional needs but also equips itself with features that give it a competitive advantage by providing the ability to increase user engagement and satisfaction.

5.3.5.1 *ACL Messages*

The JADE MAS framework has ACL messages as a basic communication medium across agents in the system. Through the use of a structured messaging format, ACL allows agents to efficiently and effectively exchange information with one another, so that their interactions are coherent and beneficial. In each message, there is a conversation ID which is the unique identifier of a specific conversation, which makes easy filtering and managing ongoing conversations. The importance of this organization is particularly crucial in multiple topics are being discussed simultaneously which may lead to agent confusion due to the confusion of agents to figure out what topic they should talk about. Additionally, ACL messages can also provide a trigger to agents to perform actions in response to the content and context of the communication. This feature helps the project to be able to give real time updates and interaction in the taxi service application and passengers and drivers get to see those updates and get help timely. The project adopts ACL messages for better communication among the agents, and this improves their competitive position through better responsiveness and user engagement.

5.3.5.2 *Behaviour Libraries*

The JADE MAS framework has behavior libraries, each one containing a collection of predefined behaviors that agents can adapt to execute their tasks in an efficient way, matching the demands of the specific project. For example, One-Shot is a behavior designed for single action tasks which is perfect for a task like responding to a ride request. On the other hand, Cyclic behavior is oriented for the continuous monitoring tasks, in other words, agents can periodically check for updates or changes in their environment so that it is favorable for real time tracking within a taxi service application. With timed execution, Ticker behavior makes it easy to schedule actions, choosing to perform tasks at timed intervals, like refreshing user location data. The

Waker behavior allows agents to pause and resume their activities in response to specific triggers so that they can respond to events as they happen. The inclusion of these diverse behaviors helps make the project more adaptable and responsive, and at the same time ensures that it could respond to the changing needs of users, while staying competitive in the market.

5.3.5.3 *DF Service*

The JADE MAS framework's DF (Directory Facilitator) service is responsible for publishing and discovering agent services, as it were, a 'yellow pages' directory. Agents can register their functionalities and available services as this service provides the ability for other agents in the multi agent system to easily access them. The project achieves this by taking advantage of the DF service to guarantee that all agents can easily locate and interact with the desired services, facilitating cooperative and resource sharing. This functionality is very helpful in a taxi application where agents from drivers, passengers, and spring boot server connector agents need to work together to make the best ride-share matching and to boost application functionality. This structured approach to service registration and discovery not only makes operations simpler but also puts the project on the side of competition that cannot provide the same organized approach, giving the system increased reliability and responsiveness.

5.3.6 Java language (Back-End)

This project is built upon the Java programming language, which provides a solid, consistent, and widely used platform for the back-end development of our scalable applications. Its object-oriented nature provides structure and organization of the complex such as a taxi service application. The strong typing and exhaustive error checking in Java helps build stable and maintainable codebase while keeping stress on high performance and reliability of the project.

On top of that, Java's rich ecosystem contains a variety of libraries and frameworks like Spring Boot and JADE that fit into and boost the features of this project. The ease by which this was integrated is a huge advantage over competitors using less mature languages or frameworks. Furthermore, Java's platform independence further means that an application will be able to run anywhere the Java Virtual Machine (JVM) supports it, making it more flexible to deploy. The strengths of Java are leveraged by the project to address the complex needs of real time data processing and multi agent communication in the context of mobile transport applications, thereby providing a more optimal user experience in the overcrowded market of such applications.

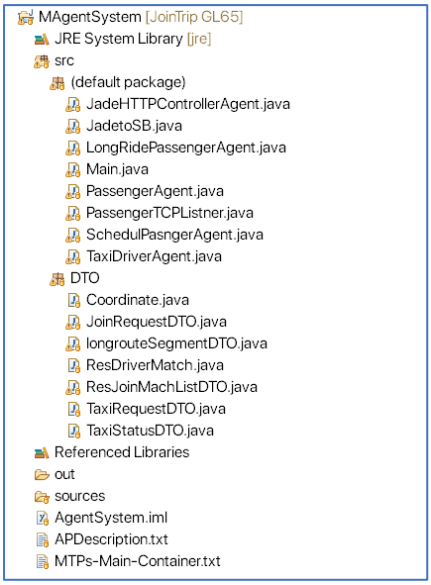


Figure 40: Working tree for Jade framework fort back-end

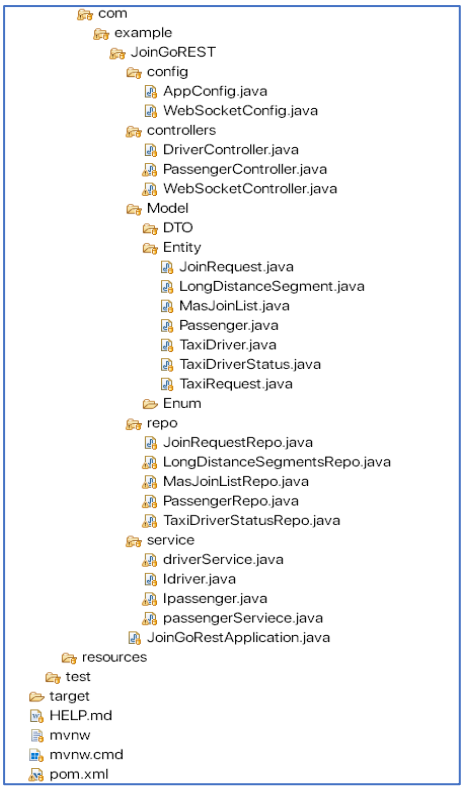


Figure 40: Working tree of Spring Boot framework for Back-end

5.3.6.1 *Object Oriented Programming (OOP)*

This project is built upon the Java programming language, which provides a solid, consistent, and widely used platform for the back-end development of our scalable applications. Its object-oriented nature provides structure and organization of the complex such as a taxi service application. However, it should be mentioned that the proving strength of types, comprehensive error checking guarantees overall stability and maintainability of the codebase which is probably crucial item for a project that has such demands in terms of the performance and reliability.

In addition, Java has a large set of libraries and frameworks like Spring Boot, JADE and others which are integrated with this project to speed up development and add functionality. The ease by which this was integrated is a huge advantage over competitors using less mature languages or frameworks. It also has platform independence so the application works on any platform that has a Java Virtual Machine (JVM). The strengths of Java are leveraged by the project to address the complex needs of real time data processing and multi agent communication in the context of mobile transport applications, thereby providing a more optimal user experience in the overcrowded market of such applications.

5.3.6.2 *gson library*

A Real Time Communication based taxi service application has to interface between Javabased objects and JSON text, which is supplied by the gson library. When it comes to communication through TCP where only strings can be sent, complex Java objects can be converted into JSON data for easier transport of data over the network for example ride requests and driver updates. Its simple and clean API helps make serialization and deserialization uncomplicated for there is no need to do manual parsing of the incoming JSON string back to Java objects. It is for efficiency concerns that GSON is also preferred over complex backend technology for application development which helps optimize an application's performance and fulfill the project need of fast data transfer.

5.3.7 Eclipse Java IDE (Back-End)

Eclipse Java IDE is one of the critical tools employed in the backend development of this project since it provides a Java development appropriate environment. Its rich set of utilities, such as code completion, syntax highlighting, presence of debugging tools among others, helps in increasing the efficiency of the users and ease the process of coding. The IDE accommodates different types of plugins which can be modified for each project such as Spring Boot and JADE for easy technology integration. On top of that, one of the notable merits of Eclipse is that large projects are manageable which enable the developers to work on projects with complicated codes without so much stamping around. Also, the enormous active support what we call the community support is yet another resource for the developers as they assist in troubleshooting and provision of other related information. In a nutshell, the reasons why this software is better than other IDEs is mainly because it comes equipped with numerous features, wide range of functionality, and high-level support for Java which meets the various requirements of this taxi service application.

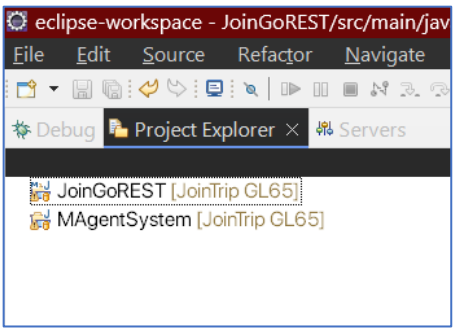


Figure 41:Both Java projects Develop in eclipse IDE

5.3.8 MySql (Database)

In this application, we selected MySQL as our relational database management system because it offers supports in terms of data integrity, assessment and mapping of different entities within the system. The usage of SQL, which is the standard programming language for databases, for example, in this project would give the

warranty that these components are interacting in a reliable manner. It is also worth noting that Mysql's advanced querying features allow the app to effectively and quickly keep track of, look up and organize data relationships which is very crucial because of the nature of a taxi service application. Based on the above facts, relational data management can become a limitation in applications that do not employ SQL – MySQL manages to provide the necessary tools for maintaining appropriate relationships between various entities. This is very important because it ensures that all the information regarding the passenger, the driver and the trip are well knitted together. Therefore, it is for these reasons Mysql is the most suitable database offering as it meets the needed database performance and scalability concerns of the application as well as the relational needs for the managed data.

5.4 Summary

To wrap up, this section dealt with various applications, tools, and technologies used to build the project, and their properties as well as their contributions to the intended target of the application. In this way, the project incorporates such frameworks as React Native and Spring Boot, as well as libraries like Redux and Tailwind CSS, thus establishing a solid and responsive front end without compromising on backend efficiency. Moreover, the usage of the JADE MAS framework enables agent communication and management which helps in service provision of the taxi application. Also, the selection of MySQL database system emphasizes the necessity of maintaining relations between entities by managing structured data. All these serve to not only enhance the functionality of the application but also to make it useful and marketable in fulfilling specific user and business needs.

Chapter 06

System Testing and Implementation

6.1 Introduction

You can't overestimate the necessity of system testing and implementation in completing any software application, especially for this project which is agile and user-friendly. In this chapter, we want to underline the necessity of confirming that the system in question performs as expected all the time and for all its purposes. If there are any particular features required of the project, effective system testing will pinpoint what challenges are posed and make it possible to tackle the problems before the end users suffer. For this purpose, the project combines the use of automated tests from the frameworks taken and manual tests to fully check the functionality, performance and usability of the designed system. Such combination of approaches promotes a thorough testing phase which further elevates the quality of the project and makes it unique as the focus and care on the quality of work is noted even in the stages of implementation.

6.2 Testing Objectives and Criteria

6.2.1 Functionality Testing

Functional testing is a crucial part of ensuring the accuracy and performance of essential functions particularly concerning the accuracy of the trip matching process. This testing methodology guarantees that a summary of trips is given to requests for passengers who fit various parameters of their request including but not limited to time, place, and carpooling options. Functionality testing also includes the process of assaying the data communication and data flow between the application back end and front-end layers.

For example, the project asserts the reliability of data transmission by employing the various mechanisms of entering, retrieving and changing any data which rules out the chances of data being lost or damaged. In a similar vein, testing subsumes the systems of business logic especially with regard to checking the validity of computations, transformations as well as filtering of data. This much focused testing verifies that only the suitable business rules in accordance with how trips are to be matched and managed are loaded and that the results are processed and dispatched properly.

6.2.2 Usability Testing

Usability testing is another significant activity in this project since it entails assurance of ease of use and the overall impression of the application interface. This activity is designed to test the interaction between the users and the application for optimum ease of use.

Based on the elements of the interface and the process involved in completing a trip-booking, usability testing goes a step further to ensure that users are able to complete each of the trip booking stages and the correctness of the prompts and buttons visible, the presence of buttons and links and the order of trip-booking. It emphasizes on correct prompt when requires inputs and it ensures that the validating message should be appropriate and easy to interpret so that users can correct the errors made in their inputs quickly. It can therefore be said to improve the user experience by reducing the chances of the user facing problems that may lead to dissatisfaction and therefore it serves purposes of why this platform is very good compared to the other similar ones that are already present in the facilities since it has been designed to meet the demands of a typical traveler.

6.2.3 Reliability and Resilience Testing

Performance and endurance evaluation are paramount in assessing how an application is able to maintain high performance levels within given usage time frames. This assists in examining load balancing strategies in which the aim is to ensure equal distribution of requests to web applications so that resources are optimized and server pressure kept in check. In addition, the responses that the system generates concerning the database are also assessed so as to maintain optimal response under extreme levels of load. The employment of reusable tests enhances the confidence in test results and makes it possible for evolution of the application in the future without resource contention. All these comprise the means for ascertaining that the project is practical and an economically viable option within the ever-demanding market environment of today perseverance of the users' needs.

6.3 Test Cases and Execution

6.3.1 Unit Test

Testing individual software units is an important process that allows for the testing of both the backend and frontend of the application. In the backend, which is usually in Spring Boot, the unit tests will especially be done on methods and endpoints that can stand on their own in their given roles. This means that each of the units is subjected to several variations from the outside and tests whether it is working appropriately. In most cases, this testing is conducted at the level of each separate method, i.e. on the method attempts to predict what type of data will come in and what should be returned.

This step is very important in any system in order to avoid cluttering of the system with unorganized data and to ensure the right order of operations is followed by the system.

The other case is on the unit tests on the frontend where the focus on the components is to ensure that they either do not break or works correctly when user gives certain parameters, especially types them in. Devices like Jest or Enzyme allow developers to incorporate test cases where user action is needed and confirm that the components do not only show correct information but also perform actions, wait for an action and react accordingly. Unit tests are done both in the back stage and front stage during the development process so that any error codes can be determined early enough and before the final version is reached in order to minimize complications during the actual running phase. Lastly, the use of extensive unit testing in the project is consistent with the quality expectations of the project's components and thus makes it unique from its competitors' features in terms of performance and usability.

6.3.2 Integration Testing

In the given project Integration testing mainly focuses on the incorporation of internal parts within the application and how they function as a whole, particularly the interaction between the backend Spring Boot application and the MAS. Given that the agents in the MAS make decisions, and coordinate matching trips to the users, the objectives of integration tests are to ensure that the integration of agents facilitates the data passage effectively. These tests also ensure that the requests sent from the backend to the MAS are addressed properly and returned with the expected result, so that the behaviour of the system can be verified in different conditions of usage and stress levels. This approach encourages the architecture of the system's backend to be more flexible while adjusting according to the activities of agents-based processes, which leads to better reliability.

Moreover, developing the WebSocket communication is also one of the key parts of integration of back-end services and the client interface. When integration testing WebSocket channels, we pay attention to the timely update of current status notifications, trip information, and any changes for passengers and drivers with the most appropriate response time. Users will also be able to appreciate this functionality by using the appropriate tools where the tests will be directed to check the filling of the frontend part with the content provided by the backend. It is expected that this approach will allow the user to be able to get any information they require instantly rather than having to refresh the system which is quite disappointing to the users and this is an edge over most systems especially those that do not allow interaction with the users in real time.

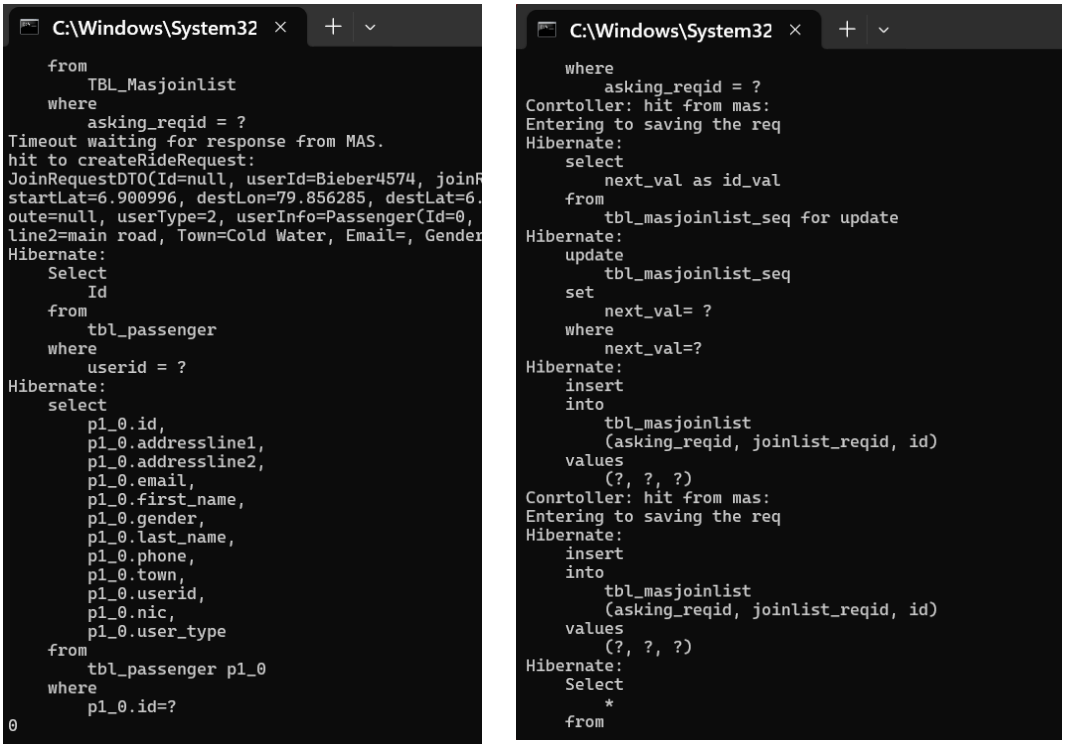


Figure 42:Monitoring Integration when applivation is Live

Finally, on the backend, interactions with the database are further checked through integration tests emphasizing the need for consistency and correctness of data across many interactions. This type of testing is performed on the system with an emphasis on storage, retrieval, and updating of information in order to validate that the system can sustain the load of access to the database without conflicts between transactions. For example, managing transactions accurately during peak traffic times prevents the loss of information, such as booking requests by passengers or drivers, information about completed trips, and payment records (figure 42). The testing framework of the project ensures that the backend performs optimally in regards to database performance even when the number of users is high, thus enhancing system performance and effective use of resources in order to meet the user's need for quick and interactive services.

6.3.3 Integrated Test class in Spring Boot

```java
package com.example.JoinGoREST;

import org.junit.jupiter.api.Test;

@SpringBootTest
class JoinGoRestApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

Figure 43: Built-in Test Class for the application given by Spring Boot

Within the Spring Boot framework, the integrated test class allows constructing, testing, and validating the project with the correct configuration of database connectivity, pinpointing issues such as circular dependency injection. This method of testing permits evaluation of the server's operational capability without switching on the live server - a very useful cost-cutting measure in development and an enhancement of overall operational efficiency. This is entirely limited to testing that can be accessed using command “mvn install” (figure 43).

```
D:\KDU\Project\JoinTrip\BackEnd\JoinGoREST\JoinGoREST>mvn install
[INFO] Scanning for projects...
[INFO]
[INFO] -----------------------< com.example:JoinGoREST >-----------------------
[INFO] Building JoinGoREST 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ JoinGoREST ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ JoinGoREST ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ JoinGoREST ---
[INFO] skip non existing resourceDirectory D:\KDU\Project\JoinTrip\BackEnd\JoinGoREST\JoinGoREST\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ JoinGoREST ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ JoinGoREST ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.JoinGoREST.JoinGoRestApplicationTests
15:13:24.860 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils -- Could not detect default configuration classes for test class [com.example.JoinGo
T.JoinGoRestApplicationTests]: JoinGoRestApplicationTests does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
15:13:24.276 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper -- Found @SpringBootConfiguration com.example.JoinGoREST.JoinGoRestApplication for test
ss com.example.JoinGoREST.JoinGoRestApplicationTests

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.2.4)

2024-11-01T15:13:24.752+05:30  INFO 26252 --- [JoinGoREST] [           main] c.e.J.JoinGoRestApplicationTests         : Starting JoinGoRestApplicationTests using Java 17 with PID 26252
arted by AD. MSI in D:\KDU\Project\JoinTrip\BackEnd\JoinGoREST\JoinGoREST)
2024-11-01T15:13:24.754+05:30  INFO 26252 --- [JoinGoREST] [           main] c.e.J.JoinGoRestApplicationTests         : No active profile set, falling back to 1 default profile: "defaul
2024-11-01T15:13:26.656+05:30  INFO 26252 --- [JoinGoREST] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2024-11-01T15:13:26.758+05:30  INFO 26252 --- [JoinGoREST] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 85 ms. Found 5 JPA re
itory interfaces.
2024-11-01T15:13:27.526+05:30  INFO 26252 --- [JoinGoREST] [           main] o.hibernate.jpa.internal.util.LogHelper  : HHH000204: Processing PersistenceUnitInfo [name: default]
```

Figure 44: Application Spring Boot Test class execute.

6.3.4 End-to-End Tests

In this project, End-to-End (E2E) testing incorporates the whole therapy; the actions of the user are emulated at different stages of usage of the system so that complete satisfaction is achieved. This testing strategy involves the process of registration, which

includes user profile creation and checks whether the system's registration process, profile creation is working smoothly and effective details are captured without any hitches. The test cases evaluate both double-checking the shown section and the particular person's interference have linguistic input for the confirmation of closing profile creation. This is done to aid capture of new users into the system effectively since creating a profile is not a hard task.

Another important functional area that is tested is the location selection as well as creating a trip specified with pick-up and drop-off locations. E2E tests ensure that location input is both correctly interpreted and presented within the application such that the user is able to easily pick it or change it. This includes the functionality whereby the system is able to distinguish between different types of rides that a user's input may suggest, for example if the User is trying to create a trip which falls under shared, scheduled or long-trip rides (figure 44). These tests make sure that the design of the interface remains simple and usable at all times thereby minimizing user mistakes and improving overall usability.

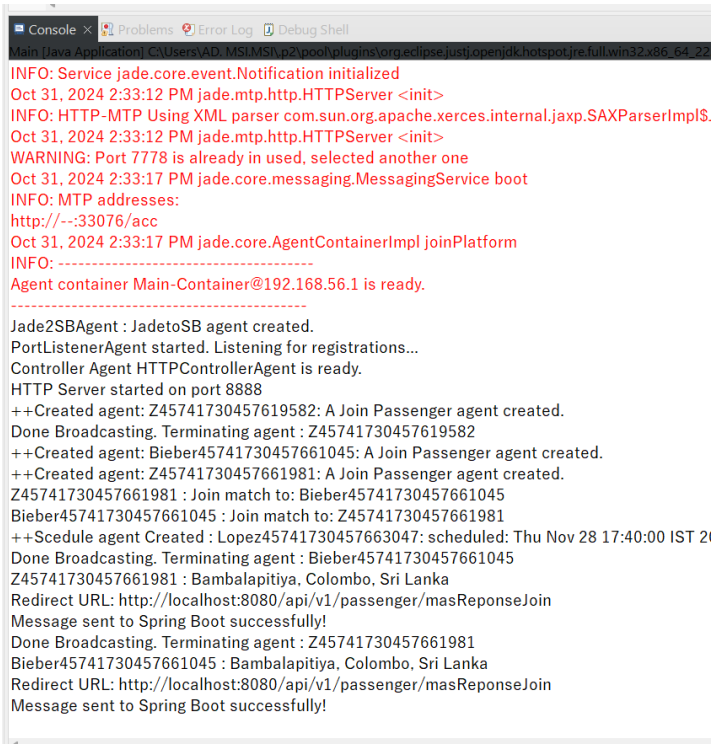


Figure 45: Monitoring for delivery in Jade MAS

An extra layer of E2E testing emphasizes the pilotization of the passenger and driver matching feature in a way that user preferences and provided drivers' criteria are incorporated into trip suggestions. This also includes checking how well the system supports the idea of drivers having preferences on the routes, the windows available to them and other adjustable parameters, so that they can pick on trips they want to take. The tests check how the system assigns drivers to passengers based on the updated

spatial and temporal constraints i.e. preferences on the driver's side are also catered to within the configuration of the request and the passenger's need addressed.

Lastly, E2E tests address the processes of trip matching with the help of hypothetical ride requests and system behaviors. In this case the back end's multi agent system, MAS, and its recommendation algorithms are evaluated on the accuracy of matching trips. Appropriate tests ensure that responses of a matched trip sent to the user are within an expected period, match the user's preferences and type of trip, and the user's availability. The E2E tests guarantee the end-to-end performance of the platform in matching users with their desired trips, from making a ride request to receiving trip suggestions, enhancing the confidence of the user in the project's capability of delivering a fast, smart, and useful ride matching-services.

6.4 Implementation

The approach to delivering and implementing this project for stakeholders revolves around building a MVP that encapsulates the core functionalities and unique features. This MVP serves as an introductory model, allowing stakeholders to interact directly with the system and gain hands-on experience with its capabilities, such as optimized trip matching, user-friendly interfaces, and real-time tracking. By introducing stakeholders to these key features in a controlled environment, the MVP demonstrates the project's potential impact and practicality, allowing for early feedback and alignment with real-world requirements.

6.4.1 Introduce Application to Users

Additionally, training workshops will be conducted in order to help the users understand the application, its usage and guidelines in a better way. This will help contrive an easier way for all the users including, drivers, passengers of the app, and stakeholders to use the application. The training will include how to navigate the various features and what a user has to know in order to have the best experience and to prepare the stakeholders to use the app with full knowledge of its purpose and the workflow of its operations.

This gradual introduction of system features minimizes the challenges associated with moving from MVP to a fully available product. In a cycle of different stages as obtaining data and new knowledge, improvement of the MVP might be done so as to arrive at the ultimate version whose system is responsive to the market's needs. Since the users have been engaged from the beginning, and their concerns have been taken care of in the development of the solution, the end product is likely to be more market friendly.

6.4.2 User Acceptance Testing (UAT)

The main objective of User Acceptance Testing (UAT) in this project is to quickly create and deliver an MVP to obtain genuine user feedback and confirm that the unique application features are as per intended users. UAT also justifies distinguishable core project functionalities that high success rate and fast trip-matching capabilities emphasized by the project- for instance, scheduling of join trips, arrangements of long trips as well as completing long trips with the assistance of local cab drivers who help out on the trip- provide features such capabilities which advantage the project in the solutions offered over the normal ride hailing applications, especially in regions where travel is mainly communal and flexibility is of paramount importance. We pay attention to how and when users access and engage those features during UAT, and we receive feedback on the usability, system responsiveness, and trip-matching accuracy. This step is important in understanding the factors and impacts of the user's benefits defined for the application e.g. waiting times, convenience and so on. It is also where user weaknesses that may include issues with navigating through features or waiting for a load are recorded to serve the purpose of enhancing the application's usability. UAT feedback leads to implementations that ensure that the application not only serves purpose required but appeals to the user. Regular UAT with user's input provides non-steady data to shape the last version which fits the user most, thus proving the application is ready for full release. This flexible UAT approach increases the competitiveness of the project by ensuring that every improvement is made and in every way the solution fits the end user.

6.5 Testing Results and Analysis

This stage is extremely important for the project because it provides information on to what extent the project's operational goals are met; for example, performance in trip-matching accuracy, user satisfaction and stability of the system as a whole. Based on these results we may distinguish the successful aspects of the design and the aspects that should be improved. For instance, the positive feedback on using the join-trip setting option attests to its necessity while the reaction time metrics help to focus on improving aspects which directly impact user experience.

In order to prepare for the mass market introduction of the application it is critical to understand the testing results and all their implications; this is particularly important for the features such as – multi-drivers management for a long-distance journey – which must work correctly in practice. In this perspective, it is also proved that the project maintains the main distinguishing factor from competitors: it provides quick, dependable, and easy to use services in circumstances where traditional systems fail. Consequently, this allows the project to carry on without reservations relying on this data as a basis for its relevance and addressing the concerns of its stakeholders.

6.6 Challenges and Limitations in Testing

6.6.1 High Resource Usage

The various challenges faced in the testing phase especially in mobile application development. One of the major challenges faced was the high resource consumption that was required to run several emulators at the same time. Emulators are important in that they enable the testing team to create different spatial contexts. However, they usually require a lot of processing capabilities especially when the testing features such as trip-matching that depend on real-time data and real-time response along with other agents are present. Since all features had to be tested together with the application, we had to run more than one instance of the application. This requirement however at times resulted in an increase in the testing period and resulted in more resource-related problems (figure 37).

6.6.2 Time Sensitivity in tests Cases

There were also some restrictions in the aspect of features that were time-dependent, for instance, trip reservations and time-based alerts which required a lot of precision. Testing was necessary for these time-dependent components to avoid any glitches when a user is working with the application, however, the presence of delays and time drift in the emulated environments would at times reduce the effectiveness of the testing. In addition, there were problems associated with debugging actions based on timing, as the imposition of real-time testing conditions often prolonged the exercise, affecting development time as a whole.

However, the testing environment was controlled in a way that aspects of the real world were attained with a few tweaking of the time and resource allocation. All these limitations pointed towards a requirement for better testing techniques for MAS oriented mobile applications and indeed influenced the approach of testing such applications in the subsequent versions so that the testing process was less resource demanding.

6.7 Summary

In this chapter, methodologies and steps of testing will be discussed in detail in order to validate and improve core functionality of the project. This chapter highlights the importance of testing at every level of the system to ensure that it is working as expected, starting from unit tests that check an individual function and up to integration tests that involve different components of the system, such as interaction of MAS with other components via sockets, and connections to Database over Web Services. The project engaged in end-to-end testing and gave emphasis particularly to trip matching,

trip scheduling and how passenger and driver modules interact with each other ensuring that the application is providing an integrated and user-friendly experience in much the same way that the users expect it to. In addition, this also helped to understand how effective the system would be in practice, as UAT was based on the feedback from potential users of the developed application. It must be noted that there were some difficulties related to this process, especially since deploying MVP as a mobile service meant also ensuring that testing could be performed on devices with different configurations in parallel and within very tight schedules. The results of the testing therefore were also useful for further development showing positive readiness of the project for implementation, and its ability to function as a reliable, flexible and user-friendly service in real life applications.

Chapter 07

System Evaluation

7.1 Introduction

Assessment of any system is an important process that allows one to understand whether the created application is up to the end users' expectations and whether it works as it was intended. In the case of "Join Go", an on-demand mobile application facilitating taxi sharing, extensive evaluation shall be performed to test the functionalities, capability and the quality of the system in general. This chapter focuses and expounds on the detailed evaluation techniques used to test the application in order to achieve the users' requirements as well as the objectives of the project. The evaluation process may require a number of assessment criteria which together determine the overall success of “Join Go” in accomplishing its set aim. Examining user interface, design, and other application performance metrics, this chapter will show the major benefits of the application and its possible drawbacks. Rather, the intention of the evaluation is to prove that ‘Join Go’ not only works as it should, but the users of the application will enjoy using it which sets the stage for its usage in the real world.

7.2 Project Practice Evaluation

Particularly, this appraisal examines the project framework of the "Join Go" project model from the perspective of its different phases of growth as well as the modifications incorporated to make the project more effective. This appraisal shall be concerned with the efficiency of the approaches used, the issues faced in actual implementation, and the solutions found for those issues. Each evaluation is an investigation of three main processes which are analysis, planning and performing.

In the analysis process, the pertained objectives at the beginning of the project are revisited and checks are done on their relation with the so far realized progress of the project. It examines how these aspects, in particular, the ethical trip matching, the incorporation of a multilayered agent system (MAS) for enhancing the travel-sharing experience and other such unique features of the project, serve the intended users. The primary strength of this evaluation is distinguishing the user-oriented highs and lows to suffix unmet demands of users.

In this stage, there are some ways providers approached the entire project cycle, one of them being MVP development to allow users to provide their feedback early on. This stage also deals with mechanisms for stakeholder feedback to be put in place as a way to meet their expectations. In addition, it explains the reasons for any changes introduced into the original timetable of the project following the growing needs or other difficulties, thereby taking care that the project is flexible and user-oriented.

7.2.1 Analyze Stage

In the analysis stage of the "Join Go" project, we delved into the bizarre needs of the ride sharing market, focusing on how easy and efficient the users want it to be. This part consisted of comprehensive analysis aimed to find the gaps within the transportation market that encouraged us to come up with a solution to those gaps. We used a range of approaches, including potential user questionnaires, commuter interviews and current ride sharing tendencies. These approaches helped us in exploring users' needs and their problems and desires.

A very important moment of this stage was defining correctly our audience. Who were the end users for the "Join Go" service and how could we benefit from it was a question that assisted in fine tuning our approach and reduced chances of altering a lot of things even when development was at an advanced stage. By doing so, target populations and their particular kinds of transportation problems were identified in order to make sure that the solution offered was fitting to the market that was intended. This approach helped to determine the key users of the service which was very important in the design and implementation stages and contributed to the success of the project in the competition.

The last stage of the analysis yielded such an insight into the project that one could start sketching those characteristic features, which would ensure the attractiveness of "Join Go" offering, i.e. how it is going to be different from other services. Such features as provision of ethical trip matching, and trimodal scheduling capabilities for cross-state travel, emerged as pivotal features based on user comments. This ambition of delivering a solution that goes beyond meeting the users' needs and actually gratifying them puts "Join Go" in a different category of businesses in the market. The gap is not simply filled hence the service that is geared towards making the safe commute enjoyable is there.

7.2.2 Design Stage

The design stage should be regarded as the most important step of the assignment because it is responsible for the mechanics of the "Join Go" system and how it will interface with users. This stage works towards building a system design from the detailed requirements collected, ensuring that the system is easy to use while being

practical for taxi users who want to access quality taxi services. Several important issues during the design phase encompass designing user interfaces to ease interaction with the system.

From the aspect of the project, which includes the aesthetics of architecture and its principles, the project proposes a simple and contemporary look of the user interface that is the order of the day for a typical user. The interconnecting of various modules is executed with consideration of the need to eliminate the bottlenecks of the processes involved in user data, trip matching, payment, and any other systems intact. The blueprint for every sub-system architecture is conceived in such a way that the individual sub-systems are interdependent and work toward achieving the objectives of the system overall. This design consideration does not only improve the usability of the system but it also enhances the market potency of "Join Go" in the ride sharing business as it is customized to the users' needs whilst maintaining performance and reliability.

7.2.3 Development Stage

The development stage is perhaps the most important phase in the creation of the "Join Go" aa system, responsible for the correct functioning of every element of the project. This includes the total system deployment and very suitable applied technologies, which serve to improve the performance of the system as well as to enhance the experience of the user. The objective of the application is clear and that is to design a strong application for the commuting public in need of a taxi that they can trust and will ensure the communication between the passenger and the driver is easy.

In the course of the development, special attention is paid to the compatibility of different technologies and their ability to integrate. This includes the choice of the proper frameworks and tools for the front end and back end of the application to manage end users' requests promptly, match trips in 'real' time and process payments in a secure way. Moreover, while developing the application, people are involved in a way that allows 'community outsourcing' so that other ideas that will enhance the system's R Eng less subsequent system development. This communication helps in recognizing possible roadblocks as well as shortcomings thereby supporting the development of a solution that understands the consumers' expectations as well as the shifts in the market.

The difficulties present in the web application development stage are of great magnitude as the agile methodology is being applied, thus this stage is one of the most challenging in the lifecycle of the project. Other conditions such as modular fabrication and the cyclic element of software development are taken into account to make certain that construction of every module will be done correctly and that same module can be modified or changed easily in future. This policy not only simplifies the processes of development but also ensures that future improvements will be made over "Join Go" sus.

7.2.4 Test and Implementation Stage

The testing and implementation phase constitutes a very significant point in the timeline of the 'Join Go' project since it implies change from the construction phase to an operational application. At this stage, it usually begins with the testing effort that is organized and planned in each and every aspect so that the system implemented is working properly aside from being able to meet user requirements. Different testing techniques are incorporated in measuring the application's functional and non-functional attributes like performance and reliability so that any possible exposures are fixed within the right schedule.

Even during testing, records of mistakes and suggestions for improvements on the work from the experts are carefully maintained by the development team which enables them to improve the product. The process concerned with improving strategy recognition entails confirming that all the elements are functioning properly as per the regulation and all the systems are within the required specifications. This step is important as far as the quality of the application and its users' satisfaction levels is concerned.

Key activities during the testing phase include:

- Establishing a comprehensive testing strategy to evaluate system performance.
- Documenting errors and correction requests from subject matter experts to ensure accountability.
- Verifying strategy recognition to confirm that the system meets all functional requirements.

Once testing is complete, the implementation phase follows, which involves the execution of the deployment strategy. Critical criteria are considered during this phase, including:

- Developing an effective implementation framework that outlines the steps for integrating the system.
- Fostering a supportive environment to facilitate successful deployment and user acceptance.
- Ensuring the system fits within practical contexts by adapting to real-world use cases.

Prior to launch, thorough system tests are conducted to confirm that the application is free of errors and that all functionalities are operating as intended.

Thus far, the focus has been on taking the necessary actions so that the conceptual vision can be translated into reality to achieve the project objectives. Now comes the difficult part: developing an all-encompassing plan for implementation because any of its elements may go wrong and impact on the project's direction. The deployment is achieved by means of proper testing procedures and execution practices, facilitating movement from production stage to operational stage with ease. This phase involves data collection in 2 aspects, which includes:

- Planning for operational and non-operational requirements of the intended system.
- Executing the integration of "Join Go" into real-world scenarios.

The goal of the project is to come up with an efficient and functional vehicle-sharing mobile application that is sensitive to the needs of the end users and biases integration problems and glitches towards the bare minimum possible by the intelligent management of this testing and implementation stage.

7.3 Functional Requirements of the System Evaluation

As far as the taxi app called “Join Go” is concerned, it is of utmost importance to assess the functional requirements of the system in order to safeguard the satisfaction of the users. These requirements are the backbone of the application, providing direction in its development and ensuring that the final output is dependable and efficient for a passenger and a driver. Below, each functional requirement is elaborated in order to underscore the importance of every functional requirement and the distinct features that “Join Go” possesses over and above its competitors.

7.3.1 Compatible Trip Matching on the Go

The application serves to link the passengers with the available taxi drivers who are within their vicinity along with their preferred destinations. Such arrangement is aimed at minimizing the waiting time and also the distance to be covered optimizing the transport experience for the users in need of instant mobility.

7.3.2 Passenger and Driver Management

The proficient handling of the passengers and driver profiles is critical for the effective management of the operations. A passenger is able to update personal information, check their past rides and set preferences whereas, a driver can manage trips and earnings. This dual management system increases the level of user engagement and satisfaction.

7.3.3 Ethical Passenger Trip Matching

The “Join Go” application has integrated mechanisms that fairly allocate trips among users taking into account their preferences and other moral aspects. This feature is intended to enhance fairness in ride-sharing by taking into account

factors like rides' sharing records and user scores, thereby making it more equitable for every user.

- **Appropriate Trip Suggestions**: The application assesses user habits and tastes in order to recommend specific trips to the user. This feature is intended to improve the experience by allowing the user to select options which will make them feel appreciated as well as catered for.

7.3.4 Delivery of Featured Trips

The system offers various ride options to cater to diverse user needs, enhancing the app's appeal. This includes:

7.3.4.1 *Normal Taxi:*

This service offers a ride-hailing functionality to its users where in they can call a ride or be driven to a desired place on their own terms. The main advantage of the regular taxi service is that it is straightforward since there is an app one can use to call a cab almost immediately without having to plan ahead. It is perfect for last minute travelling, for instance, to the airport, for business or social gatherings.

7.3.4.2 *Join/Shared Trip Taxi:*

Assimilate this ingenious component. It allows passengers going to the same or nearly the same direction to ride together significantly reducing the expenditure that would have otherwise been incurred. The algorithm effectively considers users by location and destination, so that passengers are optimally organized. It, therefore, makes transport cheaper and also fosters oneness in riders. In addition, it also eliminates wastage of resources in terms of the number of vehicles on the roads contributing to a greener or eco-friendly environment and reducing traffic jams.

7.3.4.3 *Scheduled Shared Taxi:*

This feature enables a user to set up shared rides ahead of time, that is, reserving a slot for a shared taxi at certain hours in the future. Whether it is for work purposes rush, meeting appointments or an event, a customer is even able to book his or her ride in advance without worrying about time constraints and can plan their program accordingly. The scheduled shared taxi feature does not only provide the advantages of having hassle-free travel arrangements but also maximizes on the join/shared trip option which is economical. This is especially beneficial to people who commute on a regular basis and do not like the uncertainties that come with taking a ride only when the need arises.

7.3.4.4 *Long-Distance Taxi:*

Designed for travelers covering long distances, long-distance taxi service is typically used for journeys between home and several cities/regions. This type of journey often contains changeover sections to allow the passengers to rest in the course of their travel. What makes this service very core is that you can connect several drivers to such long distances in order to satisfy the requirements for long journeys. Through the app, since it's worked out locally with local drivers, all the passengers along the route will also have drivers who are ready to take them, hence improving the service and considerably cutting down the waiting time. This method of traveling helps to avoid discomfort and makes it easy to travel with many bags but also includes and takes advantage of several drivers helping each other. This also applies to people who keep traveling excessively over long distances for days either due to fun or work. The challenge “Join Go” seeks to solve is that of long travel distances. With this transport service, users are guaranteed to receive proper, dependable facilitations that will meet their travel needs, and above all, they get to interact with a pool of drivers willing to work for them efficiently.

"In Joining Go," in as much as all these functional requirements are met, the goal is to create an informative and easy to use platform, which resolves other normal transport related problems and provides creative ideas that are designed for the passengers as well as the drivers clients.

7.4 Summary

Last but not the least, the system evaluation section in the project titled ‘Autonomous Join-trip Matching Travel Mobile Application Using Multi-Agent Technology’ (Join Go) reduces due consideration to the assessment done to ascertain the application’s applicability for its users and functioning capacity. Such assessment included researching the market, developing interfaces, and performing several tests in order to guarantee the validity of the project. Furthermore, the user experience has been improved by the incorporation of elements such as: sign-up, ethical trip matching, and different types of rides. This is even enhanced by the aspect of drivers going for long distance trips pooling in. In conclusion, it has been shown through this evaluation that the solution presented by “Join Go” is practicable within the transport sector and ready for successful application in the actual world.

Chapter 08

Conclusion and Recommendations

8.1 Introduction

The earlier section assessed the performance and practical implementation of the advanced and groundbreaking ‘Autonomous Join-trip Matching Travel Mobile Application using Multi-Agent Technology’ – ‘Join Go.’ This section states the overall outcomes from the research and development activities conducted during the project, stressing the extent to which the system achieved its goals in addressing the problem of commuters. It will also present suggestions on how the system may be enhanced as well as propose possible enhancement strategies related to the application of the system in the future.

8.2 Revised Objectives

Delving back into the key aims of the initiative "Join Go," this part examines the ways in which these aims were pursued and altered over the course of the project in order to meet the requirements of the ever-changing taxi sharing landscape.

1. **Critical Review of Existing Technologies:**

   The primary goal was to perform an in-depth analysis of existing ride sharing systems and Multi Agent Systems (MAS), with the intention of establishing the shortcomings of existing taxi sharing system. The analysis not just pointed out the non-existence of ethical trip matching and resource optimized routing but also showed the necessity for dynamic solutions that cope with lash congestion and improve the passenger experience. Hence, ‘Join Go’ aimed at these advanced MAS integration developments suitable for ride sharing considering the existing challenges.

2. **Selection of Suitable Technologies for Decentralized Communication:**

   One of the primary tasks was recognizing technologies that would support mobile-embedded, decentralized agent-centered communication. The communication needs of the MAS were integrated with Spring Boot for the

backend support, while React Native was adopted for frontend interaction, and purposely so. This approach made it possible for agents to interact real-time without having to store any information in one place, which was advantageous in ensuring effective communication of agents and improved agility, especially in dynamic traffic situations.

3. **Design and Development of the Join Go Application:**

   One of the main goals of the project was the application's design and implementation employing MAS for real-time decision making, geo-tracking, and ethical trip matching. It managed to combine in its design dynamic trip merger with real-time tracking which works rather easily thanks to the user-friendly interface of React Native and the backend durability of Spring Boot. Such design does allow for sharing expenses for such trips and offers options for both vehicle-sharing over long or short distances depending on the preferences of users.

4. **System Performance Evaluation:**

   Assessing the effectiveness of agent-based decision making played a crucial role in improving user experience especially when it came to different levels of traffic intensity such as during rush hours. System evaluations proved that the autonomous decisions of the agents resulted in lower costs and more efficient trips which were in line with the project's aim in user satisfaction. The improvements in user experience were achieved by enhancing the route suggestions and the waiting times which also helped 'Join Go' in its effort towards offering an affordable taxi-sharing service for environmental protection.

5. **Usability and Scalability for the Sri Lankan Market:**

   Last but not least critical goal was measuring how well "Join Go" fits the practical needs of the Sri Lankan transportation sector. Also, the scalability of the solution was tested, in order to meet the different levels of commuter needs, such as daily traveling within the city and traveling from one city to another. Evaluation findings affirm that the application possesses both usability and operational viability in the market addressing a variety of travel needs in the region.

8.3 Recommendation related to the system

- **Enhanced User Customization:**
  Further customization opportunities will enable the users to specify preferences for shared rides, routes, and duration of travel, which will enhance the user experience in "Join Go" application.

- **Machine Learning for Route Optimization:**
  If machine learning is integrated route suggestions can be variably improved as it will be able to utilize previous records in enhancing the trip matching and traffic predicting reducing unnecessary delays when there are travelers' peak periods thereby increasing the level of travel efficiency.
- **Scalability for Broader Markets:**
  In order to reach out to more users, modifying the system 'Join Go' to make it more scalable would help in penetrating areas with identical commuting patterns by adjusting agent preferences to data from different cities.
- **Regular System and Feedback Updates:**
  Properly triggered revisions and updates in line with user feedback will help sustain system effectiveness, deal with problems expeditiously, and guarantee a dependable and superior level of service to users.

8.4 Conclusion

As the name of the project implies, it is aimed developing an "Autonomous Join-trip Matching Travel Mobile Application using Multi-Agent Technology" joined with the product "Join Go". This project made remarkable achievements and built strong records which can help in the future technologies applications in the exploration. This project presented a different kind of ride-sharing concepts in which seeks for morally and effective trip joining that meet the need of shorter or longer distance commuting. The high success rate in the adoption of decentralized decision making, effective route matching, and geo-location tracking in real time depicts the benefits of Multiple Agent Technology in enhancing service delivery and reducing costs to the users.

Since this project wastes no time in looking for new opportunities for personal development, it helps different in-the-field technologies including: Spring Boot, React Native, MAS architecture and agent communication. Working in R&D translated into advanced comprehension of how A.I works and how it can be utilized, most especially in a decentralized manner using agents, which laid the groundwork for looking into how transportation systems can be made intelligent via A.I. Generally, Join Go is a convenient and creative answer to modern commuting issues aimed at the end-user, filling important shortcomings in existing systems for ride-sharing and outlining the further development of travel solutions based on multi-agent systems.

8.5 Future Enhancements regarding the project

Improvements aimed at the "Autonomous Join-trip Matching Travel Mobile Application Based on Multi-Agent Technology" (Join Go) project can be a huge asset

to the growth of the project, user satisfaction, and the application of the system in different real-life situations. Such proposed improvements consist of:

- **Advanced Predictive Analytics**: Integrating predictive analytics based on historical travel data to enhance trip-matching accuracy and improve the reliability of time and cost estimates.
- **Dynamic Pricing Models**: Implementing flexible pricing algorithms that adapt to real-time demand and supply conditions, creating a fair cost-sharing structure for passengers and drivers, especially during peak hours or long-distance trips.
- **Machine Learning for Enhanced Agent Behavior**: Leveraging machine learning to refine the decision-making capabilities of agents, improving the efficiency of route selection, passenger matching, and response to traffic changes.
- **Expanded Integration with Third-Party Services**: Developing partnerships with navigation, payment, and local service providers to offer additional in-app features such as automated toll payments, integration with public transit, or discounts on frequent routes.
- **Enhanced User Personalization**: Tailoring app features to individual preferences by allowing users to set trip priorities, select preferred vehicle types, and receive recommendations based on past behaviors and preferences.
- **Sustainability-Oriented Features**: Implementing eco-friendly ride options, including vehicle-sharing with reduced emissions, to promote environmental responsibility within the app's operations.

The mentioned possible improvements will improve Join Go's flexibility, efficiency, and user interaction which will help the application sustain its relevance in the changing ride-sharing environment.

8.6 Summary

This chapter has presented a sketch of possible future improvements to the "Autonomous Join-trip Matching Travel Mobile Application using Multi-Agent Technology" (Join Go) invention, so as to enhance its flexibility, user experience, and technology profundity. The existing proposals enhancement advanced predictive analysis, dynamic pricing, machine learning enhancement and upgrade personalization through-out the project which are expansion to the project core features. These improvements particularly look towards the future of Join Go due to the attention paid to factors such as scalability, integration with other services and even sustainability. The proposed improvements will ensure that the application is well positioned in the ride sharing market which is dynamic in nature and will help in its use in other mass transit scenarios.

References

[1] IBM, "Intelligent Transportation Systems," n.d.. [Online]. Available: http://IBM.com.

[2] BlaBlaCar, n.d.. [Online]. Available: http://BlaBlaCar.com.

[3] Waymo, n.d.. [Online]. Available: http://Waymo.com.

[4] Lyft, n.d.. [Online]. Available: http://Lyft.com.

[5] D. J. Fagnant and K. M. Kockelman, "The Future of Vehicle Sharing: A Systematic Review of the Literature," *Transport Reviews,* vol. 34, no. 6, pp. 748-774, 2014.

[6] T. D. Chen and K. M. Kockelman, "A Review of Car-Sharing Systems in the United States: Current Practices and Future Directions," *Transport Reviews,* vol. 36, no. 6, pp. 734-751, 2016.

[7] C. -Y. Tsai and S. -Y. Wong, "Investigating Factors Influencing User Acceptance of Car-Sharing Services," *Transport Policy,* vol. 42, pp. 119-130, 2015.

[8] S. K. Khatami and R. Khani, "A Comparative Study of Ride-Sharing Systems: Uber and Lyft," *Journal of Transportation Technologies,* vol. 8, no. 1, pp. 34-49, 2018.

[9] S. Gunter, "The Rise of Autonomous Vehicles: How They Will Affect Ride-Hailing and Car-Sharing," *IEEE Intelligent Transportation Systems Magazine,* vol. 9, no. 4, pp. 36-44, 2017.

[10] . A. . M. El and N. A. Yaghoob, "Sustainable Urban Mobility: An Agent-Based Approach to Transportation Modelling," *Sustainability,* vol. 13, no. 1, p. 102, 2021.

[11] A. Ahasan, "The Role of Intelligent Transport Systems in Sustainable Urban Mobility," *Smart Cities,* vol. 3, no. 1, pp. 88-101, 2020.

[12] A. Stathopoulos and A. B. Whitford, "Understanding the Future of Ride-Hailing Services: A Policy Perspective," *Transportation Research Part A: Policy and Practice,* vol. 132, pp. 62-73, 2020.

[13] C. Mulley and J. D. Nelson, "The Role of Shared Mobility in the Future of Public Transport," *Public Transport,* vol. 11, no. 1, pp. 1-13, 2019.

[14] P. Santi and a. et, "Quantifying the Benefits of Ride-Sharing Systems," *Proceedings of the National Academy of Sciences,* vol. 111, no. 23, pp. 8395-8400, 2014.

[15] T. Litman, "Transportation and Environmental Policy," 2020.

[16] R. J. Schneider and C. Lentz, "Analyzing the Economic Impacts of Car-Sharing Services," *Transport Economics,* vol. 15, no. 3, pp. 159-176, 2017.

[17] P. Zhu and S. He, "The Impact of Ride-Sharing on Public Transportation," *Transportation Research Part A: Policy and Practice,* vol. 118, pp. 186-196, 2018.

[18] J. Glover and . M. McNally, "Consumer Preferences in Ride-Sharing Applications: An Analysis of User Experience," *Journal of Transportation Research,* vol. 25, no. 2, pp. 45-58, 2018.

[19] M. Furuhata and a. et, "A Review of Ride-Sharing Operations Research," *Transportation Science,* vol. 47, no. 3, pp. 255-276, 2013.

[20] A. Henao and W. E. Marshall, "The Impact of Ride-Hailing on Public Transportation Use: A Case Study of Denver," *Transportation Research Record,* vol. 2673, no. 8, pp. 1-12, 2019.

[21] . S. Shaheen and A. Cohen, "Carsharing and Personal Vehicle Services: A Review of the Literature," *Transportation Research Board Annual Meeting,* pp. 13-2512, 2013.

[22] S. Chien, . Y. Ding and C. Wei, "Examining the Effects of Ride-Sharing Apps on Public Transport Usage," *Transport Policy,* vol. 60, pp. 39-49, 2017.

[23] Y. Zhang and A. Matz, "Understanding the Limitations of Ride-Sharing Systems in Urban Transportation," *Transportation Research Part A: Policy and Practice,* vol. 126, pp. 1-14, 2019.

[24] K. Wang and a. et, "Dynamic Ride-Sharing: A Survey of Current Research," *Transportation Research Part C: Emerging Technologies,* vol. 113, pp. 1-23, 2020.

[25] V. Kadiyali and a. et, "Economic Impacts of Ride-Sharing Services: Evidence from Uber," *Journal of Economic Perspectives,* vol. 30, no. 4, pp. 127-150.

[26] . C. Chen and a. et, "The Role of Dynamic Pricing in Ride-Hailing Services," *Operations Research Letters,* vol. 47, no. 2, pp. 100-104, 2019.

[27] M. Khosrowpour and a. et, "User Preferences in Ride-Sharing Services: A Survey," *Journal of Transport Geography,* vol. 88, pp. 102-115, 2020.

[28] . N. C. McDonald and M. Kuby, "The Effect of Ride-Hailing Services on Public Transit: A Case Study of New York City," *Transportation Research Part A: Policy and Practice,* vol. 80, pp. 182-193, 2015.

[29] . J. P. Schwieterman and a. et, "The Safety Challenge of Ride-Sharing Services: Implications for Public Policy," *Journal of Public Transportation,* vol. 21, no. 1, pp. 49-65, 2018.

[30] . J. A. Allen and a. et, "Environmental Impact of Ride-Sharing Services: A Review of the Literature," *Environmental Research Letters,* vol. 16, no. 2, 2021.

[31] T. Wenzel, "The Impact of Driver Availability and Service Quality on User Adoption of Ride-Sharing Apps," *Journal of Transportation Engineering,* vol. 145, no. 2, 2019.

APPENDIX B: Questionnaire Survey Form

Data Collection On Shared Taxi for Sri Lanka

I am a final year undergraduate of faculty of Computing in KDU, Ratmalana who is conducting an individual final year project based on above topic and hoping to collect data from expected stakeholders on the completion of the project.

By analyzing this data, we aim to tailor the Join Go application to meet local needs, promoting a sustainable, reliable, and user-friendly shared taxi experience. This survey is crucial for understanding the unique commuter demands in Sri Lanka and enabling the app to enhance urban mobility and convenience for everyday commuters. Please be kind to contribute few seconds of your time to future benefit to both of us.

diloxlyfdo@gmail.com Switch account

Not shared

* Indicates required question

How often you use taxi services day-today? *

- ◯ Daily
- ◯ Several times a week
- ◯ Occasionally (a few times a month)
- ◯ Rarely or never

Are u a Daily Commuter? *

- ◯ Yes
- ◯ No

How often you look for a taxi service in commute?

- ◯ Every commute
- ◯ Only during peak times or rush hours
- ◯ Rarely, only when necessary

Preferred medium of taxi? *

- [ ] Bike
- [ ] Tuk
- [ ] Compact Urban Car
- [ ] Cedan

How often you use mobile app for taxi service? *

- ◯ Frequently, for most rides
- ◯ Occasionally, when needed
- ◯ Rarely, prefer other methods

Average waiting time for taxi:

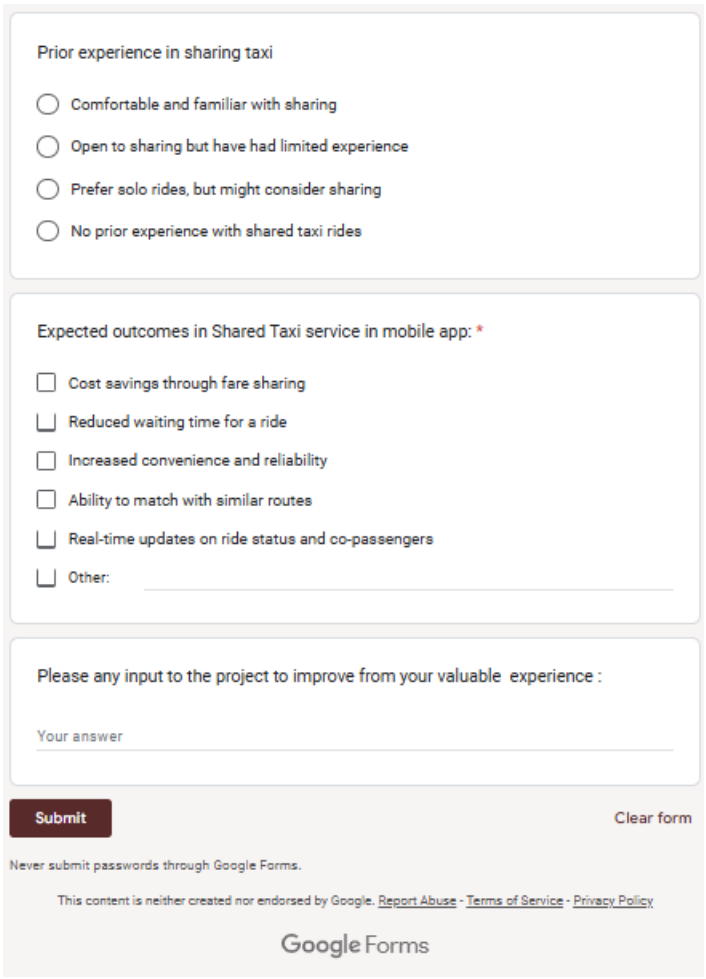
- ◯ 2 - 5 minutes
- ◯ 5 - 10 minutes
- ◯ Other: _____

Average waiting time for taxi in Rush hours: *

- ◯ 2 - 5 minutes
- ◯ 5 - 10 minutes
- ◯ 10 - 15minutes
- ◯ over 15 minutes
- ◯ Other: _____

Have experienced in sharing taxi *

- ◯ Yes
- ◯ No


Prior experience in sharing taxi

- ◯ Comfortable and familiar with sharing
- ◯ Open to sharing but have had limited experience
- ◯ Prefer solo rides, but might consider sharing
- ◯ No prior experience with shared taxi rides

Expected outcomes in Shared Taxi service in mobile app: *

- ☐ Cost savings through fare sharing
- ☐ Reduced waiting time for a ride
- ☐ Increased convenience and reliability
- ☐ Ability to match with similar routes
- ☐ Real-time updates on ride status and co-passengers
- ☐ Other:

Please any input to the project to improve from your valuable experience :

Your answer

Submit Clear form

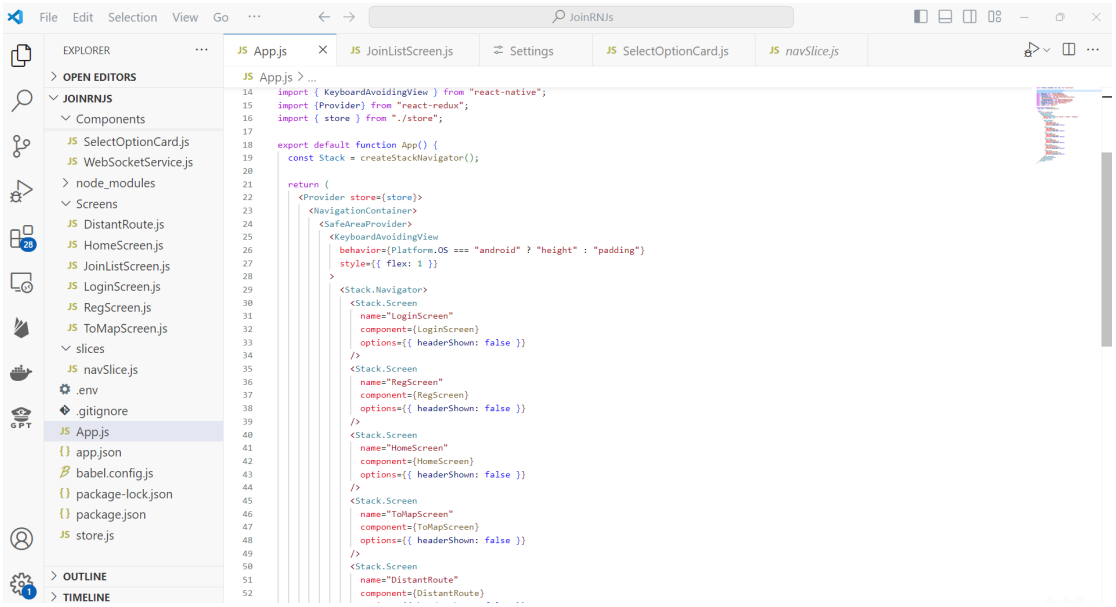
Never submit passwords through Google Forms.

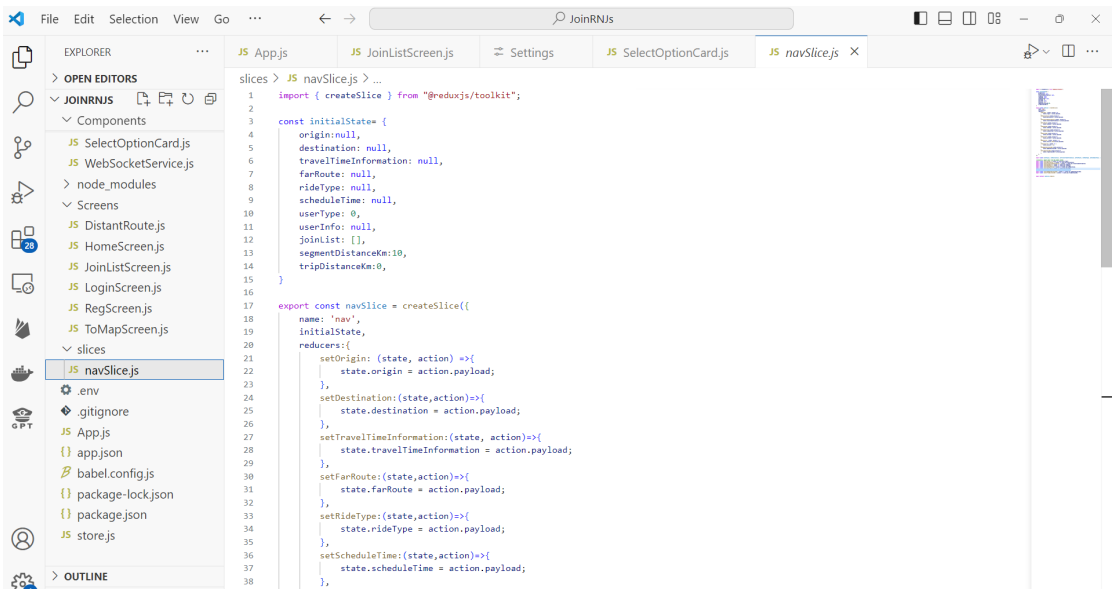
This content is neither created nor endorsed by Google. Report Abuse - Terms of Service - Privacy Policy

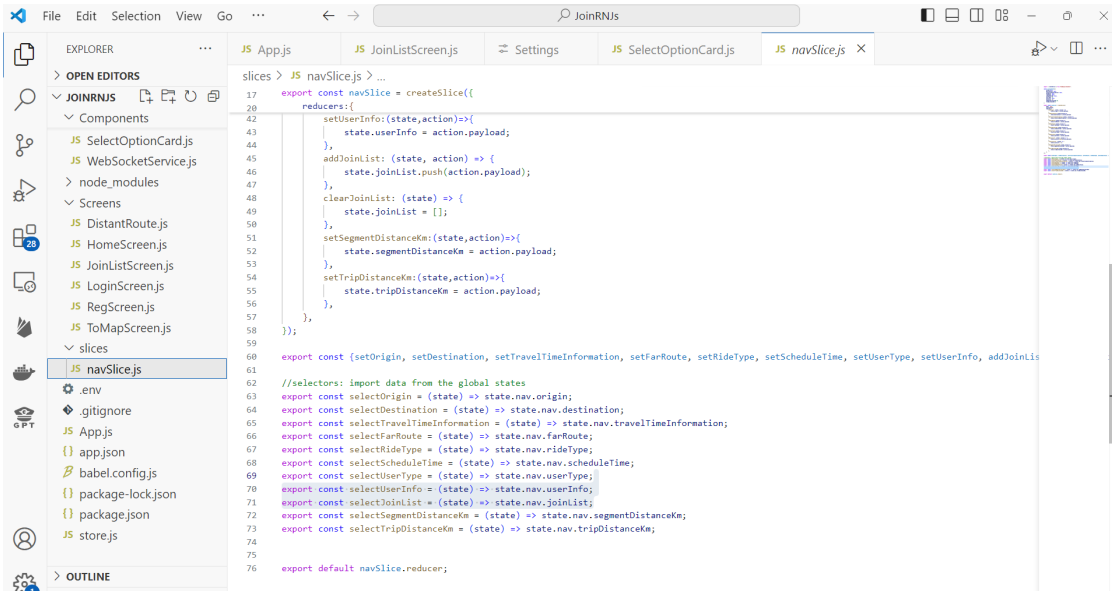
Google Forms

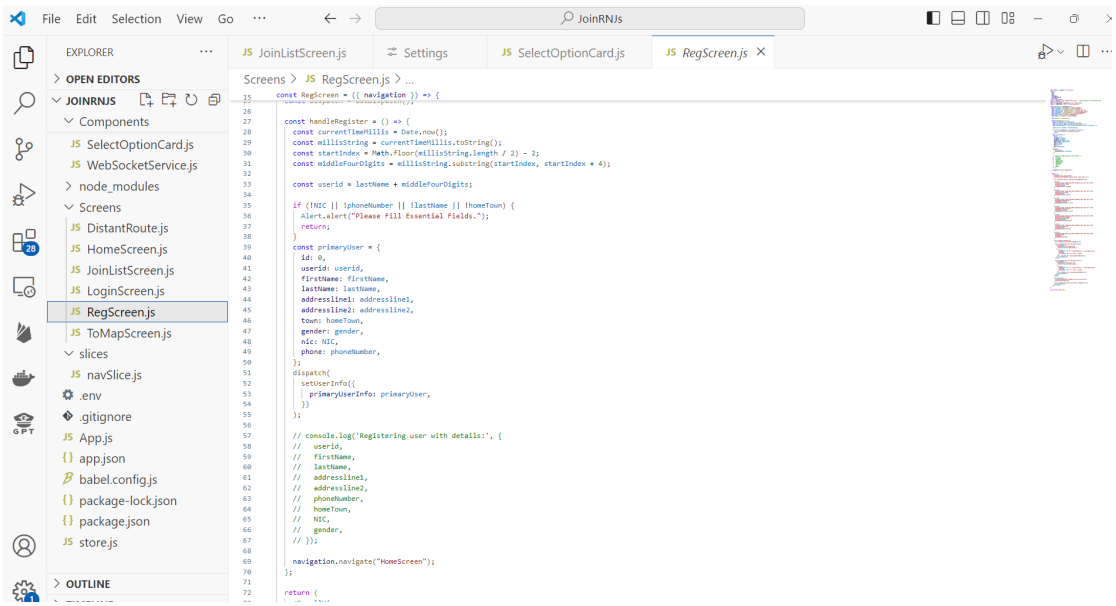
APPENDIX C: Code

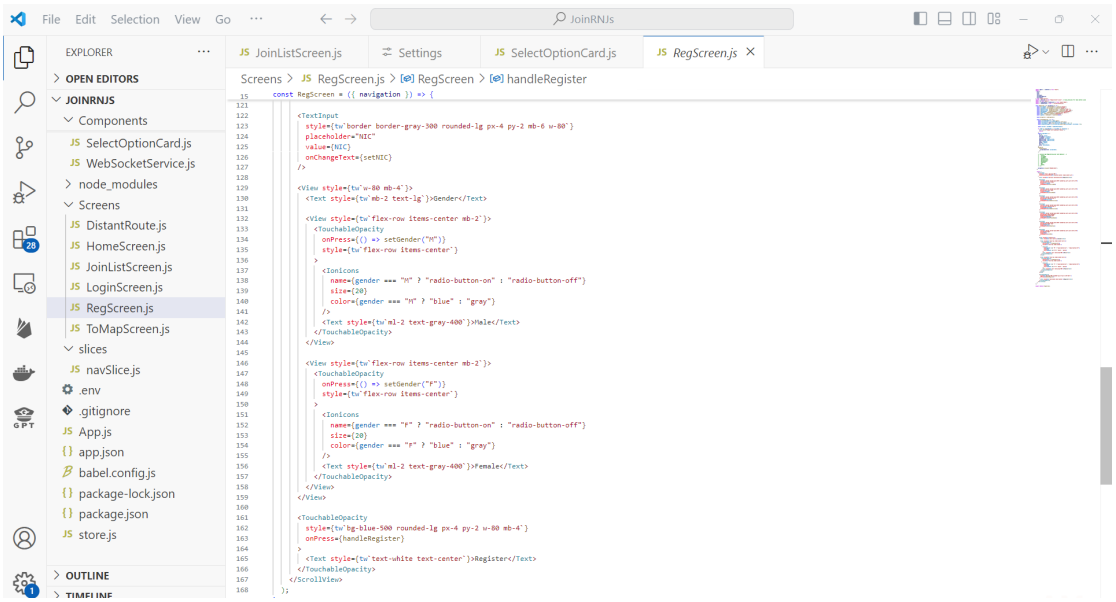
- Passenger User Front-end:
  - Presentation Layer-

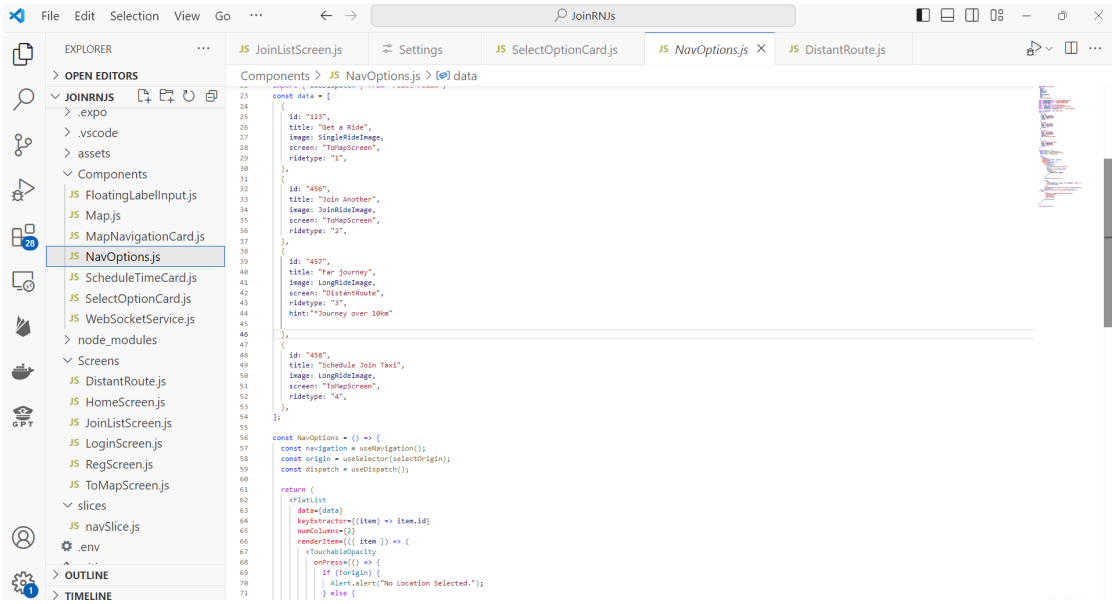


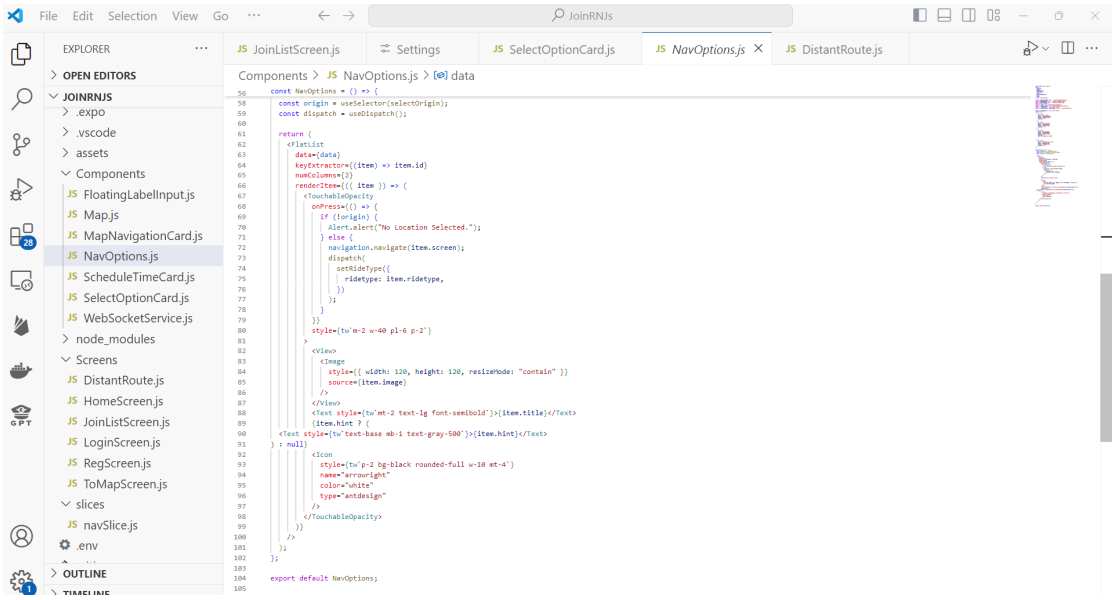


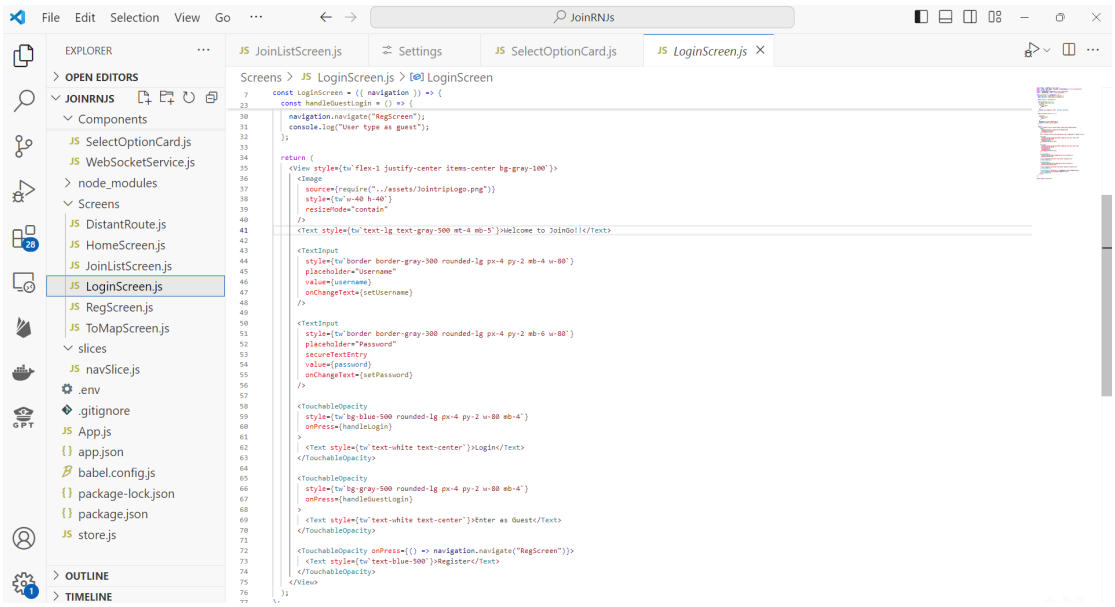

File Edit Selection View Go
JoinRNJs
EXPLORER
App.js
JoinListScreen.js
Settings
SelectOptionCard.js
navSlice.js
slices > navSlice.js > ...
OPEN EDITORS
JOINRNJS
Components
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
.gitignore
App.js
app.json
babel.config.js
package-lock.json
package.json
store.js
OUTLINE
export const navSlice = createSlice({
reducers:{
setUserInfo:(state,action)=>{
state.userInfo = action.payload;
},
addJoinList: (state, action) => {
state.joinList.push(action.payload);
},
clearJoinList: (state) => {
state.joinList = [];
},
setSegmentDistanceKm:(state,action)=>{
state.segmentDistanceKm = action.payload;
},
setTripDistanceKm:(state,action)=>{
state.tripDistanceKm = action.payload;
},
},
});
export const {setOrigin, setDestination, setTravelTimeInformation, setFarRoute, setRideType, setScheduleTime, setUserType, setUserInfo, addJoinLis
//selectors: import data from the global states
export const selectOrigin = (state) => state.nav.origin;
export const selectDestination = (state) => state.nav.destination;
export const selectTravelTimeInformation = (state) => state.nav.travelTimeInformation;
export const selectFarRoute = (state) => state.nav.farRoute;
export const selectRideType = (state) => state.nav.rideType;
export const selectScheduleTime = (state) => state.nav.scheduleTime;
export const selectUserType = (state) => state.nav.userType;
export const selectUserInfo = (state) => state.nav.userInfo;
export const selectJoinList = (state) => state.nav.joinList;
export const selectSegmentDistanceKm = (state) => state.nav.segmentDistanceKm;
export const selectTripDistanceKm = (state) => state.nav.tripDistanceKm;
export default navSlice.reducer;

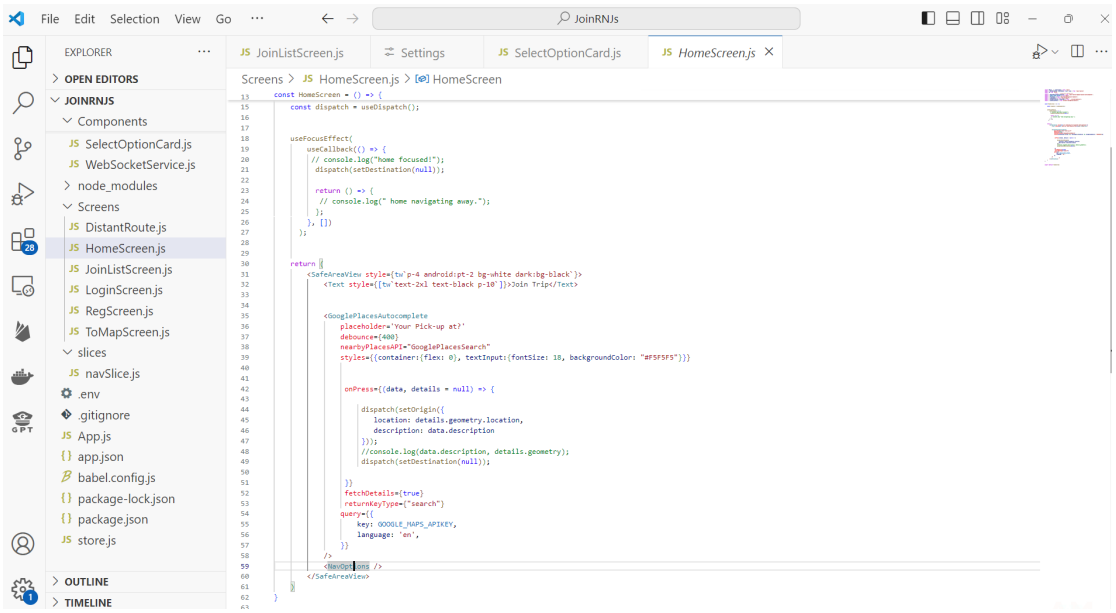

File Edit Selection View Go
JoinRNJs
EXPLORER
JoinListScreen.js
Settings
SelectOptionCard.js
RegScreen.js
Screens > RegScreen.js > ...
OPEN EDITORS
JOINRNJS
Components
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
.gitignore
App.js
app.json
babel.config.js
package-lock.json
package.json
store.js
OUTLINE
const RegScreen = ({ navigation }) => {
const handleRegister = () => {
const currentTimeMillis = Date.now();
const millisString = currentTimeMillis.toString();
const startIndex = Math.floor(millisString.length / 2) - 2;
const middleFourDigits = millisString.substring(startIndex, startIndex + 4);
const userid = lastName + middleFourDigits;
if (!NIC || !phoneNumber || !lastName || !homeTown) {
Alert.alert("Please Fill Essential Fields.");
return;
}
const primaryUser = {
id: 0,
userid: userid,
firstName: firstName,
lastName: lastName,
addressline1: addressline1,
addressline2: addressline2,
town: homeTown,
gender: gender,
nic: NIC,
phone: phoneNumber,
};
dispatch(
setUserInfo({
primaryUserInfo: primaryUser,
})
);
// console.log('Registering user with details:', {
//   userid,
//   firstName,
//   lastName,
//   addressline1,
//   addressline2,
//   phoneNumber,
//   homeTown,
//   NIC,
//   gender,
// });
navigation.navigate("HomeScreen");
};
return (

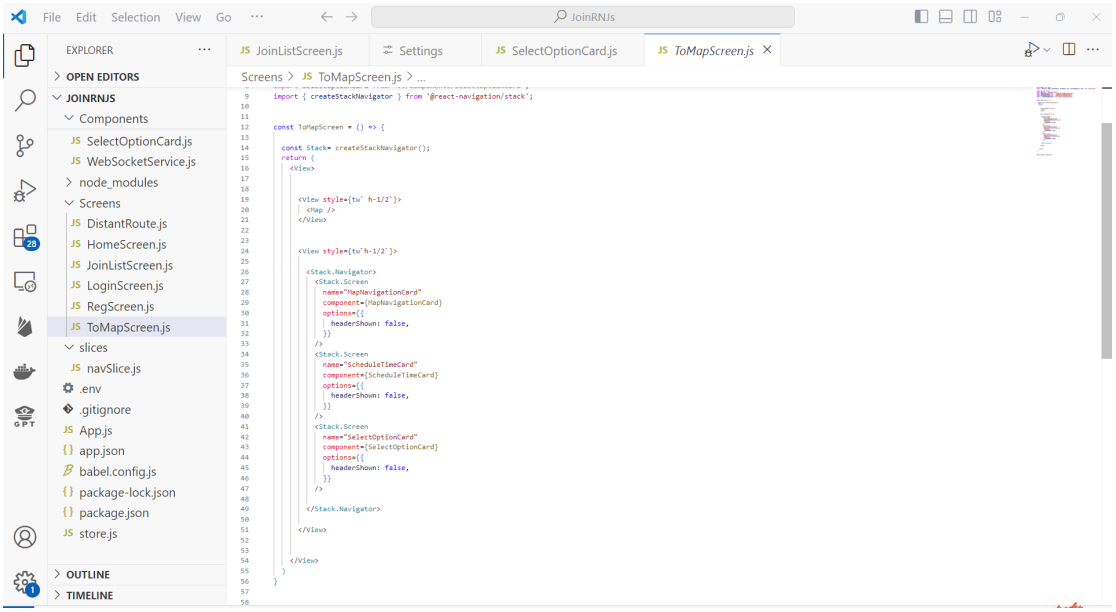

File Edit Selection View Go
JoinRNJs
EXPLORER
JS JoinListScreen.js
Settings
JS SelectOptionCard.js
JS RegScreen.js
Screens > JS RegScreen.js > RegScreen > handleRegister
OPEN EDITORS
JOINRNJS
Components
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
.gitignore
App.js
app.json
babel.config.js
package-lock.json
package.json
store.js
OUTLINE
TIMELINE

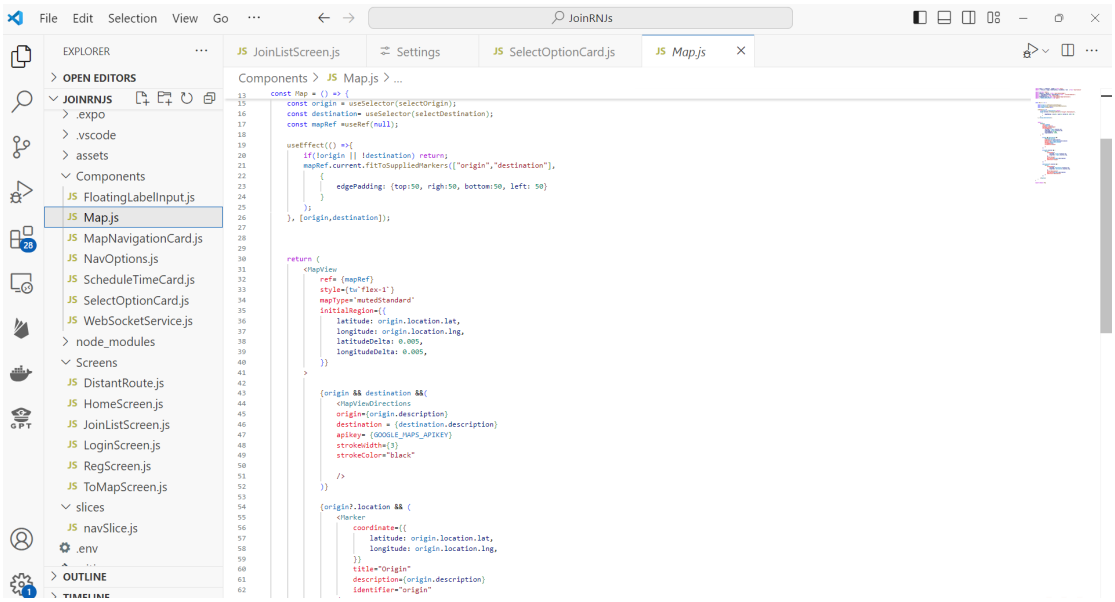

File Edit Selection View Go
JoinRNJs
EXPLORER
JS JoinListScreen.js
Settings
JS SelectOptionCard.js
JS NavOptions.js
JS DistantRoute.js
Components > JS NavOptions.js > data
OPEN EDITORS
JOINRNJS
.expo
.vscode
assets
Components
FloatingLabelInput.js
Map.js
MapNavigationCard.js
NavOptions.js
ScheduleTimeCard.js
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
OUTLINE
TIMELINE

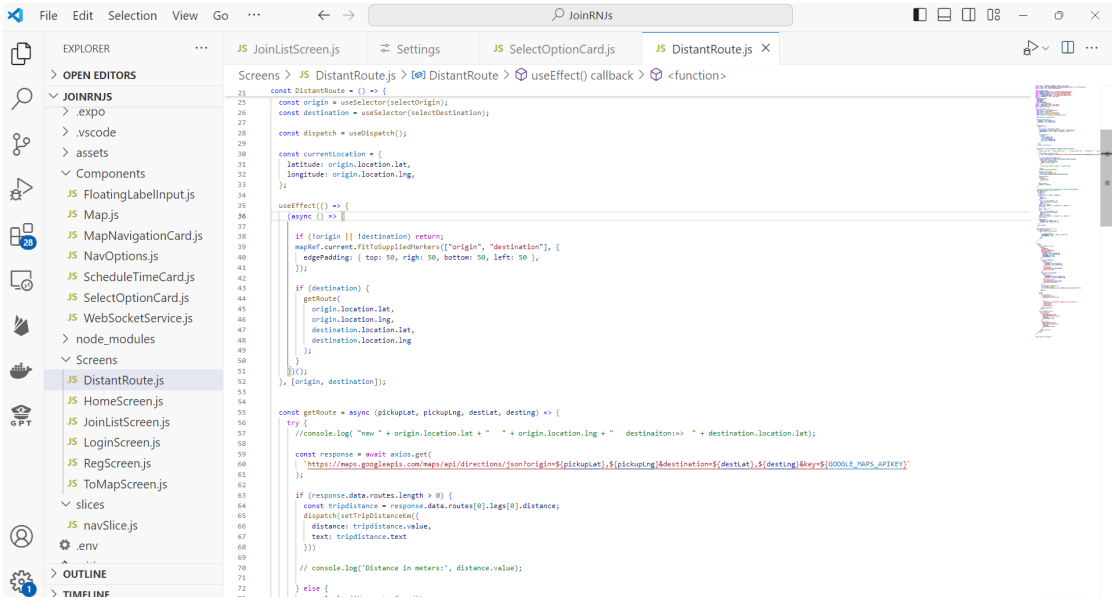

File Edit Selection View Go
JoinRNJs
EXPLORER
JoinListScreen.js
Settings
SelectOptionCard.js
NavOptions.js
DistantRoute.js
Components > NavOptions.js > data
OPEN EDITORS
JOINRNJS
.expo
.vscode
assets
Components
FloatingLabelInput.js
Map.js
MapNavigationCard.js
NavOptions.js
ScheduleTimeCard.js
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
OUTLINE
TIMELINE

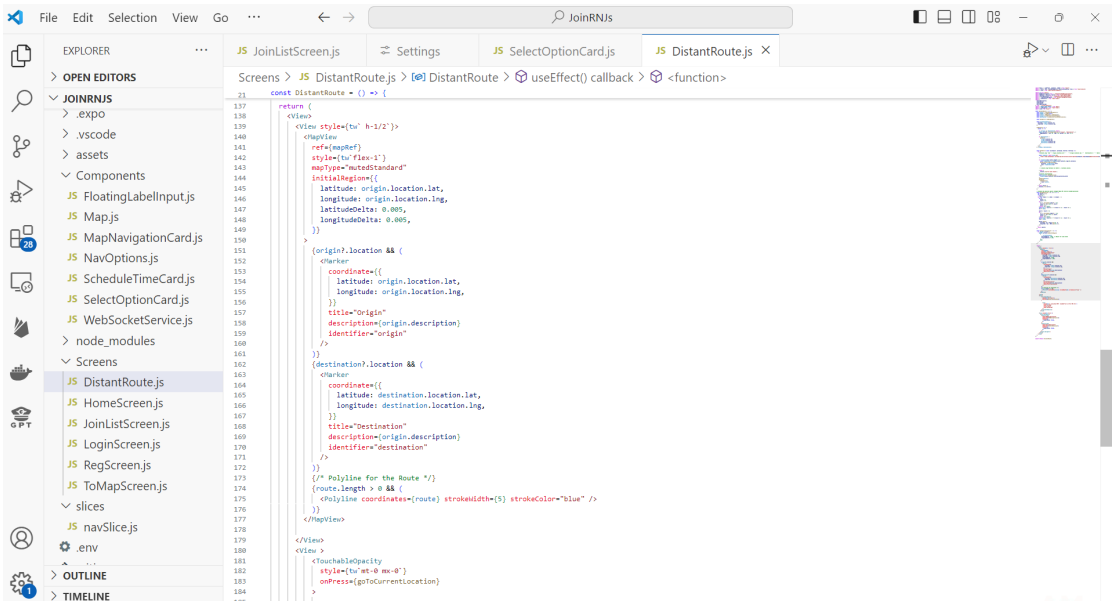

File Edit Selection View Go
JoinRNJs
EXPLORER
JoinListScreen.js
Settings
SelectOptionCard.js
LoginScreen.js
Screens > LoginScreen.js > LoginScreen
OPEN EDITORS
JOINRNJS
Components
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
.gitignore
App.js
app.json
babel.config.js
package-lock.json
package.json
store.js
OUTLINE
TIMELINE

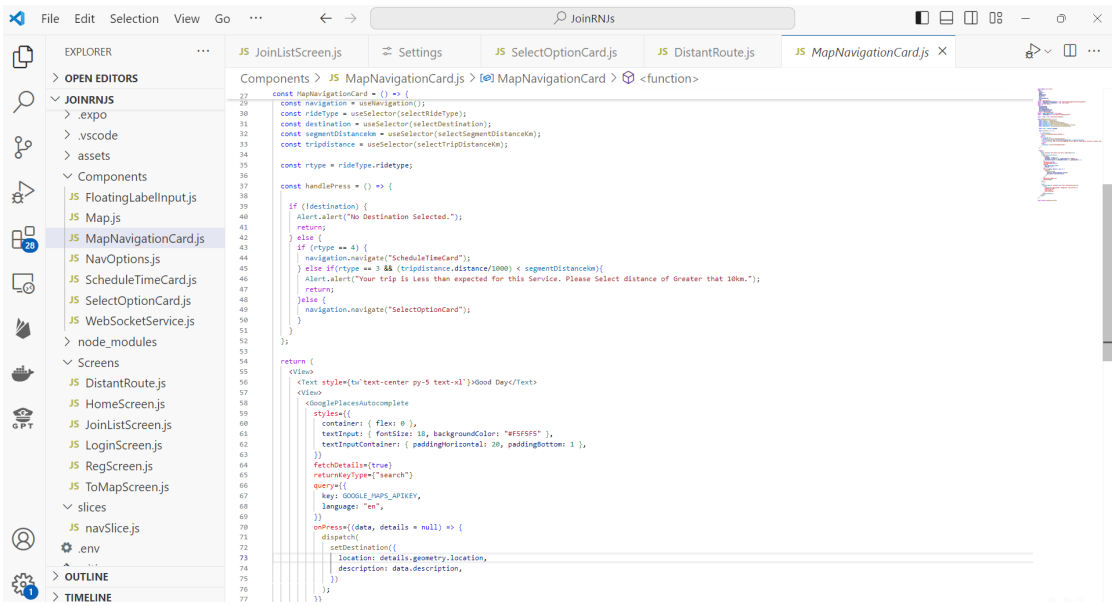

File Edit Selection View Go
JoinRNJs
EXPLORER
OPEN EDITORS
JOINRNJS
Components
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
.gitignore
App.js
app.json
babel.config.js
package-lock.json
package.json
store.js
OUTLINE
TIMELINE
JoinListScreen.js
Settings
SelectOptionCard.js
HomeScreen.js
Screens > HomeScreen.js > HomeScreen
const HomeScreen = () => {
const dispatch = useDispatch();
useFocusEffect(
useCallback(() => {
// console.log("home focused!");
dispatch(setDestination(null));
return () => {
// console.log(" home navigating away.");
};
}, [])
);
return (
<SafeAreaView style={tw`p-4 android:pt-2 bg-white dark:bg-black`}>
<Text style={[tw`text-2xl text-black p-10`]}>Join Trip</Text>
<GooglePlacesAutocomplete
placeholder='Your Pick-up at?'
debounce={400}
nearbyPlacesAPI="GooglePlacesSearch"
styles={{container:{flex: 0}, textInput:{fontSize: 18, backgroundColor: "#F5F5F5"}}}
onPress={(data, details = null) => {
dispatch(setOrigin({
location: details.geometry.location,
description: data.description
}));
//console.log(data.description, details.geometry);
dispatch(setDestination(null));
}}
fetchDetails={true}
returnKeyType={"search"}
query={{
key: GOOGLE_MAPS_APIKEY,
language: 'en',
}}
/>
<NavOptions />
</SafeAreaView>
)
}

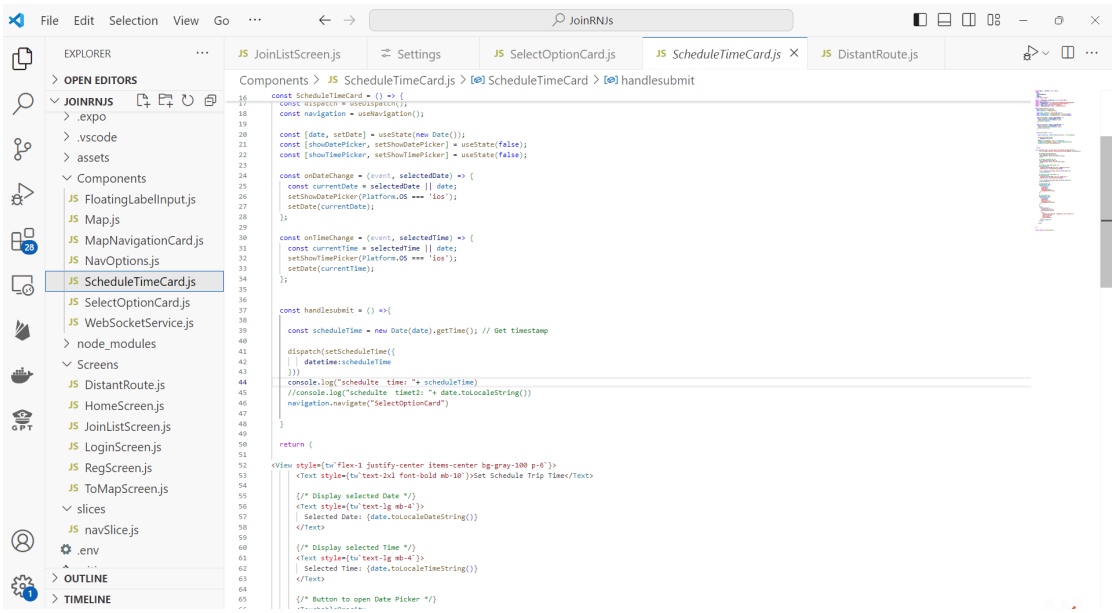

File Edit Selection View Go
JoinRNJs
EXPLORER
OPEN EDITORS
JOINRNJS
Components
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
.gitignore
App.js
app.json
babel.config.js
package-lock.json
package.json
store.js
OUTLINE
TIMELINE
JoinListScreen.js
Settings
SelectOptionCard.js
ToMapScreen.js
Screens > ToMapScreen.js > ...
import { createStackNavigator } from '@react-navigation/stack';
const ToMapScreen = () => {
const Stack= createStackNavigator();
return (
<View>
<View style={tw` h-1/2`}>
<Map />
</View>
<View style={tw`h-1/2`}>
<Stack.Navigator>
<Stack.Screen
name="MapNavigationCard"
component={MapNavigationCard}
options={{
headerShown: false,
}}
/>
<Stack.Screen
name="ScheduleTimeCard"
component={ScheduleTimeCard}
options={{
headerShown: false,
}}
/>
<Stack.Screen
name="SelectOptionCard"
component={SelectOptionCard}
options={{
headerShown: false,
}}
/>
</Stack.Navigator>
</View>
</View>
)
}

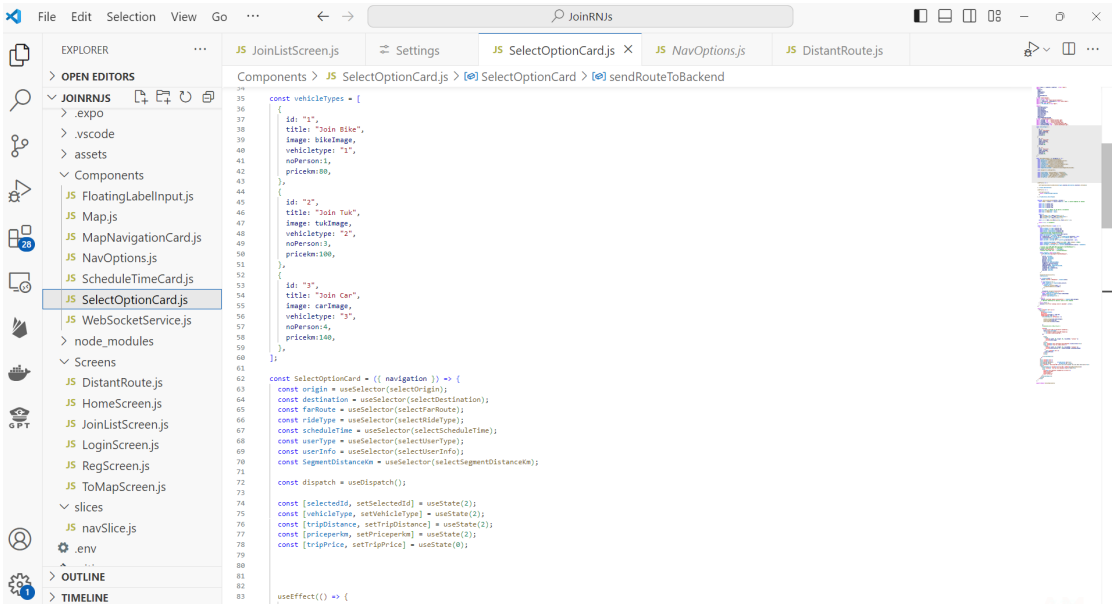

File Edit Selection View Go
JoinRNJs
EXPLORER
OPEN EDITORS
JOINRNJS
.expo
.vscode
assets
Components
FloatingLabelInput.js
Map.js
MapNavigationCard.js
NavOptions.js
ScheduleTimeCard.js
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
OUTLINE
JoinListScreen.js
Settings
SelectOptionCard.js
Map.js
Components > Map.js > ...
const Map = () => {
const origin = useSelector(selectOrigin);
const destination= useSelector(selectDestination);
const mapRef =useRef(null);
useEffect(() =>{
if(!origin || !destination) return;
mapRef.current.fitToSuppliedMarkers(["origin","destination"],
{
edgePadding: {top:50, righ:50, bottom:50, left: 50}
}
);
}, [origin,destination]);
return (
<MapView
ref= {mapRef}
style={tw`flex-1`}
mapType='mutedStandard'
initialRegion={{
latitude: origin.location.lat,
longitude: origin.location.lng,
latitudeDelta: 0.005,
longitudeDelta: 0.005,
}}
>
{origin && destination &&(
<MapViewDirections
origin={origin.description}
destination = {destination.description}
apikey= {GOOGLE_MAPS_APIKEY}
strokeWidth={3}
strokeColor="black"
/>
)}
{origin?.location && (
<Marker
coordinate={{
latitude: origin.location.lat,
longitude: origin.location.lng,
}}
title="Origin"
description={origin.description}
identifier="origin"

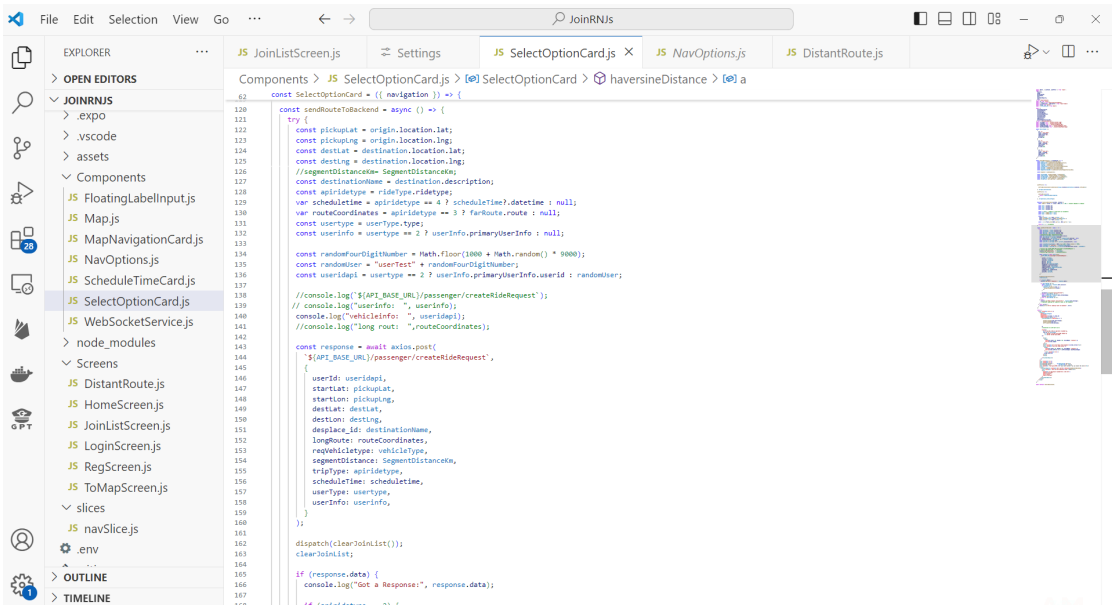

File Edit Selection View Go
JoinRNJs
EXPLORER
OPEN EDITORS
JOINRNJS
.expo
.vscode
assets
Components
FloatingLabelInput.js
Map.js
MapNavigationCard.js
NavOptions.js
ScheduleTimeCard.js
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
OUTLINE
JoinListScreen.js
Settings
SelectOptionCard.js
DistantRoute.js
Screens > DistantRoute.js > DistantRoute > useEffect() callback > <function>
const DistantRoute = () => {
const origin = useSelector(selectOrigin);
const destination = useSelector(selectDestination);
const dispatch = useDispatch();
const currentLocation = {
latitude: origin.location.lat,
longitude: origin.location.lng,
};
useEffect(() => {
(async () => {
if (!origin || !destination) return;
mapRef.current.fitToSuppliedMarkers(["origin", "destination"], {
edgePadding: { top: 50, righ: 50, bottom: 50, left: 50 },
});
if (destination) {
getRoute(
origin.location.lat,
origin.location.lng,
destination.location.lat,
destination.location.lng
);
}
})();
}, [origin, destination]);
const getRoute = async (pickupLat, pickupLng, destLat, destLng) => {
try {
//console.log("new " + origin.location.lat + " " + origin.location.lng + " destinaiton:=> " + destination.location.lat);
const response = await axios.get(
`https://maps.googleapis.com/maps/api/directions/json?origin=${pickupLat},${pickupLng}&destination=${destLat},${destLng}&key=${GOOGLE_MAPS_APIKEY}`
);
if (response.data.routes.length > 0) {
const tripdistance = response.data.routes[0].legs[0].distance;
dispatch(setTripDistanceKm({
distance: tripdistance.value,
text: tripdistance.text
}))
// console.log('Distance in meters:', distance.value);
} else {

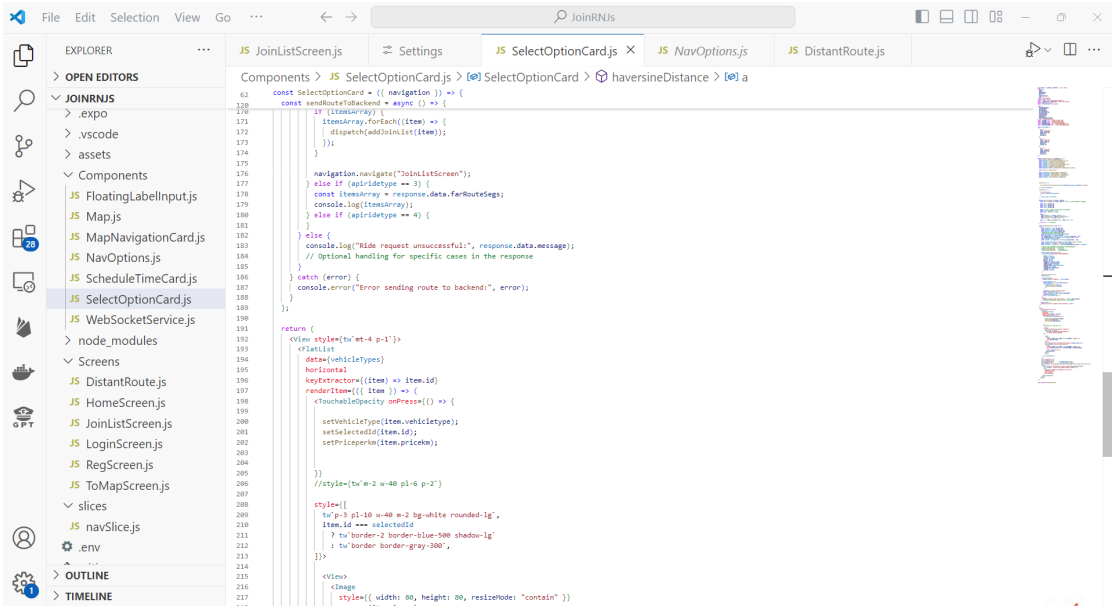

File Edit Selection View Go
JoinRNJs
EXPLORER
OPEN EDITORS
JOINRNJS
.expo
.vscode
assets
Components
FloatingLabelInput.js
Map.js
MapNavigationCard.js
NavOptions.js
ScheduleTimeCard.js
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
OUTLINE
TIMELINE
JoinListScreen.js
Settings
SelectOptionCard.js
DistantRoute.js
Screens > DistantRoute.js > DistantRoute > useEffect() callback > <function>
const DistantRoute = () => {
return (
<View>
<View style={tw` h-1/2`}>
<MapView
ref={mapRef}
style={tw`flex-1`}
mapType="mutedStandard"
initialRegion={{
latitude: origin.location.lat,
longitude: origin.location.lng,
latitudeDelta: 0.005,
longitudeDelta: 0.005,
}}
>
{origin?.location && (
<Marker
coordinate={{
latitude: origin.location.lat,
longitude: origin.location.lng,
}}
title="Origin"
description={origin.description}
identifier="origin"
/>
)}
{destination?.location && (
<Marker
coordinate={{
latitude: destination.location.lat,
longitude: destination.location.lng,
}}
title="Destination"
description={origin.description}
identifier="destination"
/>
)}
{/* Polyline for the Route */}
{route.length > 0 && (
<Polyline coordinates={route} strokeWidth={5} strokeColor="blue" />
)}
</MapView>
</View>
<View >
<TouchableOpacity
style={tw`mt-0 mx-0`}
onPress={goToCurrentLocation}
>


File Edit Selection View Go
JoinRNJs
EXPLORER
OPEN EDITORS
JOINRNJS
.expo
.vscode
assets
Components
FloatingLabelInput.js
Map.js
MapNavigationCard.js
NavOptions.js
ScheduleTimeCard.js
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
OUTLINE
TIMELINE
JoinListScreen.js
Settings
SelectOptionCard.js
DistantRoute.js
MapNavigationCard.js
Components > MapNavigationCard.js > MapNavigationCard > <function>
const MapNavigationCard = () => {
const navigation = useNavigation();
const rideType = useSelector(selectRideType);
const destination = useSelector(selectDestination);
const segmentDistancekm = useSelector(selectSegmentDistancekm);
const tripdistance = useSelector(selectTripDistanceKm);
const rtype = rideType.ridetype;
const handlePress = () => {
if (!destination) {
Alert.alert("No Destination Selected.");
return;
} else {
if (rtype == 4) {
navigation.navigate("ScheduleTimeCard");
} else if(rtype == 3 && (tripdistance.distance/1000) < segmentDistancekm){
Alert.alert("Your trip is Less than expected for this Service. Please Select distance of Greater that 10km.");
return;
}else {
navigation.navigate("SelectOptionCard");
}
}
};
return (
<View>
<Text style={tw`text-center py-5 text-xl`}>Good Day</Text>
<View>
<GooglePlacesAutocomplete
styles={{
container: { flex: 0 },
textInput: { fontSize: 18, backgroundColor: "#F5F5F5" },
textInputContainer: { paddingHorizontal: 20, paddingBottom: 1 },
}}
fetchDetails={true}
returnKeyType={"search"}
query={{
key: GOOGLE_MAPS_APIKEY,
language: "en",
}}
onPress={(data, details = null) => {
dispatch(
setDestination({
location: details.geometry.location,
description: data.description,
})
);
}}

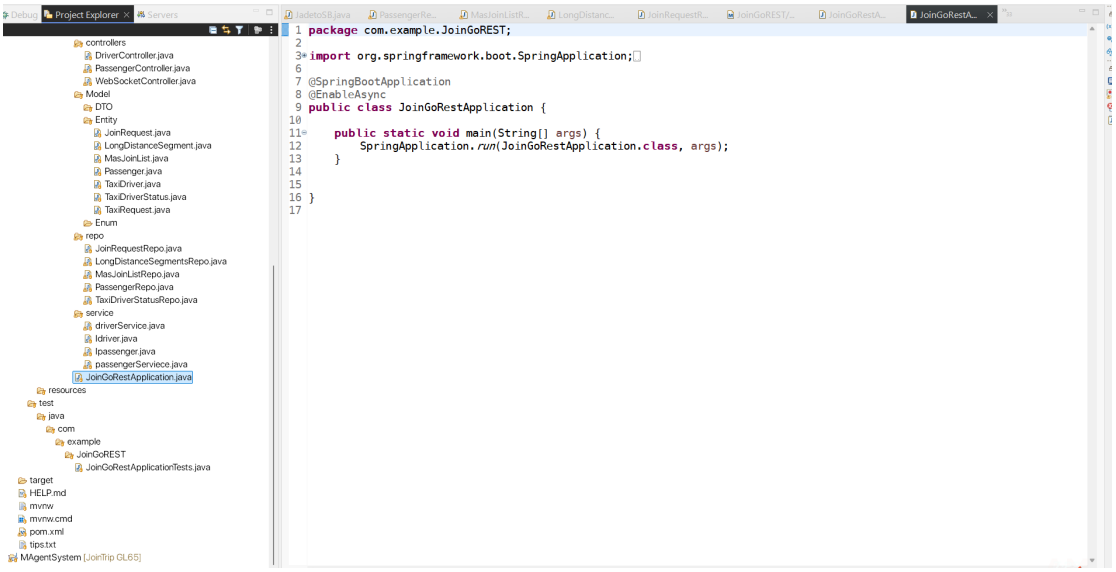

File Edit Selection View Go
JoinRNJs
EXPLORER
JoinListScreen.js
Settings
SelectOptionCard.js
ScheduleTimeCard.js
DistantRoute.js
Components > ScheduleTimeCard.js > ScheduleTimeCard > handlesubmit
OPEN EDITORS
JOINRNJS
.expo
.vscode
assets
Components
FloatingLabelInput.js
Map.js
MapNavigationCard.js
NavOptions.js
ScheduleTimeCard.js
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
OUTLINE
TIMELINE

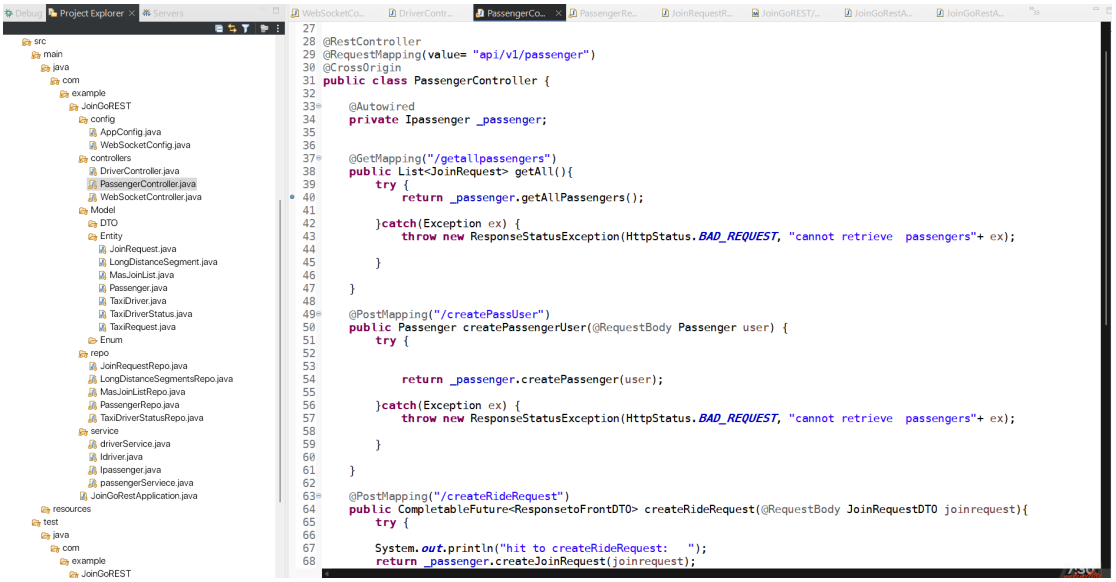

File Edit Selection View Go
JoinRNJs
EXPLORER
JoinListScreen.js
Settings
SelectOptionCard.js
NavOptions.js
DistantRoute.js
Components > SelectOptionCard.js > SelectOptionCard > sendRouteToBackend
OPEN EDITORS
JOINRNJS
.expo
.vscode
assets
Components
FloatingLabelInput.js
Map.js
MapNavigationCard.js
NavOptions.js
ScheduleTimeCard.js
SelectOptionCard.js
WebSocketService.js
node_modules
Screens
DistantRoute.js
HomeScreen.js
JoinListScreen.js
LoginScreen.js
RegScreen.js
ToMapScreen.js
slices
navSlice.js
.env
OUTLINE
TIMELINE

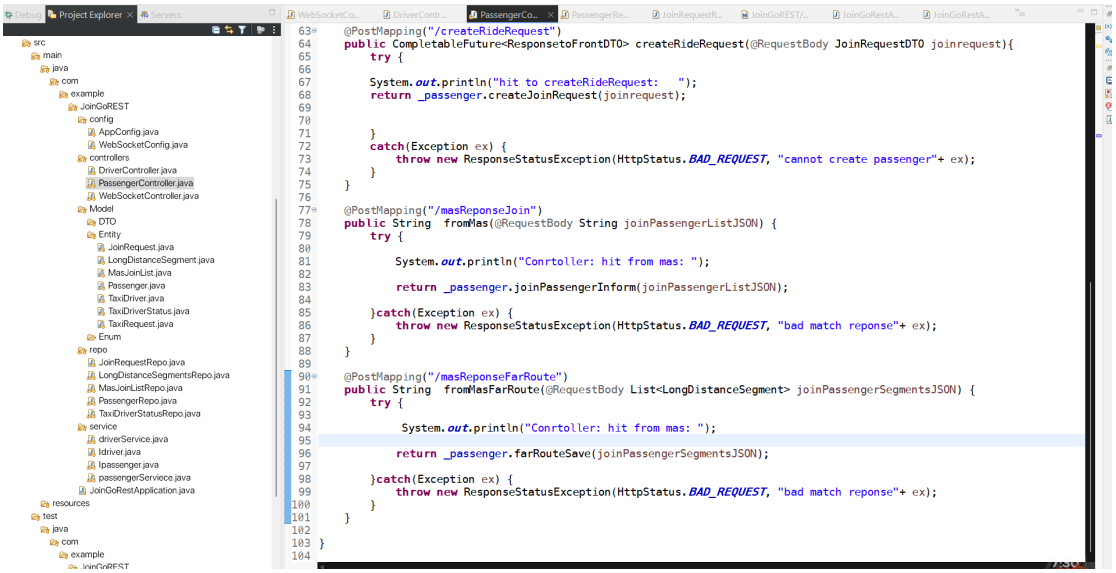


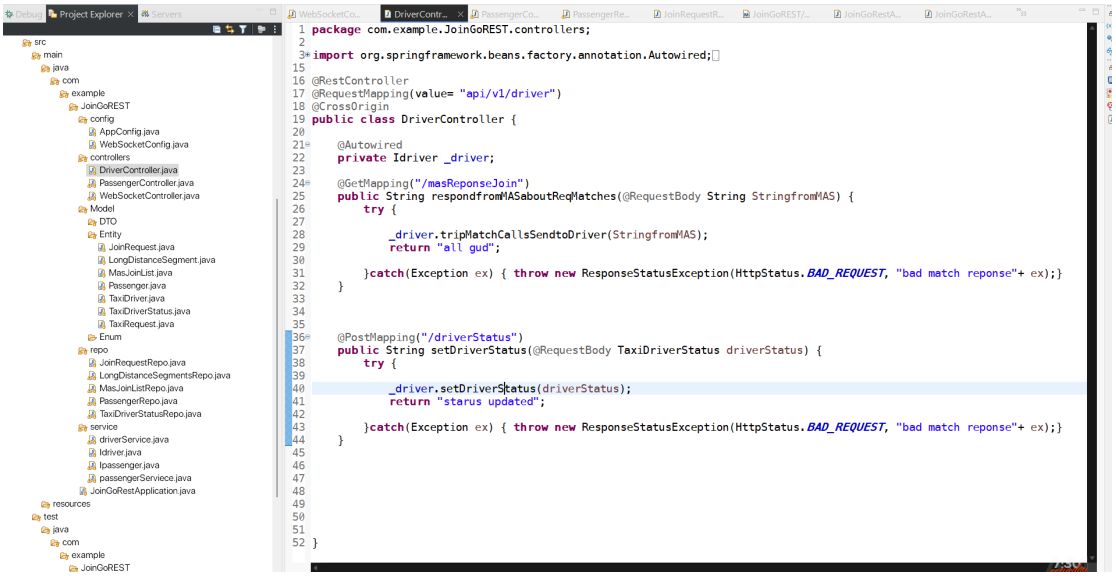


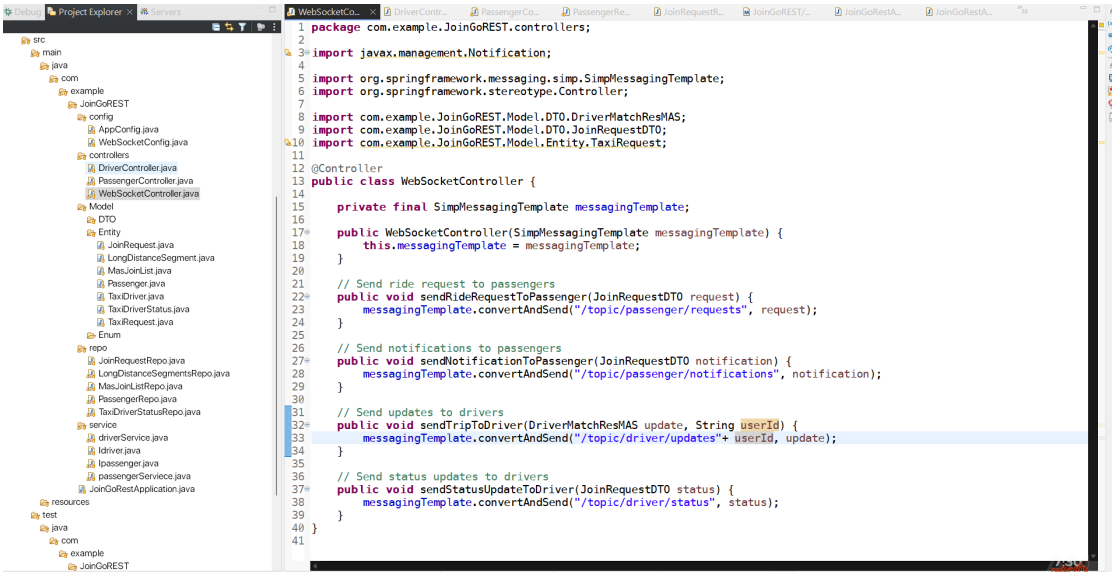
- Rest Back-end:
  - Business Layer-

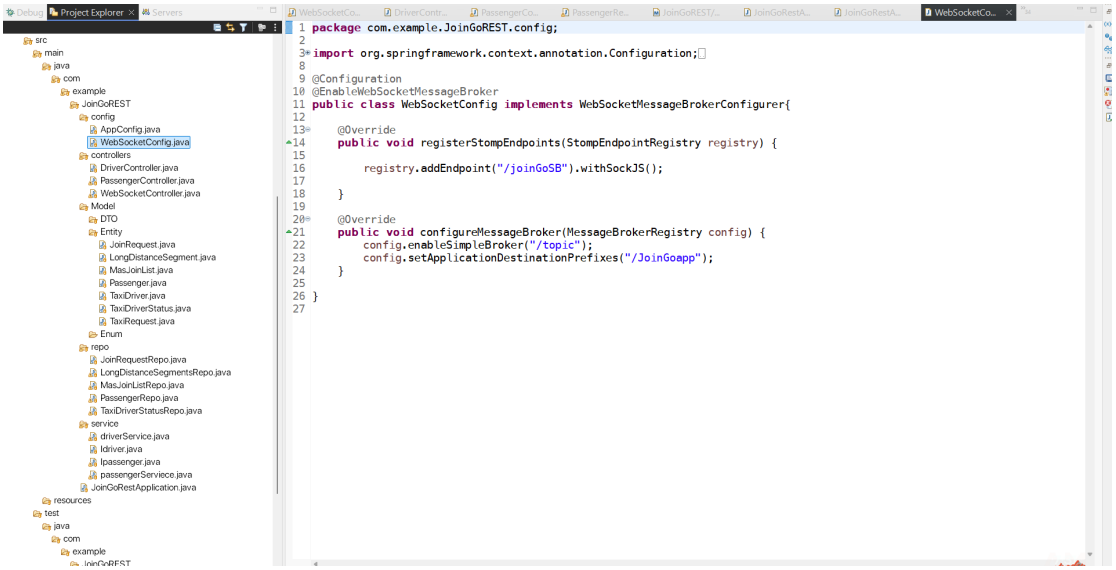

Project Explorer
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
HELP.md
mvnw
mvnw.cmd
pom.xml
tips.txt
MAgentSystem [JoinTrip GL65]
package com.example.JoinGoREST;
import org.springframework.boot.SpringApplication;
@SpringBootApplication
@EnableAsync
public class JoinGoRestApplication {
public static void main(String[] args) {
SpringApplication.run(JoinGoRestApplication.class, args);
}
}

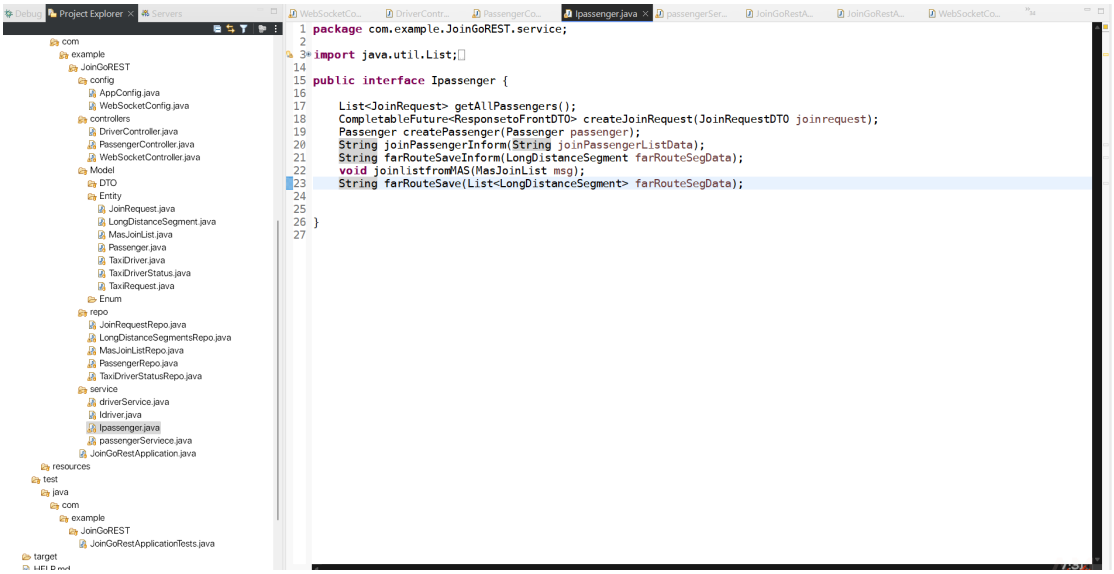

Project Explorer
src
main
java
com
example
JoinGoREST
config
AppConfig.java
WebSocketConfig.java
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
@RestController
@RequestMapping(value= "api/v1/passenger")
@CrossOrigin
public class PassengerController {
@Autowired
private Ipassenger _passenger;
@GetMapping("/getallpassengers")
public List<JoinRequest> getAll(){
try {
return _passenger.getAllPassengers();
}catch(Exception ex) {
throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "cannot retrieve passengers"+ ex);
}
}
@PostMapping("/createPassUser")
public Passenger createPassengerUser(@RequestBody Passenger user) {
try {
return _passenger.createPassenger(user);
}catch(Exception ex) {
throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "cannot retrieve passengers"+ ex);
}
}
@PostMapping("/createRideRequest")
public CompletableFuture<ResponsetoFrontDTO> createRideRequest(@RequestBody JoinRequestDTO joinrequest){
try {
System.out.println("hit to createRideRequest: ");
return _passenger.createJoinRequest(joinrequest);

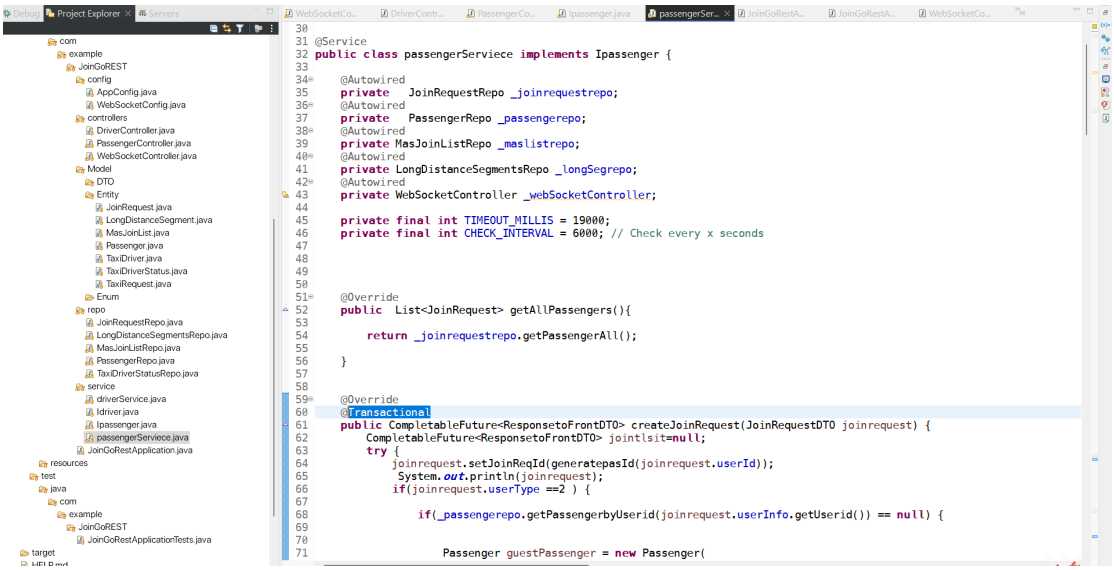

Project Explorer
src
main
java
com
example
JoinGoREST
config
AppConfig.java
WebSocketConfig.java
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
@PostMapping("/createRideRequest")
public CompletableFuture<ResponsetoFrontDTO> createRideRequest(@RequestBody JoinRequestDTO joinrequest){
try {
System.out.println("hit to createRideRequest: ");
return _passenger.createJoinRequest(joinrequest);
}
catch(Exception ex) {
throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "cannot create passenger"+ ex);
}
}
@PostMapping("/masReponseJoin")
public String fromMas(@RequestBody String joinPassengerListJSON) {
try {
System.out.println("Conrtoller: hit from mas: ");
return _passenger.joinPassengerInform(joinPassengerListJSON);
}catch(Exception ex) {
throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "bad match reponse"+ ex);
}
}
@PostMapping("/masReponseFarRoute")
public String fromMasFarRoute(@RequestBody List<LongDistanceSegment> joinPassengerSegmentsJSON) {
try {
System.out.println("Conrtoller: hit from mas: ");
return _passenger.farRouteSave(joinPassengerSegmentsJSON);
}catch(Exception ex) {
throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "bad match reponse"+ ex);
}
}
}

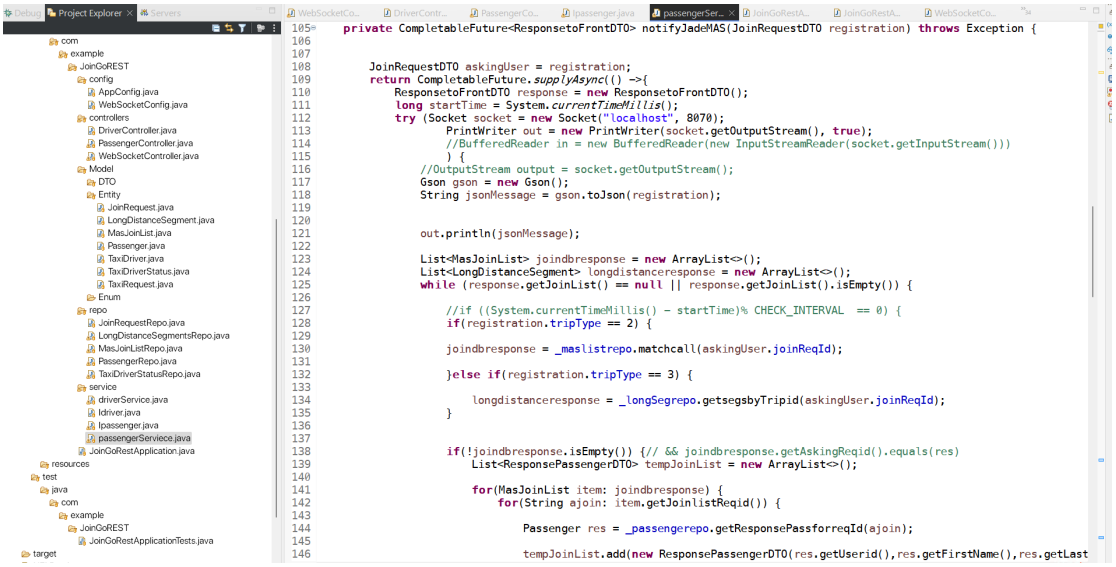

Project Explorer
src
main
java
com
example
JoinGoREST
config
AppConfig.java
WebSocketConfig.java
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
package com.example.JoinGoREST.controllers;
import org.springframework.beans.factory.annotation.Autowired;
@RestController
@RequestMapping(value= "api/v1/driver")
@CrossOrigin
public class DriverController {
@Autowired
private Idriver _driver;
@GetMapping("/masReponseJoin")
public String respondfromMASaboutReqMatches(@RequestBody String StringfromMAS) {
try {
_driver.tripMatchCallsSendtoDriver(StringfromMAS);
return "all gud";
}catch(Exception ex) { throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "bad match reponse"+ ex);}
}
@PostMapping("/driverStatus")
public String setDriverStatus(@RequestBody TaxiDriverStatus driverStatus) {
try {
_driver.setDriverStatus(driverStatus);
return "starus updated";
}catch(Exception ex) { throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "bad match reponse"+ ex);}
}
}

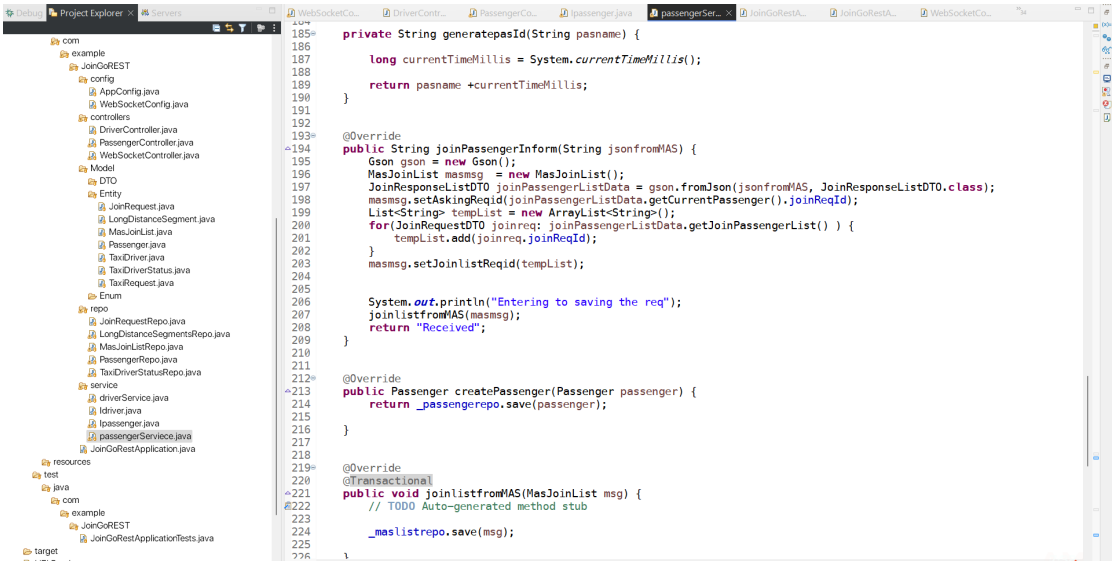

Project Explorer
src
main
java
com
example
JoinGoREST
config
AppConfig.java
WebSocketConfig.java
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
WebSocketCo...
package com.example.JoinGoREST.controllers;
import javax.management.Notification;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.stereotype.Controller;
import com.example.JoinGoREST.Model.DTO.DriverMatchResMAS;
import com.example.JoinGoREST.Model.DTO.JoinRequestDTO;
import com.example.JoinGoREST.Model.Entity.TaxiRequest;
@Controller
public class WebSocketController {
private final SimpMessagingTemplate messagingTemplate;
public WebSocketController(SimpMessagingTemplate messagingTemplate) {
this.messagingTemplate = messagingTemplate;
}
// Send ride request to passengers
public void sendRideRequestToPassenger(JoinRequestDTO request) {
messagingTemplate.convertAndSend("/topic/passenger/requests", request);
}
// Send notifications to passengers
public void sendNotificationToPassenger(JoinRequestDTO notification) {
messagingTemplate.convertAndSend("/topic/passenger/notifications", notification);
}
// Send updates to drivers
public void sendTripToDriver(DriverMatchResMAS update, String userId) {
messagingTemplate.convertAndSend("/topic/driver/updates"+ userId, update);
}
// Send status updates to drivers
public void sendStatusUpdateToDriver(JoinRequestDTO status) {
messagingTemplate.convertAndSend("/topic/driver/status", status);
}
}

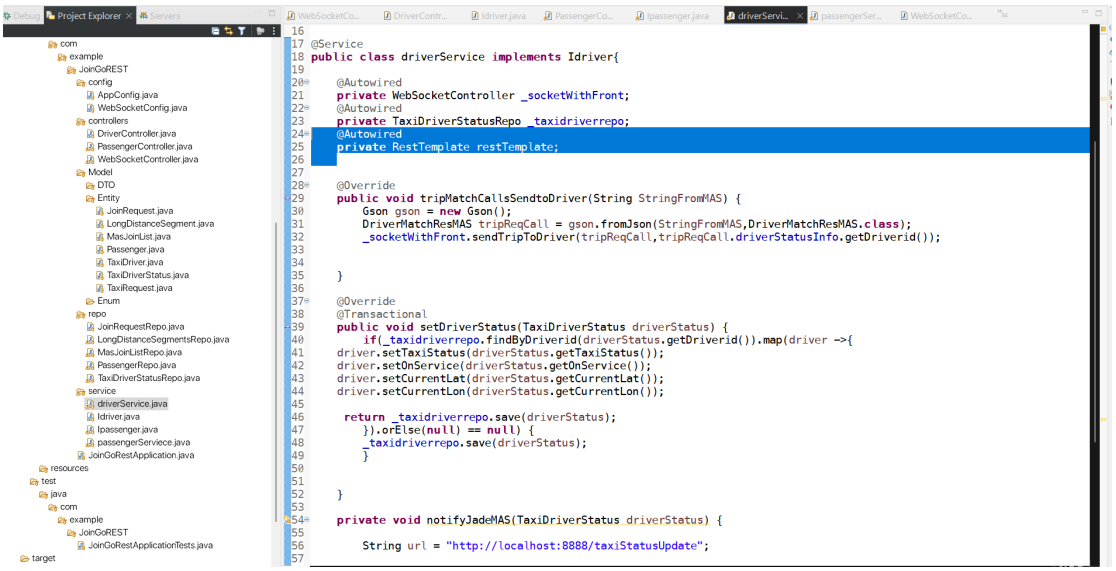

Project Explorer
src
main
java
com
example
JoinGoREST
config
AppConfig.java
WebSocketConfig.java
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
WebSocketCo...
package com.example.JoinGoREST.config;
import org.springframework.context.annotation.Configuration;
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer{
@Override
public void registerStompEndpoints(StompEndpointRegistry registry) {
registry.addEndpoint("/joinGoSB").withSockJS();
}
@Override
public void configureMessageBroker(MessageBrokerRegistry config) {
config.enableSimpleBroker("/topic");
config.setApplicationDestinationPrefixes("/JoinGoapp");
}
}


Project Explorer
com
example
JoinGoREST
config
AppConfig.java
WebSocketConfig.java
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
Ipassenger.java
package com.example.JoinGoREST.service;
import java.util.List;
public interface Ipassenger {
List<JoinRequest> getAllPassengers();
CompletableFuture<ResponsetoFrontDTO> createJoinRequest(JoinRequestDTO joinrequest);
Passenger createPassenger(Passenger passenger);
String joinPassengerInform(String joinPassengerListData);
String farRouteSaveInform(LongDistanceSegment farRouteSegData);
void joinlistfromMAS(MasJoinList msg);
String farRouteSave(List<LongDistanceSegment> farRouteSegData);
}

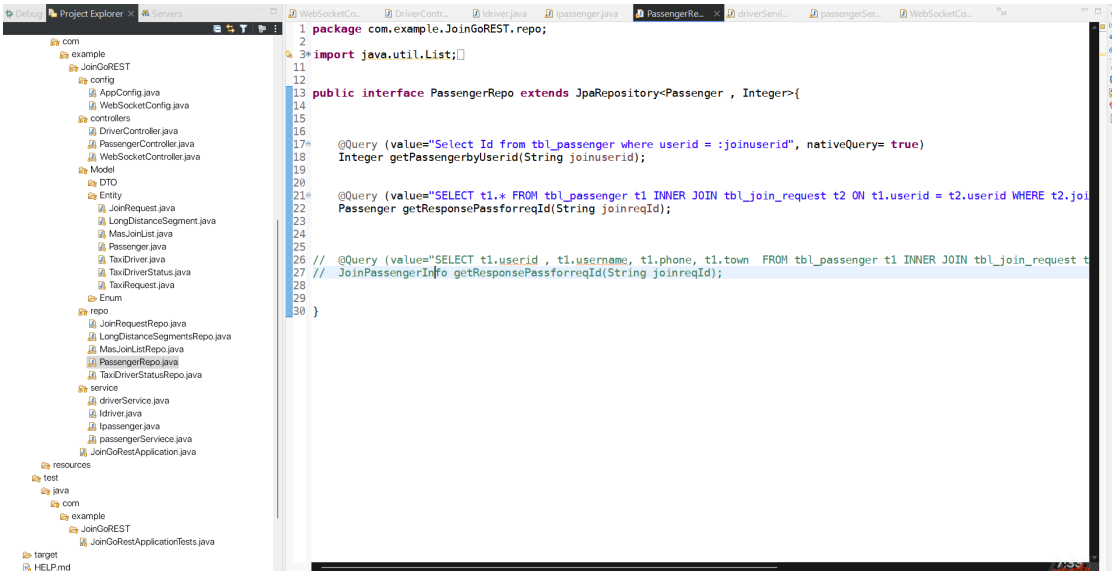

Project Explorer
com
example
JoinGoREST
config
AppConfig.java
WebSocketConfig.java
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
@Service
public class passengerServiece implements Ipassenger {
@Autowired
private JoinRequestRepo _joinrequestrepo;
@Autowired
private PassengerRepo _passengerepo;
@Autowired
private MasJoinListRepo _maslistrepo;
@Autowired
private LongDistanceSegmentsRepo _longSegrepo;
@Autowired
private WebSocketController _webSocketController;
private final int TIMEOUT_MILLIS = 19000;
private final int CHECK_INTERVAL = 6000; // Check every x seconds
@Override
public List<JoinRequest> getAllPassengers(){
return _joinrequestrepo.getPassengerAll();
}
@Override
@Transactional
public CompletableFuture<ResponsetoFrontDTO> createJoinRequest(JoinRequestDTO joinrequest) {
CompletableFuture<ResponsetoFrontDTO> jointlsit=null;
try {
joinrequest.setJoinReqId(generatepasId(joinrequest.userId));
System.out.println(joinrequest);
if(joinrequest.userType ==2) {
if(_passengerepo.getPassengerbyUserid(joinrequest.userInfo.getUserid()) == null) {
Passenger guestPassenger = new Passenger(

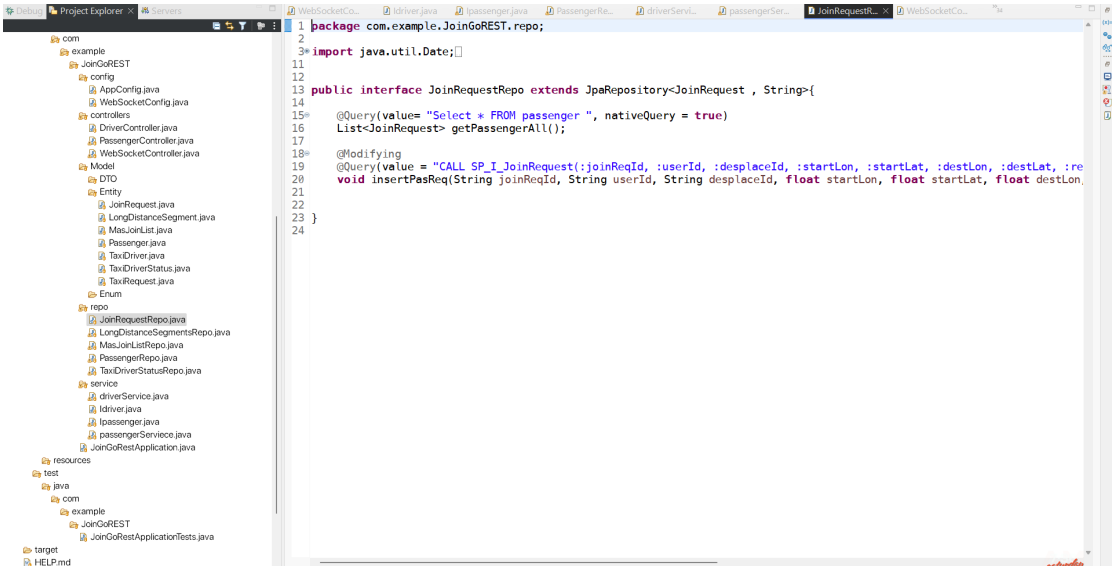

Project Explorer
com
example
JoinGoREST
config
AppConfig.java
WebSocketConfig.java
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
private CompletableFuture<ResponsetoFrontDTO> notifyJadeMAS(JoinRequestDTO registration) throws Exception {
JoinRequestDTO askingUser = registration;
return CompletableFuture.supplyAsync(() ->{
ResponsetoFrontDTO response = new ResponsetoFrontDTO();
long startTime = System.currentTimeMillis();
try (Socket socket = new Socket("localhost", 8070);
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
//BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()))
) {
//OutputStream output = socket.getOutputStream();
Gson gson = new Gson();
String jsonMessage = gson.toJson(registration);
out.println(jsonMessage);
List<MasJoinList> joindbresponse = new ArrayList<>();
List<LongDistanceSegment> longdistanceresponse = new ArrayList<>();
while (response.getJoinList() == null || response.getJoinList().isEmpty()) {
//if ((System.currentTimeMillis() - startTime)% CHECK_INTERVAL == 0) {
if(registration.tripType == 2) {
joindbresponse = _maslistrepo.matchcall(askingUser.joinReqId);
}else if(registration.tripType == 3) {
longdistanceresponse = _longSegrepo.getsegsbyTripid(askingUser.joinReqId);
}
if(!joindbresponse.isEmpty()) {// && joindbresponse.getAskingReqid().equals(res)
List<ResponsePassengerDTO> tempJoinList = new ArrayList<>();
for(MasJoinList item: joindbresponse) {
for(String ajoin: item.getJoinlistReqid()) {
Passenger res = _passengerepo.getResponsePassforreqId(ajoin);
tempJoinList.add(new ResponsePassengerDTO(res.getUserid(),res.getFirstName(),res.getLast

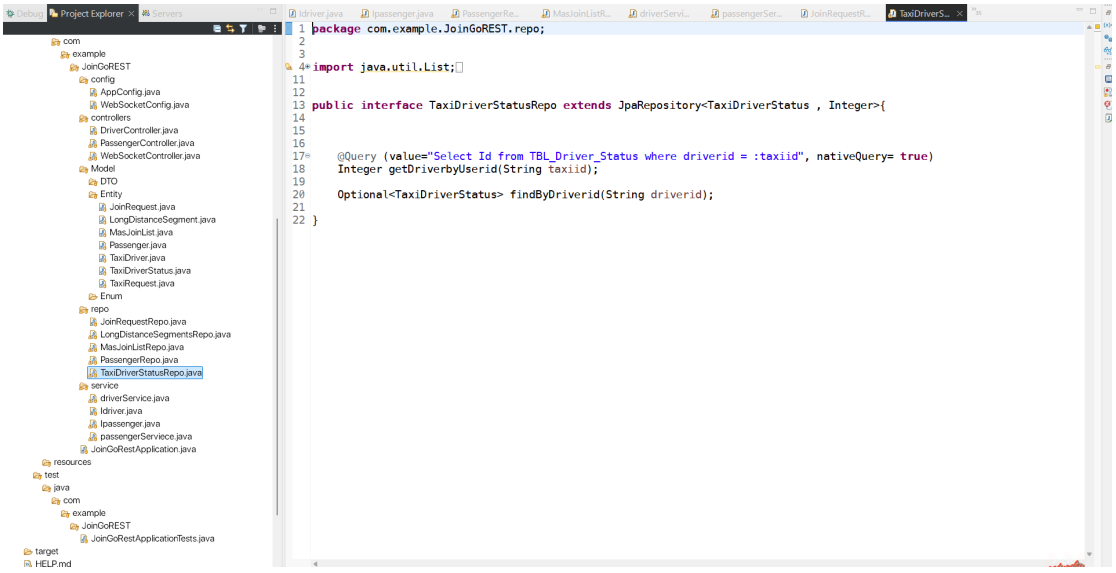

Project Explorer
com
example
JoinGoREST
config
AppConfig.java
WebSocketConfig.java
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
private String generatepasId(String pasname) {
long currentTimeMillis = System.currentTimeMillis();
return pasname +currentTimeMillis;
}
@Override
public String joinPassengerInform(String jsonfromMAS) {
Gson gson = new Gson();
MasJoinList masmsg = new MasJoinList();
JoinResponseListDTO joinPassengerListData = gson.fromJson(jsonfromMAS, JoinResponseListDTO.class);
masmsg.setAskingReqid(joinPassengerListData.getCurrentPassenger().joinReqId);
List<String> tempList = new ArrayList<String>();
for(JoinRequestDTO joinreq: joinPassengerListData.getJoinPassengerList()) {
tempList.add(joinreq.joinReqId);
}
masmsg.setJoinlistReqid(tempList);
System.out.println("Entering to saving the req");
joinlistfromMAS(masmsg);
return "Received";
}
@Override
public Passenger createPassenger(Passenger passenger) {
return _passengerepo.save(passenger);
}
@Override
@Transactional
public void joinlistfromMAS(MasJoinList msg) {
// TODO Auto-generated method stub
_maslistrepo.save(msg);

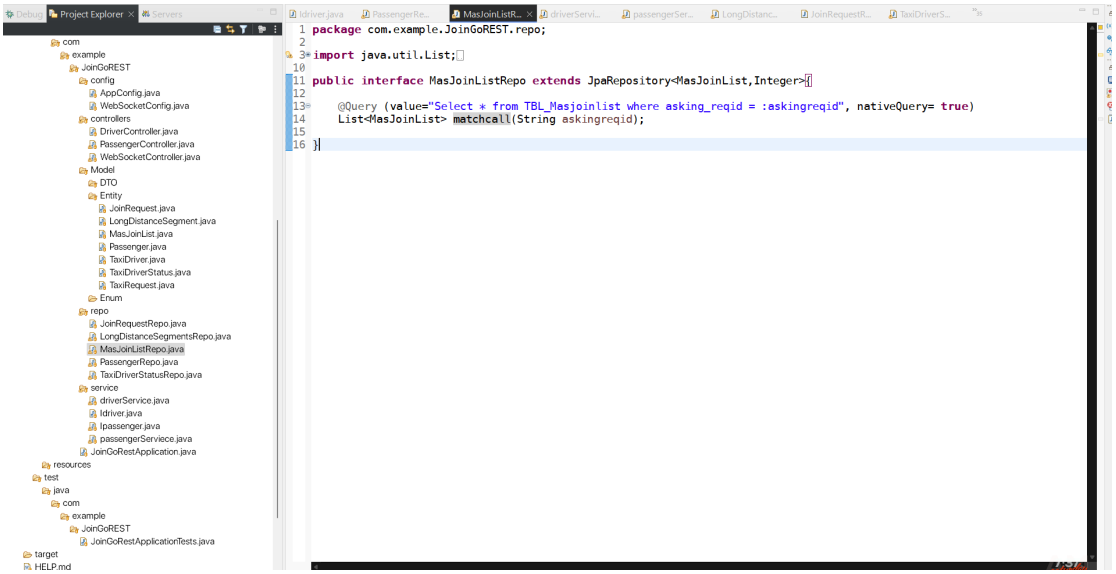

Project Explorer
com
example
JoinGoREST
config
AppConfig.java
WebSocketConfig.java
controllers
DriverController.java
PassengerController.java
WebSocketController.java
Model
DTO
Entity
JoinRequest.java
LongDistanceSegment.java
MasJoinList.java
Passenger.java
TaxiDriver.java
TaxiDriverStatus.java
TaxiRequest.java
Enum
repo
JoinRequestRepo.java
LongDistanceSegmentsRepo.java
MasJoinListRepo.java
PassengerRepo.java
TaxiDriverStatusRepo.java
service
driverService.java
Idriver.java
Ipassenger.java
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
@Service
public class driverService implements Idriver{
@Autowired
private WebSocketController _socketWithFront;
@Autowired
private TaxiDriverStatusRepo _taxidriverrepo;
@Autowired
private RestTemplate restTemplate;
@Override
public void tripMatchCallsSendtoDriver(String StringFromMAS) {
Gson gson = new Gson();
DriverMatchResMAS tripReqCall = gson.fromJson(StringFromMAS,DriverMatchResMAS.class);
_socketWithFront.sendTripToDriver(tripReqCall,tripReqCall.driverStatusInfo.getDriverid());
}
@Override
@Transactional
public void setDriverStatus(TaxiDriverStatus driverStatus) {
if(_taxidriverrepo.findByDriverid(driverStatus.getDriverid()).map(driver ->{
driver.setTaxiStatus(driverStatus.getTaxiStatus());
driver.setOnService(driverStatus.getOnService());
driver.setCurrentLat(driverStatus.getCurrentLat());
driver.setCurrentLon(driverStatus.getCurrentLon());
return _taxidriverrepo.save(driverStatus);
}).orElse(null) == null) {
_taxidriverrepo.save(driverStatus);
}
}
private void notifyJadeMAS(TaxiDriverStatus driverStatus) {
String url = "http://localhost:8888/taxiStatusUpdate";

- Persistence Layer-

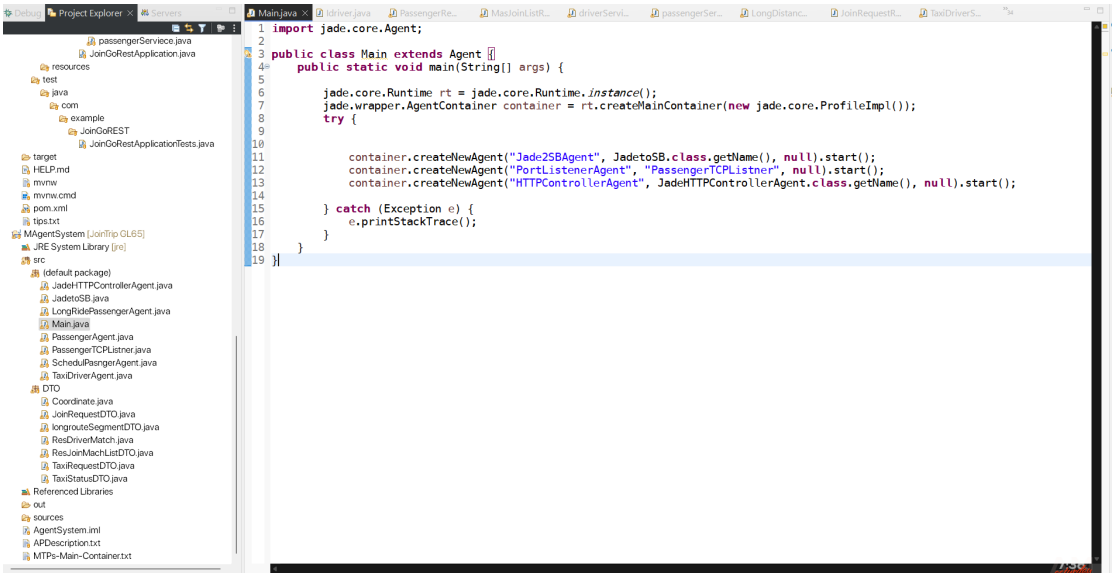






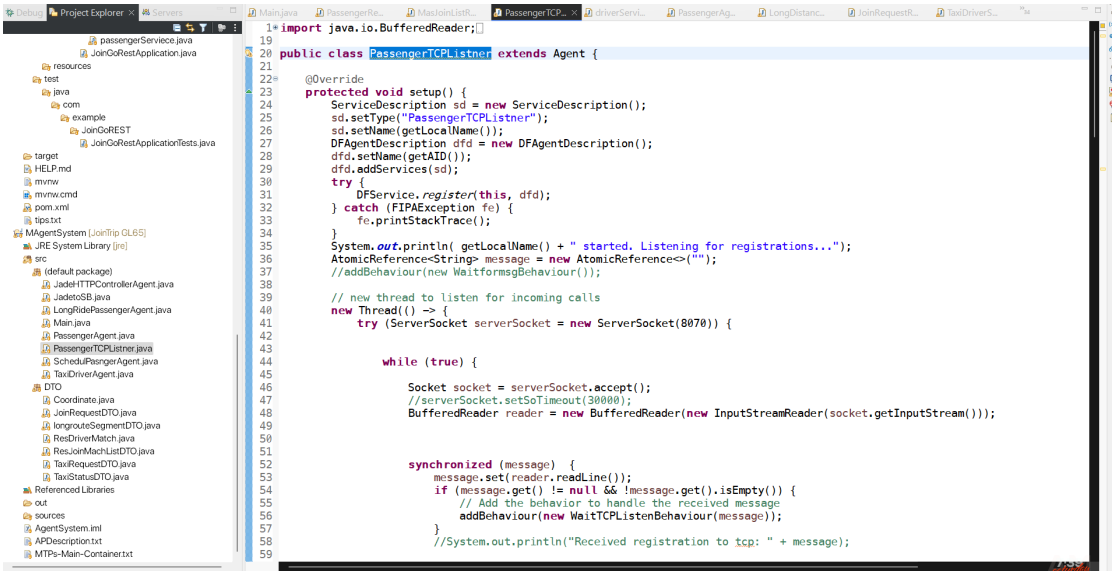


- JADE MAS:
  - Main



```
import jade.core.Agent;

public class Main extends Agent {
    public static void main(String[] args) {

        jade.core.Runtime rt = jade.core.Runtime.instance();
        jade.wrapper.AgentContainer container = rt.createMainContainer(new jade.core.ProfileImpl());
        try {


            container.createNewAgent("Jade2SBAgent", JadetoSB.class.getName(), null).start();
            container.createNewAgent("PortListenerAgent", "PassengerTCPListner", null).start();
            container.createNewAgent("HTTPControllerAgent", JadeHTTPControllerAgent.class.getName(), null).start();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

- PassengerTCPListner Agent class-

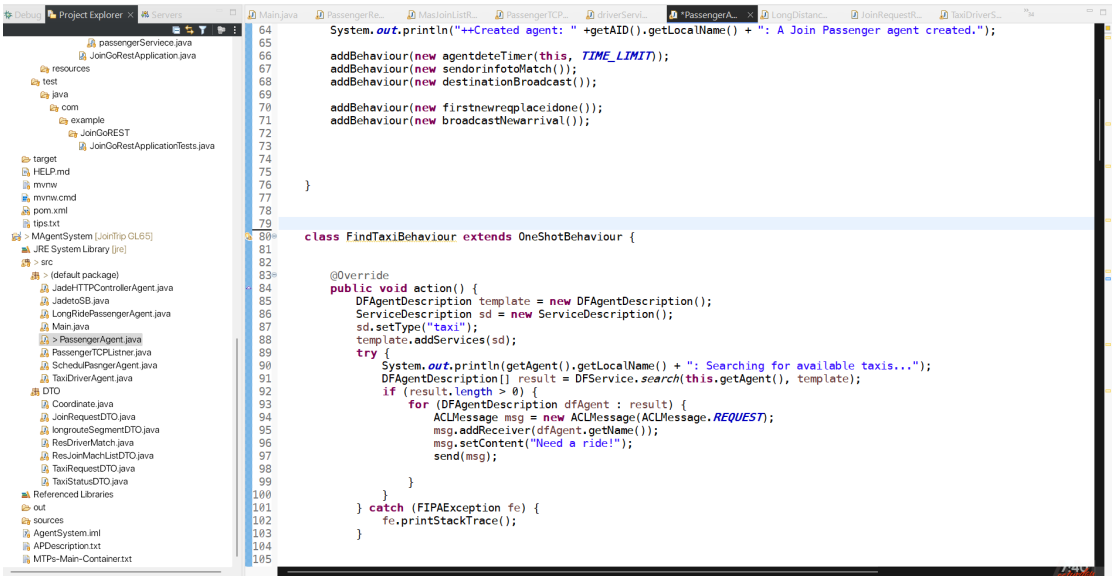


```
import java.io.BufferedReader;

public class PassengerTCPListner extends Agent {

    @Override
    protected void setup() {
        ServiceDescription sd = new ServiceDescription();
        sd.setType("PassengerTCPListner");
        sd.setName(getLocalName());
        DFAgentDescription dfd = new DFAgentDescription();
        dfd.setName(getAID());
        dfd.addServices(sd);
        try {
            DFService.register(this, dfd);
        } catch (FIPAException fe) {
            fe.printStackTrace();
        }
        System.out.println( getLocalName() + " started. Listening for registrations...");
        AtomicReference<String> message = new AtomicReference<>("");
        //addBehaviour(new WaitformsgBehaviour());

        // new thread to listen for incoming calls
        new Thread(() -> {
            try (ServerSocket serverSocket = new ServerSocket(8070)) {


                while (true) {

                    Socket socket = serverSocket.accept();
                    //serverSocket.setSoTimeout(30000);
                    BufferedReader reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));


                    synchronized (message)  {
                        message.set(reader.readLine());
                        if (message.get() != null && !message.get().isEmpty()) {
                            // Add the behavior to handle the received message
                            addBehaviour(new WaitTCPListenBehaviour(message));
                        }
                        //System.out.println("Received registration to tcp: " + message);
```

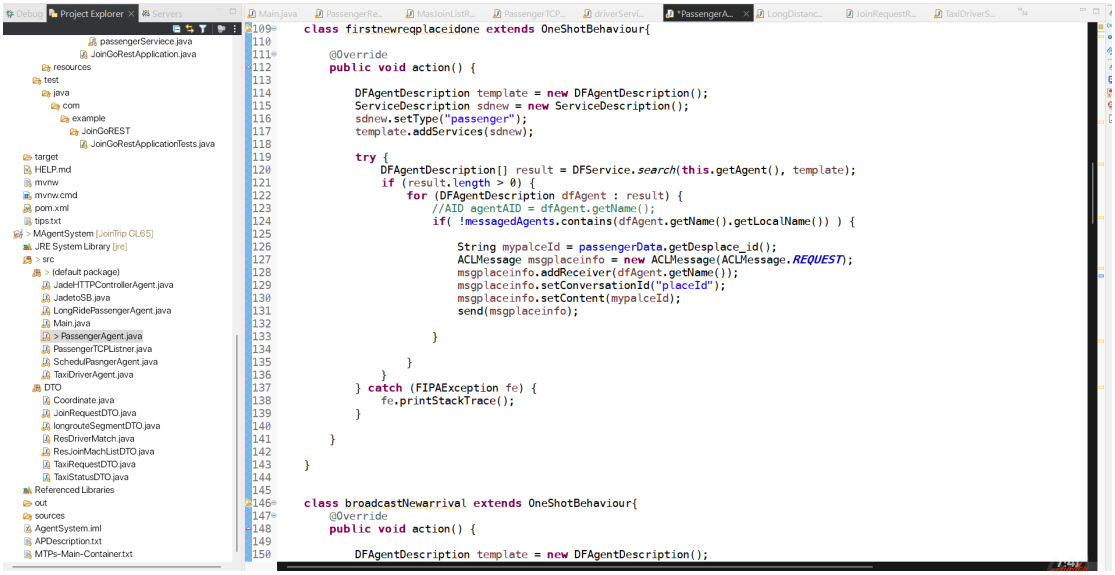
```
class WaitTCPListenBehaviour extends OneShotBehaviour {

    private AtomicReference<String> message;

    public WaitTCPListenBehaviour(AtomicReference<String> message){
        this.message =message;
    }

    @Override
    public void action(){
        if(this.message != null && !this.message.get().isEmpty()){

            Gson gson = new Gson();


            JoinRequestDTO passengerData = gson.fromJson(this.message.get(), JoinRequestDTO.class);

            createPassengerAgent(passengerData);

        }else {
            System.out.println("Message is null or empty");
        }


    }

    private void createPassengerAgent(JoinRequestDTO passenger) {
        try {
            // Get the current container
            AgentContainer container = getContainerController();

            Object[] args = new Object[] { passenger };
            if(passenger.getTripType() == 2) {
                AgentController passengerAgent = container.createNewAgent(passenger.getJoinReqId(), PassengerAgent.class.g
                passengerAgent.start();
            }else if(passenger.getTripType() == 4){
                AgentController schedulepassengerAgent = container.createNewAgent(passenger.getJoinReqId(), SchedulPasnger
                schedulepassengerAgent.start();
            }else if(passenger.getTripType() == 3) {
                AgentController longridepassengerAgent = container.createNewAgent(passenger.getJoinReqId(), LongRidePassen
                longridepassengerAgent.start();
```

- PassengerAgent class-

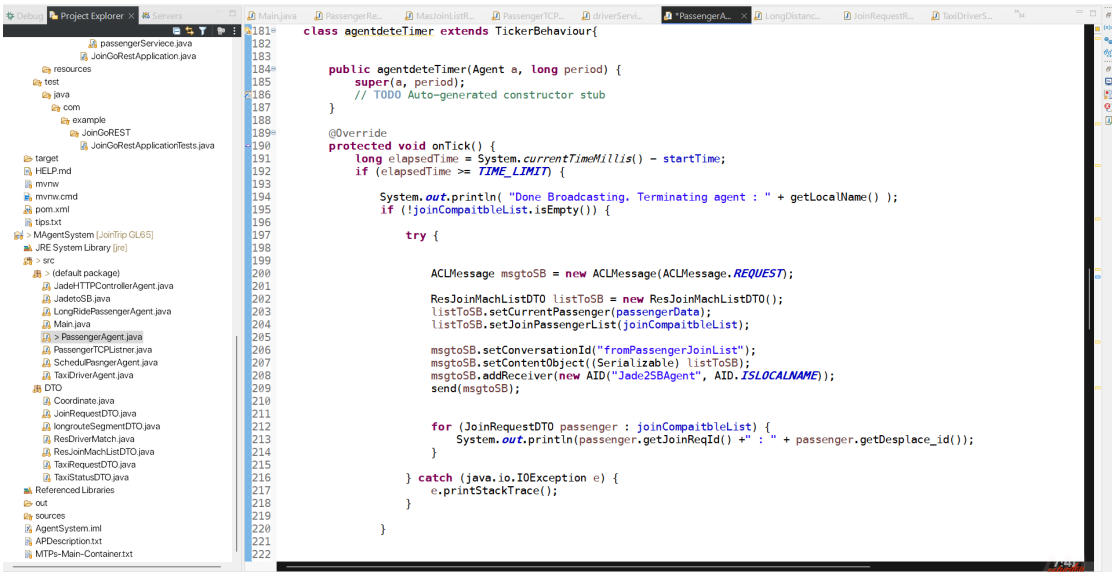
```
public class PassengerAgent extends Agent {

    private long startTime;
    private static final long TIME_LIMIT = 10000;
    private static int    closeradius= 2;
    private  JoinRequestDTO passengerData = new JoinRequestDTO();
    //private JoinRequestDTO comparePassenger = new JoinRequestDTO();
    private List<String> destPlaceIdCheckednequlList = new ArrayList<>();
    private List<JoinRequestDTO> joinCompaitbleList = new ArrayList<>();
    private List<String> messagedAgents = new ArrayList<>();


    @Override
    protected void setup() {

        startTime = System.currentTimeMillis();


        ServiceDescription sd = new ServiceDescription();
        sd.setType("passenger");
        sd.setName(getLocalName());
        DFAgentDescription dfd = new DFAgentDescription();
        dfd.setName(getAID());
        dfd.addServices(sd);
        try {
            DFService.register(this, dfd);
        } catch (FIPAException fe) {
            fe.printStackTrace();
        }

        Object[] args = getArguments();
        if (args != null && args.length > 0) {
            // Assuming the first argument is the PassengerDTO object
            passengerData = (JoinRequestDTO) args[0];
        } else {
            System.out.println("No data received.");
        }

        destPlaceIdCheckednequlList.add(getLocalName());
        messagedAgents.add(getLocalName());
```



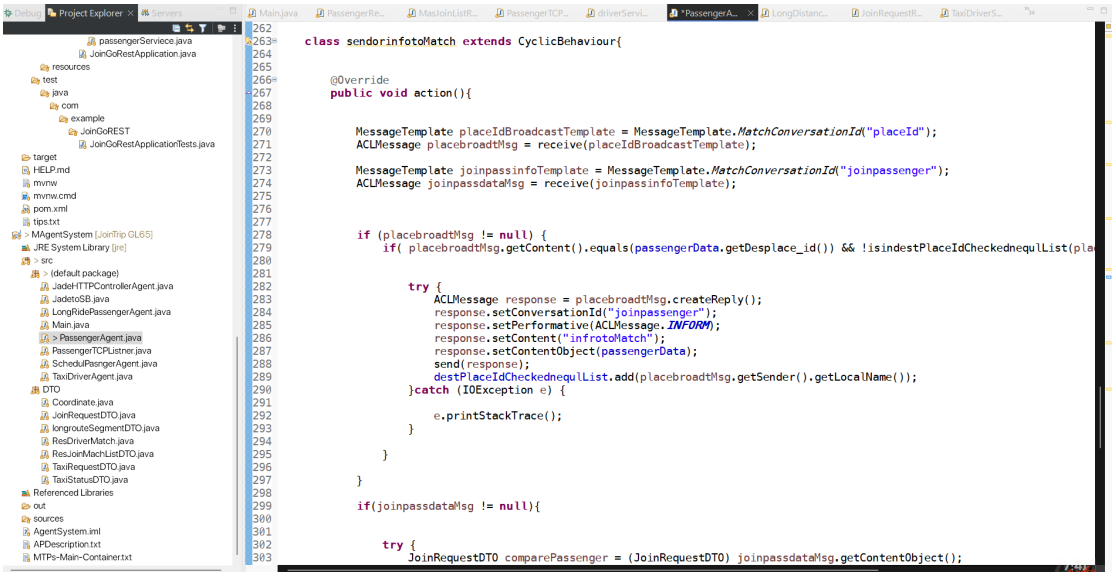
```
        System.out.println("++Created agent: " +getAID().getLocalName() + ": A Join Passenger agent created.");

        addBehaviour(new agentdeteTimer(this, TIME_LIMIT));
        addBehaviour(new sendorinfotoMatch());
        addBehaviour(new destinationBroadcast());

        addBehaviour(new firstnewreqplaceidone());
        addBehaviour(new broadcastNewarrival());




    }


    class FindTaxiBehaviour extends OneShotBehaviour {


        @Override
        public void action() {
            DFAgentDescription template = new DFAgentDescription();
            ServiceDescription sd = new ServiceDescription();
            sd.setType("taxi");
            template.addServices(sd);
            try {
                System.out.println(getAgent().getLocalName() + ": Searching for available taxis...");
                DFAgentDescription[] result = DFService.search(this.getAgent(), template);
                if (result.length > 0) {
                    for (DFAgentDescription dfAgent : result) {
                        ACLMessage msg = new ACLMessage(ACLMessage.REQUEST);
                        msg.addReceiver(dfAgent.getName());
                        msg.setContent("Need a ride!");
                        send(msg);

                    }
                }
            } catch (FIPAException fe) {
                fe.printStackTrace();
            }
```



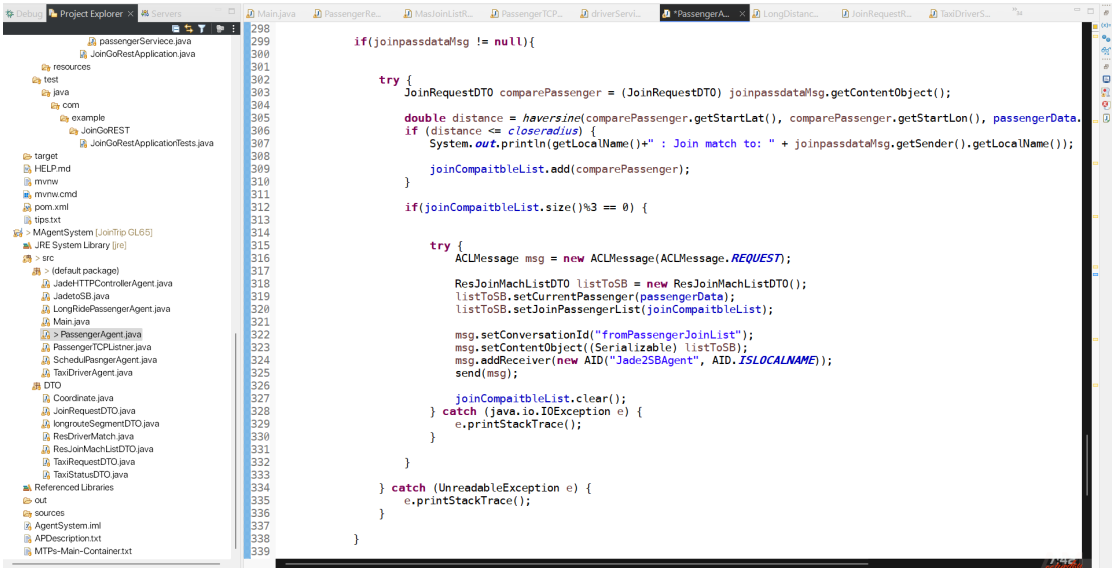
```
    class firstnewreqplaceidone extends OneShotBehaviour{

        @Override
        public void action() {

            DFAgentDescription template = new DFAgentDescription();
            ServiceDescription sdnew = new ServiceDescription();
            sdnew.setType("passenger");
            template.addServices(sdnew);

            try {
                DFAgentDescription[] result = DFService.search(this.getAgent(), template);
                if (result.length > 0) {
                    for (DFAgentDescription dfAgent : result) {
                        //AID agentAID = dfAgent.getName();
                        if( !messagedAgents.contains(dfAgent.getName().getLocalName()) ) {

                            String mypalceId = passengerData.getDesplace_id();
                            ACLMessage msgplaceinfo = new ACLMessage(ACLMessage.REQUEST);
                            msgplaceinfo.addReceiver(dfAgent.getName());
                            msgplaceinfo.setConversationId("placeId");
                            msgplaceinfo.setContent(mypalceId);
                            send(msgplaceinfo);

                        }

                    }
                }
            } catch (FIPAException fe) {
                fe.printStackTrace();
            }

        }
    }


    class broadcastNewarrival extends OneShotBehaviour{
        @Override
        public void action() {

            DFAgentDescription template = new DFAgentDescription();
```


Project Explorer
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
HELP.md
mvnw
mvnw.cmd
pom.xml
tips.txt
> MAgentSystem [JoinTrip GL65]
JRE System Library [jre]
> src
> (default package)
JadeHTTPControllerAgent.java
JadetoSB.java
LongRidePassengerAgent.java
Main.java
> PassengerAgent.java
PassengerTCPListner.java
SchedulPasngerAgent.java
TaxiDriverAgent.java
DTO
Coordinate.java
JoinRequestDTO.java
longrouteSegmentDTO.java
ResDriverMatch.java
ResJoinMachListDTO.java
TaxiRequestDTO.java
TaxiStatusDTO.java
Referenced Libraries
out
sources
AgentSystem.iml
APDescription.txt
MTPs-Main-Container.txt
class agentdeteTimer extends TickerBehaviour{
public agentdeteTimer(Agent a, long period) {
super(a, period);
// TODO Auto-generated constructor stub
}
@Override
protected void onTick() {
long elapsedTime = System.currentTimeMillis() - startTime;
if (elapsedTime >= TIME_LIMIT) {
System.out.println("Done Broadcasting. Terminating agent : " + getLocalName());
if (!joinCompaitbleList.isEmpty()) {
try {
ACLMessage msgtoSB = new ACLMessage(ACLMessage.REQUEST);
ResJoinMachListDTO listToSB = new ResJoinMachListDTO();
listToSB.setCurrentPassenger(passengerData);
listToSB.setJoinPassengerList(joinCompaitbleList);
msgtoSB.setConversationId("fromPassengerJoinList");
msgtoSB.setContentObject((Serializable) listToSB);
msgtoSB.addReceiver(new AID("Jade2SBAgent", AID.ISLOCALNAME));
send(msgtoSB);
for (JoinRequestDTO passenger : joinCompaitbleList) {
System.out.println(passenger.getJoinReqId() +" : " + passenger.getDesplace_id());
}
} catch (java.io.IOException e) {
e.printStackTrace();
}
}

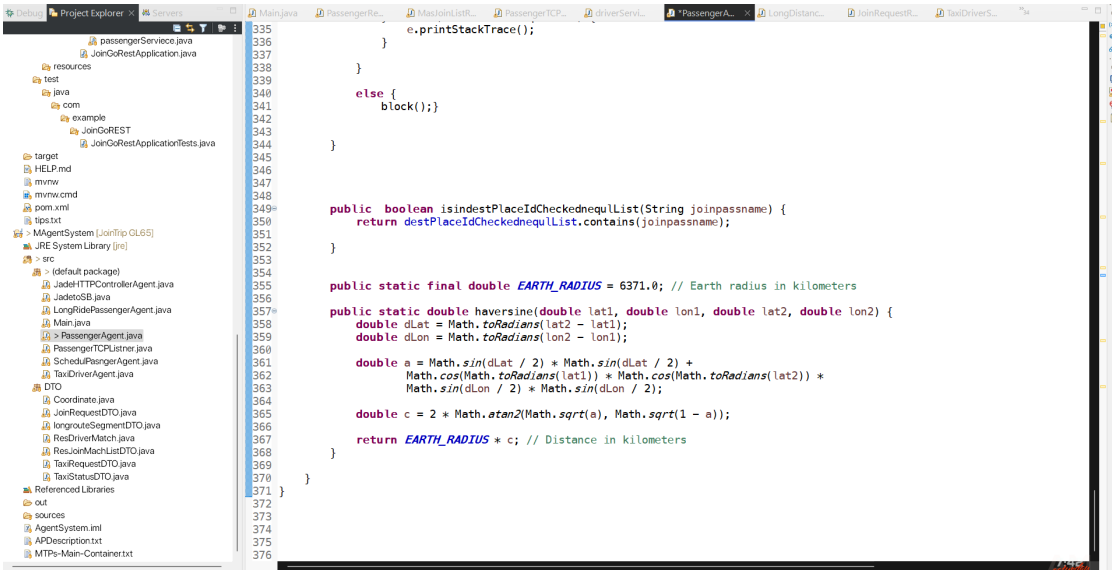

Project Explorer
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
HELP.md
mvnw
mvnw.cmd
pom.xml
tips.txt
> MAgentSystem [JoinTrip GL65]
JRE System Library [jre]
> src
> (default package)
JadeHTTPControllerAgent.java
JadetoSB.java
LongRidePassengerAgent.java
Main.java
> PassengerAgent.java
PassengerTCPListner.java
SchedulPasngerAgent.java
TaxiDriverAgent.java
DTO
Coordinate.java
JoinRequestDTO.java
longrouteSegmentDTO.java
ResDriverMatch.java
ResJoinMachListDTO.java
TaxiRequestDTO.java
TaxiStatusDTO.java
Referenced Libraries
out
sources
AgentSystem.iml
APDescription.txt
MTPs-Main-Container.txt
}
class destinationBroadcast extends CyclicBehaviour{
@Override
public void action(){
MessageTemplate newReqNoticeTemplate = MessageTemplate.MatchConversationId("newReqNotice");
ACLMessage newReqNoticeMsg = receive(newReqNoticeTemplate);
if (newReqNoticeMsg != null) {
String mypalceId = passengerData.getDesplace_id();
ACLMessage response = newReqNoticeMsg.createReply();
response.setPerformative(ACLMessage.INFORM);
response.setConversationId("placeId");
response.setContent(mypalceId);
send(response);
}else {
block();
}
}
}
class sendorinfotoMatch extends CyclicBehaviour{
@Override
public void action(){
MessageTemplate placeIdBroadcastTemplate = MessageTemplate.MatchConversationId("placeId");



```
class sendorinfotoMatch extends CyclicBehaviour{


    @Override
    public void action(){


        MessageTemplate placeIdBroadcastTemplate = MessageTemplate.MatchConversationId("placeId");
        ACLMessage placebroadtMsg = receive(placeIdBroadcastTemplate);

        MessageTemplate joinpassinfoTemplate = MessageTemplate.MatchConversationId("joinpassenger");
        ACLMessage joinpassdataMsg = receive(joinpassinfoTemplate);



        if (placebroadtMsg != null) {
            if( placebroadtMsg.getContent().equals(passengerData.getDesplace_id()) && !isindestPlaceIdCheckednequlList(pla


                try {
                    ACLMessage response = placebroadtMsg.createReply();
                    response.setConversationId("joinpassenger");
                    response.setPerformative(ACLMessage.INFORM);
                    response.setContent("infrotoMatch");
                    response.setContentObject(passengerData);
                    send(response);
                    destPlaceIdCheckednequlList.add(placebroadtMsg.getSender().getLocalName());
                }catch (IOException e) {

                    e.printStackTrace();
                }

            }

        }

        if(joinpassdataMsg != null){


            try {
                JoinRequestDTO comparePassenger = (JoinRequestDTO) joinpassdataMsg.getContentObject();
```

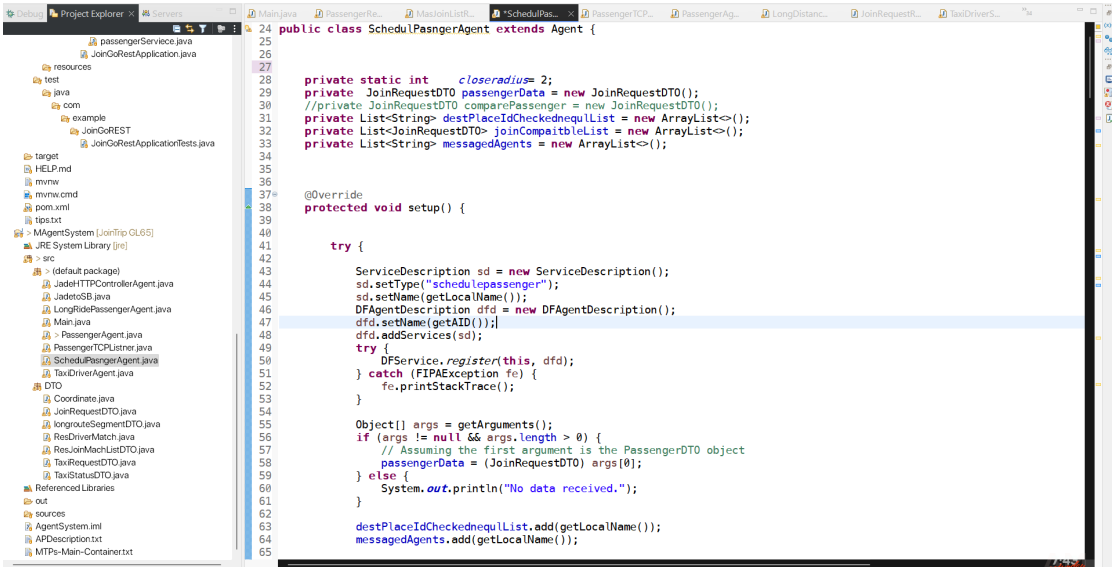


```
        if(joinpassdataMsg != null){


            try {
                JoinRequestDTO comparePassenger = (JoinRequestDTO) joinpassdataMsg.getContentObject();

                double distance = haversine(comparePassenger.getStartLat(), comparePassenger.getStartLon(), passengerData.
                if (distance <= closeradius) {
                    System.out.println(getLocalName()+" : Join match to: " + joinpassdataMsg.getSender().getLocalName());

                    joinCompaitbleList.add(comparePassenger);
                }

                if(joinCompaitbleList.size()%3 == 0) {


                    try {
                        ACLMessage msg = new ACLMessage(ACLMessage.REQUEST);

                        ResJoinMachListDTO listToSB = new ResJoinMachListDTO();
                        listToSB.setCurrentPassenger(passengerData);
                        listToSB.setJoinPassengerList(joinCompaitbleList);

                        msg.setConversationId("fromPassengerJoinList");
                        msg.setContentObject((Serializable) listToSB);
                        msg.addReceiver(new AID("Jade2SBAgent", AID.ISLOCALNAME));
                        send(msg);

                        joinCompaitbleList.clear();
                    } catch (java.io.IOException e) {
                        e.printStackTrace();
                    }

                }

            } catch (UnreadableException e) {
                e.printStackTrace();
            }

        }
```

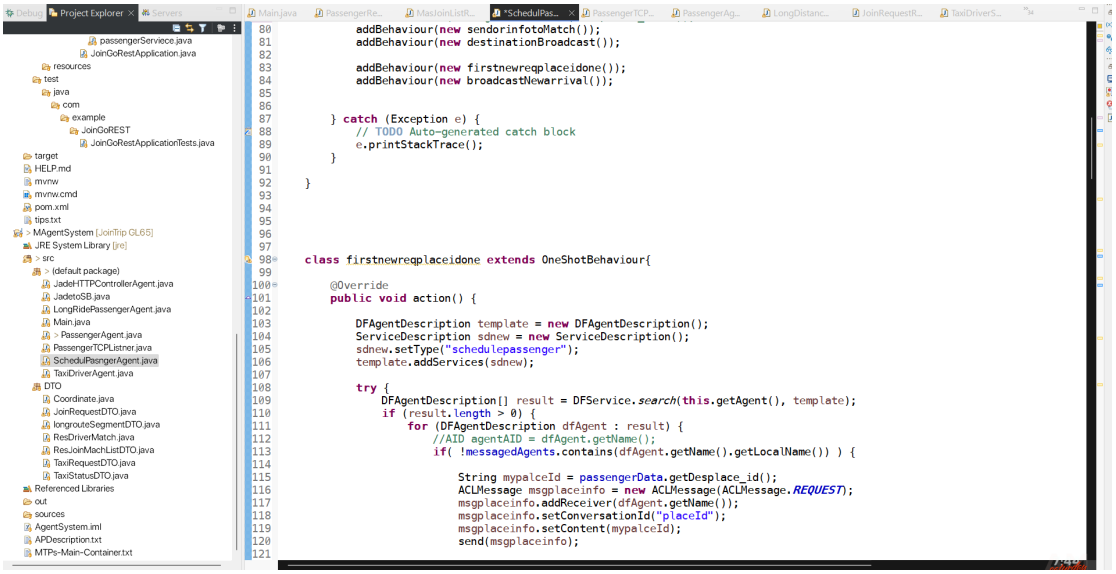


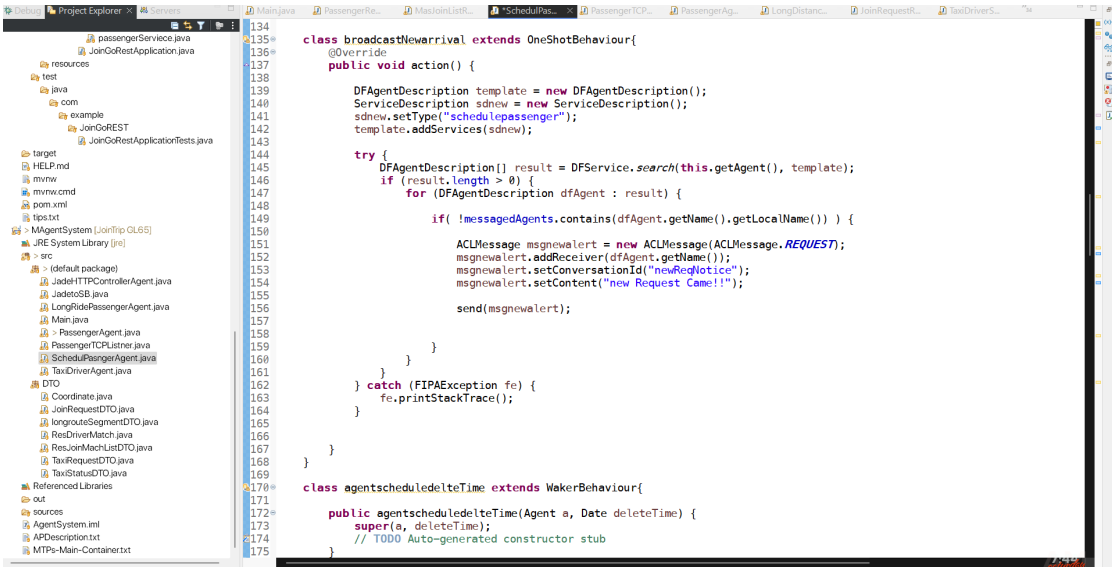
```
                e.printStackTrace();
            }

        }

        else {
            block();}


    }



    public  boolean isindestPlaceIdCheckednequlList(String joinpassname) {
        return destPlaceIdCheckednequlList.contains(joinpassname);

    }


    public static final double EARTH_RADIUS = 6371.0; // Earth radius in kilometers

    public static double haversine(double lat1, double lon1, double lat2, double lon2) {
        double dLat = Math.toRadians(lat2 - lat1);
        double dLon = Math.toRadians(lon2 - lon1);

        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
                Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2)) *
                Math.sin(dLon / 2) * Math.sin(dLon / 2);

        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));

        return EARTH_RADIUS * c; // Distance in kilometers
    }

}
}
```

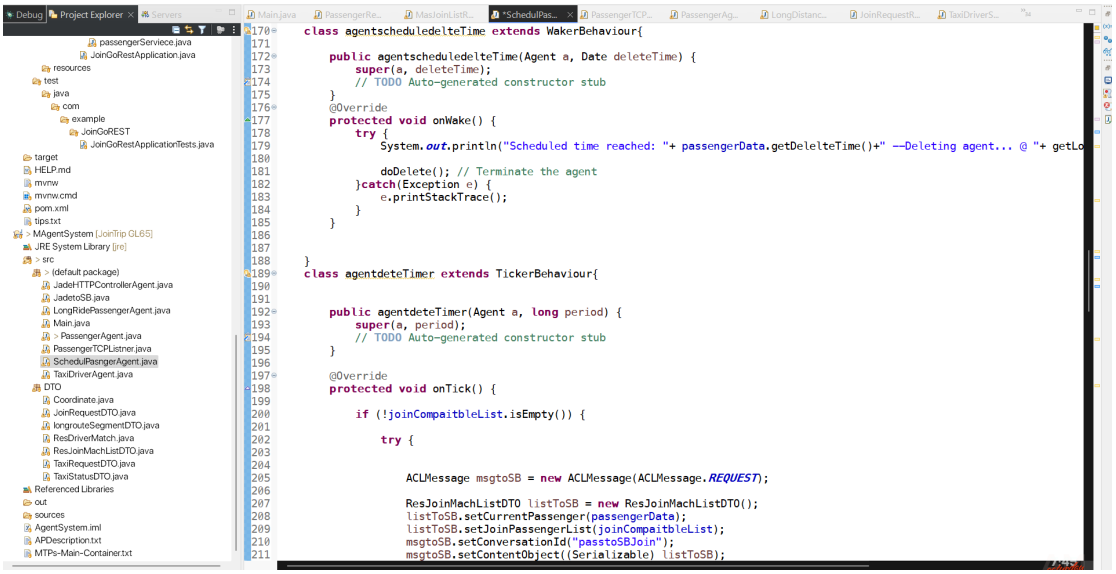
- SchedulPasngerAgent class-



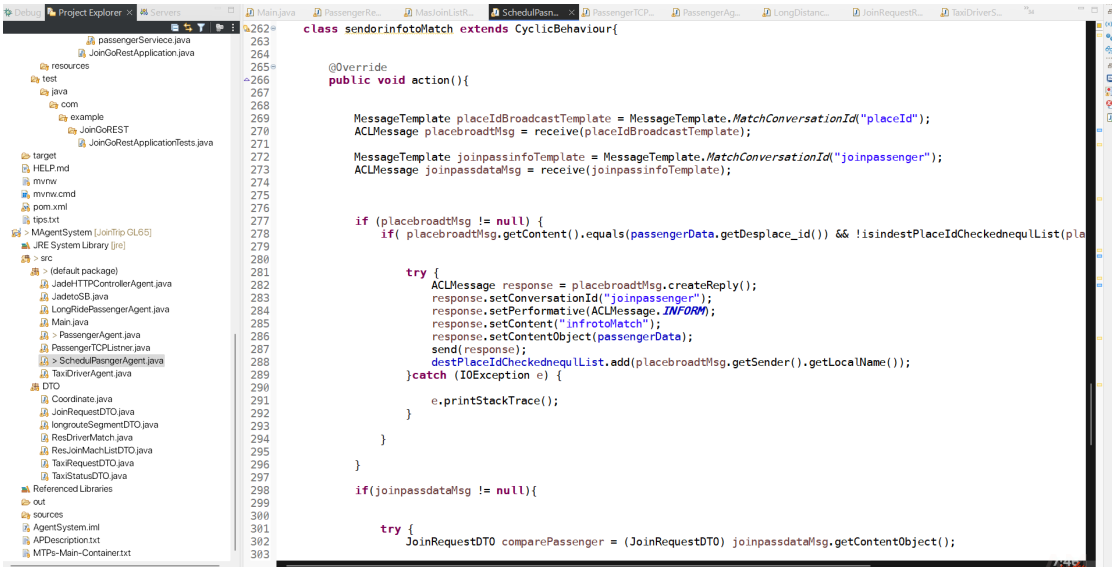
```
public class SchedulPasngerAgent extends Agent {



    private static int     closeradius= 2;
    private  JoinRequestDTO passengerData = new JoinRequestDTO();
    //private JoinRequestDTO comparePassenger = new JoinRequestDTO();
    private List<String> destPlaceIdCheckednequlList = new ArrayList<>();
    private List<JoinRequestDTO> joinCompaitbleList = new ArrayList<>();
    private List<String> messagedAgents = new ArrayList<>();


    @Override
    protected void setup() {


        try {

            ServiceDescription sd = new ServiceDescription();
            sd.setType("schedulepassenger");
            sd.setName(getLocalName());
            DFAgentDescription dfd = new DFAgentDescription();
            dfd.setName(getAID());
            dfd.addServices(sd);
            try {
                DFService.register(this, dfd);
            } catch (FIPAException fe) {
                fe.printStackTrace();
            }

            Object[] args = getArguments();
            if (args != null && args.length > 0) {
                // Assuming the first argument is the PassengerDTO object
                passengerData = (JoinRequestDTO) args[0];
            } else {
                System.out.println("No data received.");
            }

            destPlaceIdCheckednequlList.add(getLocalName());
            messagedAgents.add(getLocalName());
```


addBehaviour(new sendorinfotoMatch());
addBehaviour(new destinationBroadcast());
addBehaviour(new firstnewreqplaceidone());
addBehaviour(new broadcastNewarrival());
} catch (Exception e) {
// TODO Auto-generated catch block
e.printStackTrace();
}
}
class firstnewreqplaceidone extends OneShotBehaviour{
@Override
public void action() {
DFAgentDescription template = new DFAgentDescription();
ServiceDescription sdnew = new ServiceDescription();
sdnew.setType("schedulepassenger");
template.addServices(sdnew);
try {
DFAgentDescription[] result = DFService.search(this.getAgent(), template);
if (result.length > 0) {
for (DFAgentDescription dfAgent : result) {
//AID agentAID = dfAgent.getName();
if(!messagedAgents.contains(dfAgent.getName().getLocalName())) {
String mypalceId = passengerData.getDesplace_id();
ACLMessage msgplaceinfo = new ACLMessage(ACLMessage.REQUEST);
msgplaceinfo.addReceiver(dfAgent.getName());
msgplaceinfo.setConversationId("placeId");
msgplaceinfo.setContent(mypalceId);
send(msgplaceinfo);


class broadcastNewarrival extends OneShotBehaviour{
@Override
public void action() {
DFAgentDescription template = new DFAgentDescription();
ServiceDescription sdnew = new ServiceDescription();
sdnew.setType("schedulepassenger");
template.addServices(sdnew);
try {
DFAgentDescription[] result = DFService.search(this.getAgent(), template);
if (result.length > 0) {
for (DFAgentDescription dfAgent : result) {
if(!messagedAgents.contains(dfAgent.getName().getLocalName())) {
ACLMessage msgnewalert = new ACLMessage(ACLMessage.REQUEST);
msgnewalert.addReceiver(dfAgent.getName());
msgnewalert.setConversationId("newReqNotice");
msgnewalert.setContent("new Request Came!!");
send(msgnewalert);
}
}
}
} catch (FIPAException fe) {
fe.printStackTrace();
}
}
class agentscheduledelteTime extends WakerBehaviour{
public agentscheduledelteTime(Agent a, Date deleteTime) {
super(a, deleteTime);
// TODO Auto-generated constructor stub
}

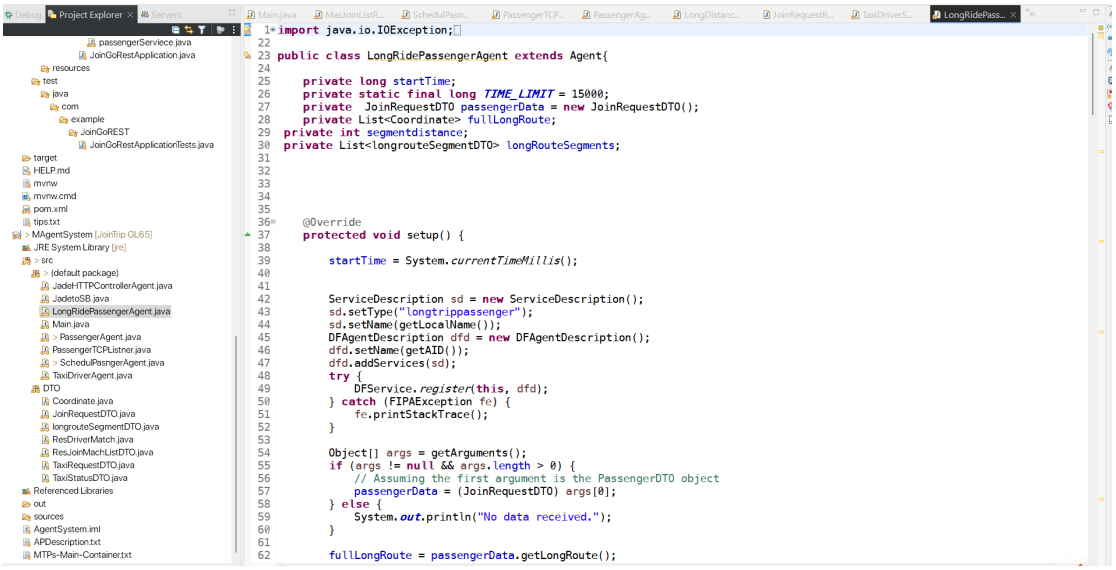


```
class agentscheduledelteTime extends WakerBehaviour{

    public agentscheduledelteTime(Agent a, Date deleteTime) {
        super(a, deleteTime);
        // TODO Auto-generated constructor stub
    }
    @Override
    protected void onWake() {
        try {
            System.out.println("Scheduled time reached: "+ passengerData.getDelelteTime()+" --Deleting agent... @ "+ getLo

            doDelete(); // Terminate the agent
        }catch(Exception e) {
            e.printStackTrace();
        }
    }


}
class agentdeteTimer extends TickerBehaviour{


    public agentdeteTimer(Agent a, long period) {
        super(a, period);
        // TODO Auto-generated constructor stub
    }

    @Override
    protected void onTick() {

        if (!joinCompaitbleList.isEmpty()) {

            try {


                ACLMessage msgtoSB = new ACLMessage(ACLMessage.REQUEST);

                ResJoinMachListDTO listToSB = new ResJoinMachListDTO();
                listToSB.setCurrentPassenger(passengerData);
                listToSB.setJoinPassengerList(joinCompaitbleList);
                msgtoSB.setConversationId("passtoSBJoin");
                msgtoSB.setContentObject((Serializable) listToSB);
```



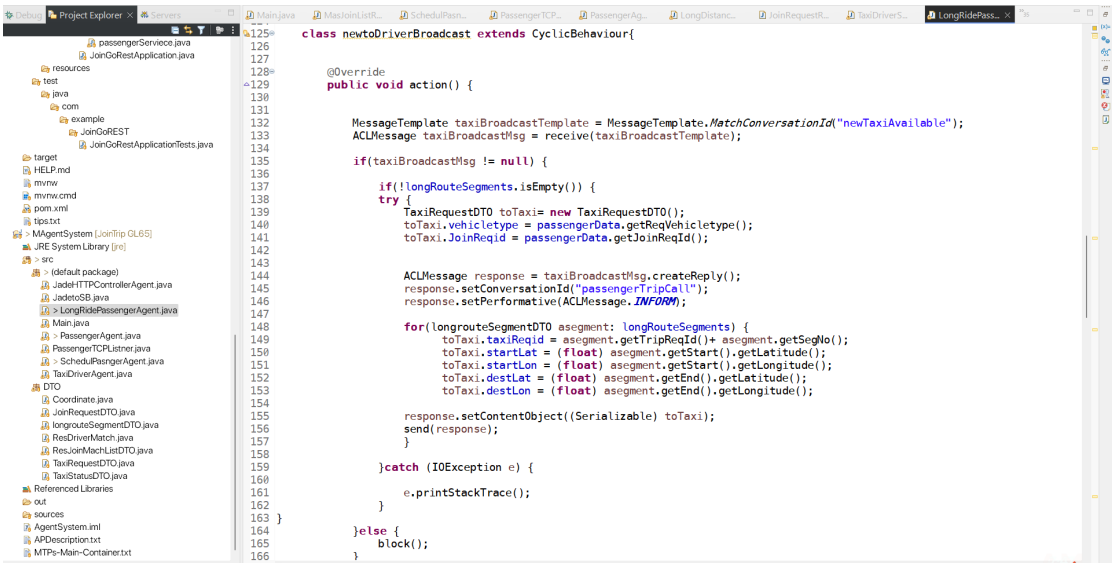
```
class sendorinfotoMatch extends CyclicBehaviour{


    @Override
    public void action(){


        MessageTemplate placeIdBroadcastTemplate = MessageTemplate.MatchConversationId("placeId");
        ACLMessage placebroadtMsg = receive(placeIdBroadcastTemplate);

        MessageTemplate joinpassinfoTemplate = MessageTemplate.MatchConversationId("joinpassenger");
        ACLMessage joinpassdataMsg = receive(joinpassinfoTemplate);



        if (placebroadtMsg != null) {
            if( placebroadtMsg.getContent().equals(passengerData.getDesplace_id()) && !isindestPlaceIdCheckednequlList(pla


                try {
                    ACLMessage response = placebroadtMsg.createReply();
                    response.setConversationId("joinpassenger");
                    response.setPerformative(ACLMessage.INFORM);
                    response.setContent("infrotoMatch");
                    response.setContentObject(passengerData);
                    send(response);
                    destPlaceIdCheckednequlList.add(placebroadtMsg.getSender().getLocalName());
                }catch (IOException e) {

                    e.printStackTrace();
                }

            }

        }

        if(joinpassdataMsg != null){


            try {
                JoinRequestDTO comparePassenger = (JoinRequestDTO) joinpassdataMsg.getContentObject();
```

- LongRidePassengerAgent class:

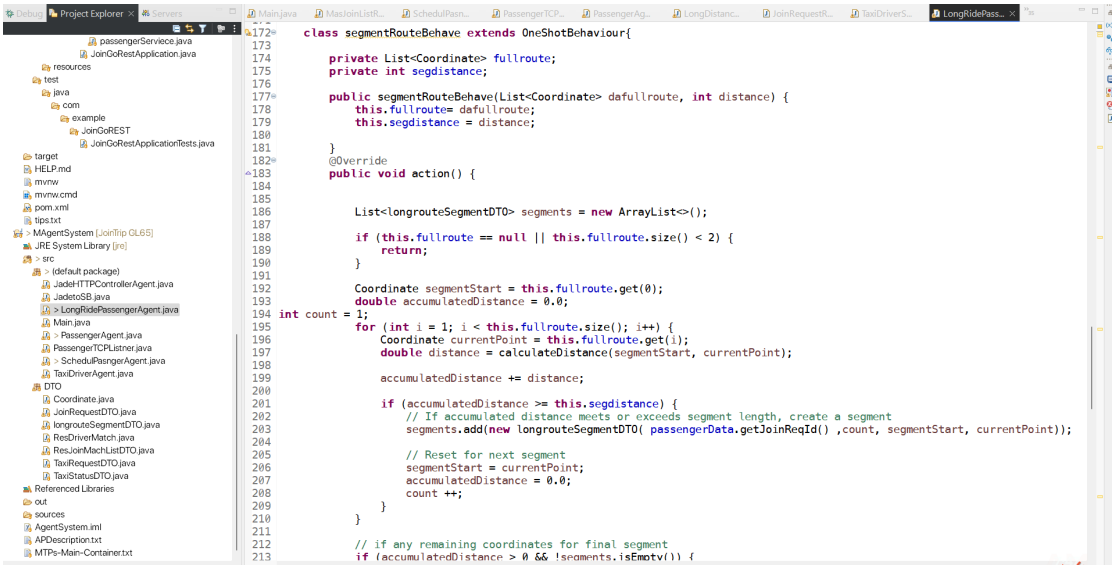


```
import java.io.IOException;

public class LongRidePassengerAgent extends Agent{

    private long startTime;
    private static final long TIME_LIMIT = 15000;
    private  JoinRequestDTO passengerData = new JoinRequestDTO();
    private List<Coordinate> fullLongRoute;
private int segmentdistance;
private List<longrouteSegmentDTO> longRouteSegments;



    @Override
    protected void setup() {

        startTime = System.currentTimeMillis();


        ServiceDescription sd = new ServiceDescription();
        sd.setType("longtrippassenger");
        sd.setName(getLocalName());
        DFAgentDescription dfd = new DFAgentDescription();
        dfd.setName(getAID());
        dfd.addServices(sd);
        try {
            DFService.register(this, dfd);
        } catch (FIPAException fe) {
            fe.printStackTrace();
        }

        Object[] args = getArguments();
        if (args != null && args.length > 0) {
            // Assuming the first argument is the PassengerDTO object
            passengerData = (JoinRequestDTO) args[0];
        } else {
            System.out.println("No data received.");
        }

        fullLongRoute = passengerData.getLongRoute();
```

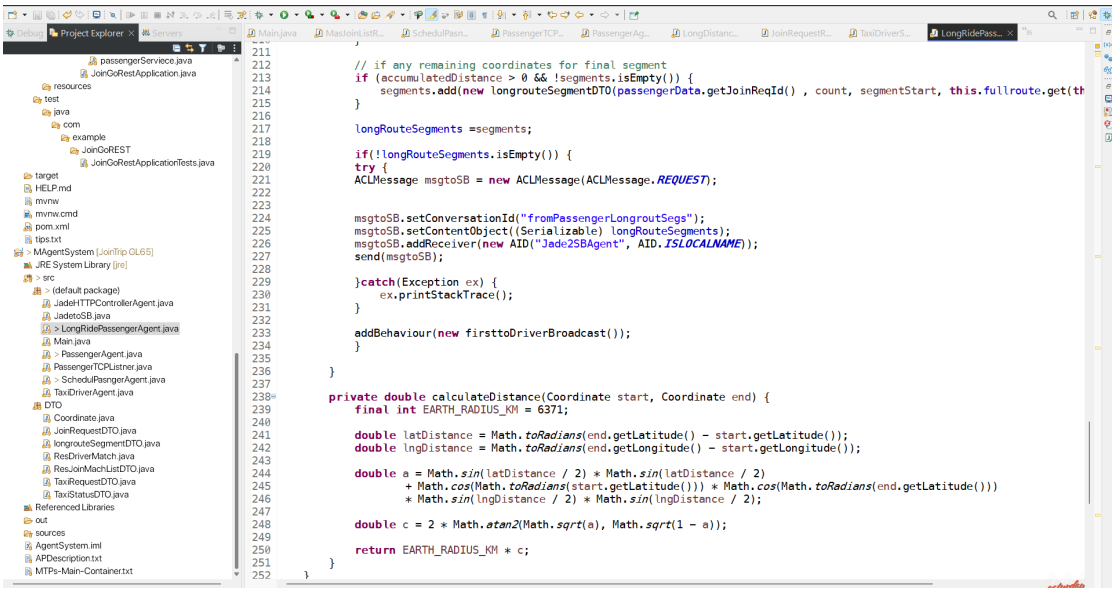
```
        addBehaviour(new agentdeltimer(this, TIME_LIMIT));
        addBehaviour(new segmentRouteBehave(fullLongRoute,segmentdistance));


        addBehaviour(new newtoDriverBroadcast());


    }


    class firsttoDriverBroadcast extends OneShotBehaviour{

        @Override
        public void action() {

            DFAgentDescription template = new DFAgentDescription();
            ServiceDescription toDriver = new ServiceDescription();
            toDriver.setType("taxidriver");
            template.addServices(toDriver);

            try {
                DFAgentDescription[] result = DFService.search(this.getAgent(), template);
                if (result.length > 0) {
                    TaxiRequestDTO toTaxi= new TaxiRequestDTO();
                    toTaxi.vehicletype = passengerData.getReqVehicletype();
                    toTaxi.JoinReqid = passengerData.getJoinReqId();


                    for (DFAgentDescription dfAgent : result) {
for(longrouteSegmentDTO asegment: longRouteSegments) {
    toTaxi.taxiReqid = asegment.getTripReqId()+ asegment.getSegNo();
                    toTaxi.startLat = (float) asegment.getStart().getLatitude();
                    toTaxi.startLon = (float) asegment.getStart().getLongitude();
                    toTaxi.destLat = (float) asegment.getEnd().getLatitude();
                    toTaxi.destLon = (float) asegment.getEnd().getLongitude();

                            ACLMessage msgnewalert = new ACLMessage(ACLMessage.INFORM);
                            msgnewalert.addReceiver(dfAgent.getName());
                            msgnewalert.setConversationId("passengerTripCall");
                            msgnewalert.setContentObject((Serializable) toTaxi);
```



```
    class newtoDriverBroadcast extends CyclicBehaviour{


        @Override
        public void action() {


            MessageTemplate taxiBroadcastTemplate = MessageTemplate.MatchConversationId("newTaxiAvailable");
            ACLMessage taxiBroadcastMsg = receive(taxiBroadcastTemplate);

            if(taxiBroadcastMsg != null) {

                if(!longRouteSegments.isEmpty()) {
                try {
                    TaxiRequestDTO toTaxi= new TaxiRequestDTO();
                    toTaxi.vehicletype = passengerData.getReqVehicletype();
                    toTaxi.JoinReqid = passengerData.getJoinReqId();


                    ACLMessage response = taxiBroadcastMsg.createReply();
                    response.setConversationId("passengerTripCall");
                    response.setPerformative(ACLMessage.INFORM);

                    for(longrouteSegmentDTO asegment: longRouteSegments) {
                          toTaxi.taxiReqid = asegment.getTripReqId()+ asegment.getSegNo();
                          toTaxi.startLat = (float) asegment.getStart().getLatitude();
                          toTaxi.startLon = (float) asegment.getStart().getLongitude();
                          toTaxi.destLat = (float) asegment.getEnd().getLatitude();
                          toTaxi.destLon = (float) asegment.getEnd().getLongitude();

                    response.setContentObject((Serializable) toTaxi);
                    send(response);
                    }

                }catch (IOException e) {

                    e.printStackTrace();
                }
}
            }else {
                block();
            }
```

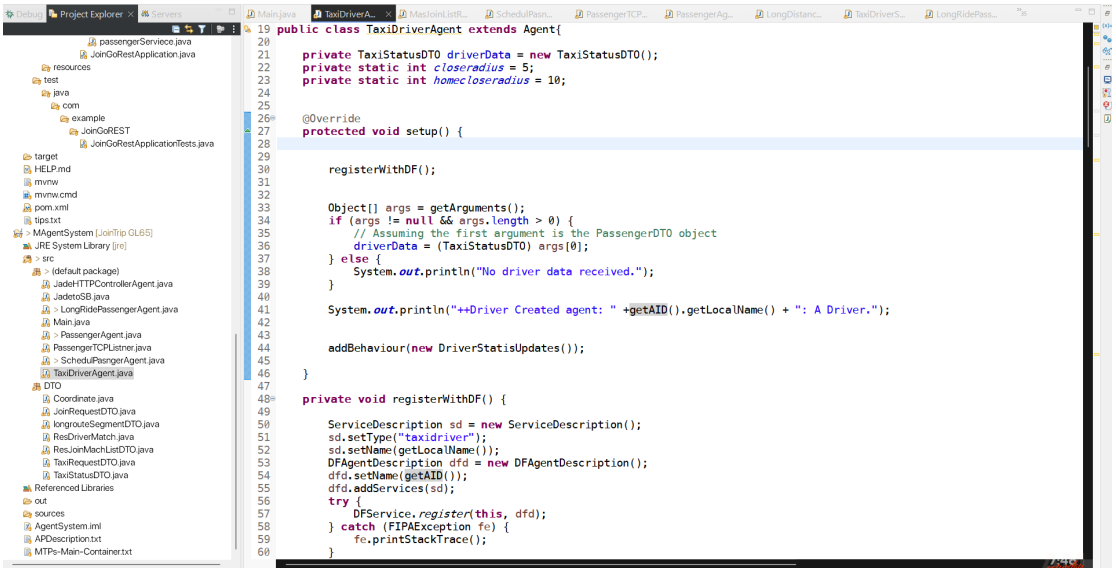


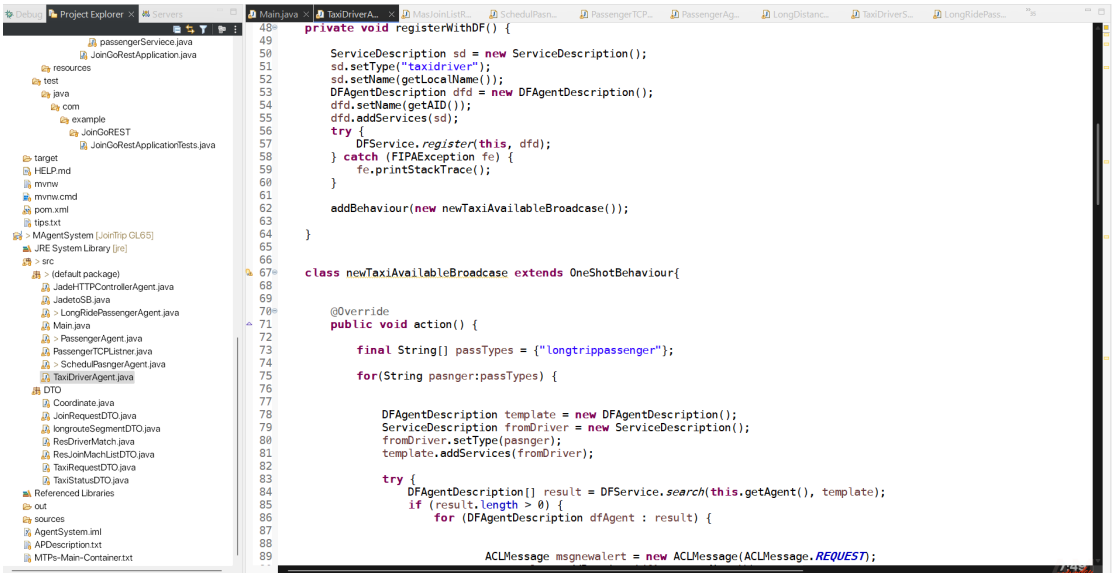
```
class segmentRouteBehave extends OneShotBehaviour{

    private List<Coordinate> fullroute;
    private int segdistance;

    public segmentRouteBehave(List<Coordinate> dafullroute, int distance) {
        this.fullroute= dafullroute;
        this.segdistance = distance;

    }
    @Override
    public void action() {


        List<longrouteSegmentDTO> segments = new ArrayList<>();

        if (this.fullroute == null || this.fullroute.size() < 2) {
            return;
        }

        Coordinate segmentStart = this.fullroute.get(0);
        double accumulatedDistance = 0.0;
int count = 1;
        for (int i = 1; i < this.fullroute.size(); i++) {
            Coordinate currentPoint = this.fullroute.get(i);
            double distance = calculateDistance(segmentStart, currentPoint);

            accumulatedDistance += distance;

            if (accumulatedDistance >= this.segdistance) {
                // If accumulated distance meets or exceeds segment length, create a segment
                segments.add(new longrouteSegmentDTO( passengerData.getJoinReqId() ,count, segmentStart, currentPoint));

                // Reset for next segment
                segmentStart = currentPoint;
                accumulatedDistance = 0.0;
                count ++;
            }
        }

        // if any remaining coordinates for final segment
        if (accumulatedDistance > 0 && !segments.isEmpty()) {
```

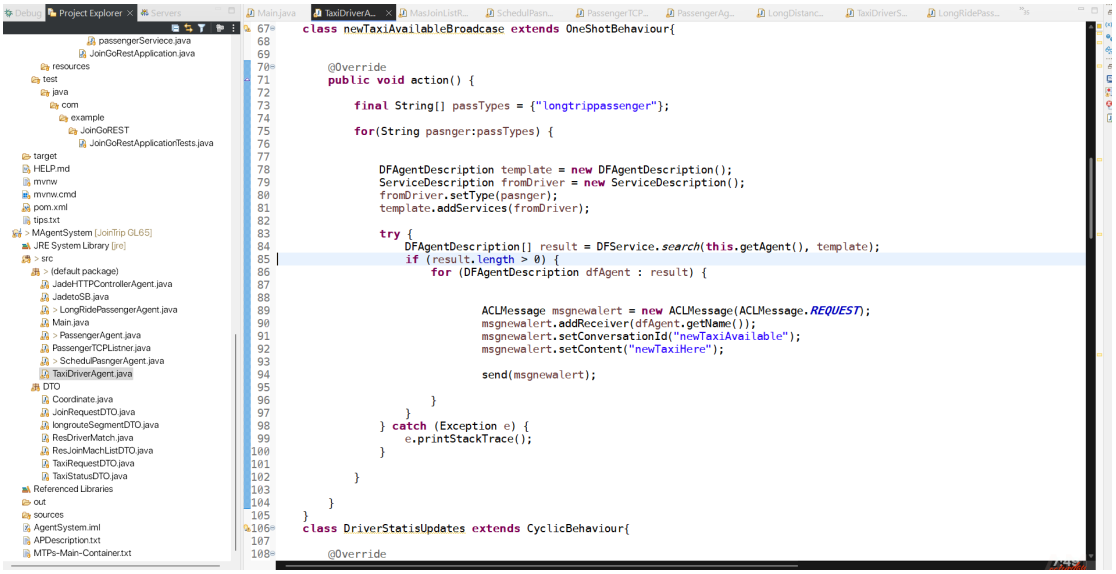


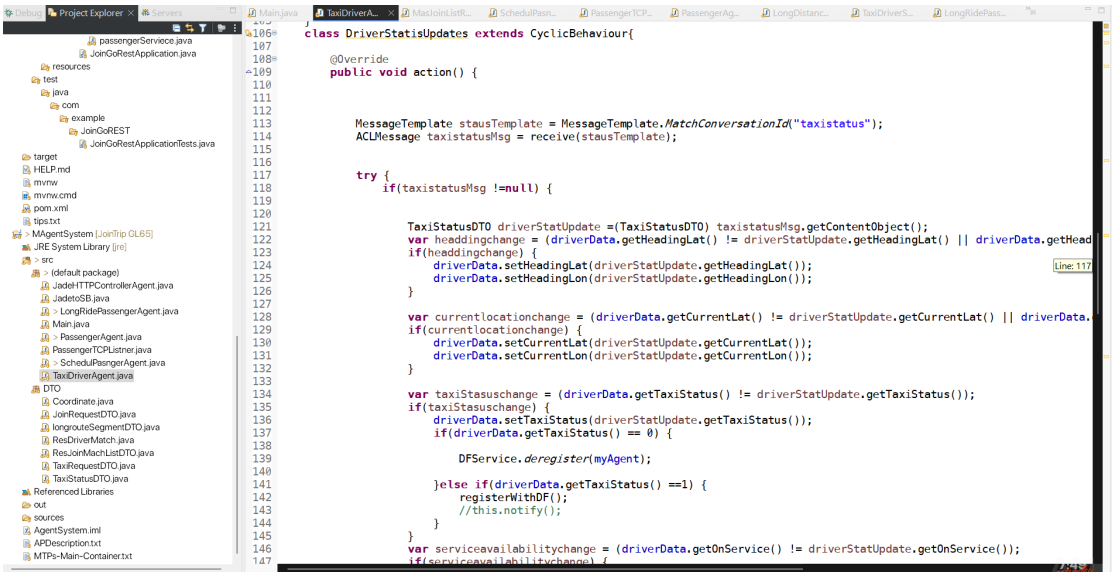
```
        // if any remaining coordinates for final segment
        if (accumulatedDistance > 0 && !segments.isEmpty()) {
            segments.add(new longrouteSegmentDTO(passengerData.getJoinReqId() , count, segmentStart, this.fullroute.get(th
        }

        longRouteSegments =segments;

        if(!longRouteSegments.isEmpty()) {
        try {
        ACLMessage msgtoSB = new ACLMessage(ACLMessage.REQUEST);


        msgtoSB.setConversationId("fromPassengerLongroutSegs");
        msgtoSB.setContentObject((Serializable) longRouteSegments);
        msgtoSB.addReceiver(new AID("Jade2SBAgent", AID.ISLOCALNAME));
        send(msgtoSB);

        }catch(Exception ex) {
            ex.printStackTrace();
        }

        addBehaviour(new firsttoDriverBroadcast());
        }

    }

    private double calculateDistance(Coordinate start, Coordinate end) {
        final int EARTH_RADIUS_KM = 6371;

        double latDistance = Math.toRadians(end.getLatitude() - start.getLatitude());
        double lngDistance = Math.toRadians(end.getLongitude() - start.getLongitude());

        double a = Math.sin(latDistance / 2) * Math.sin(latDistance / 2)
                + Math.cos(Math.toRadians(start.getLatitude())) * Math.cos(Math.toRadians(end.getLatitude()))
                * Math.sin(lngDistance / 2) * Math.sin(lngDistance / 2);

        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));

        return EARTH_RADIUS_KM * c;
    }
}
```

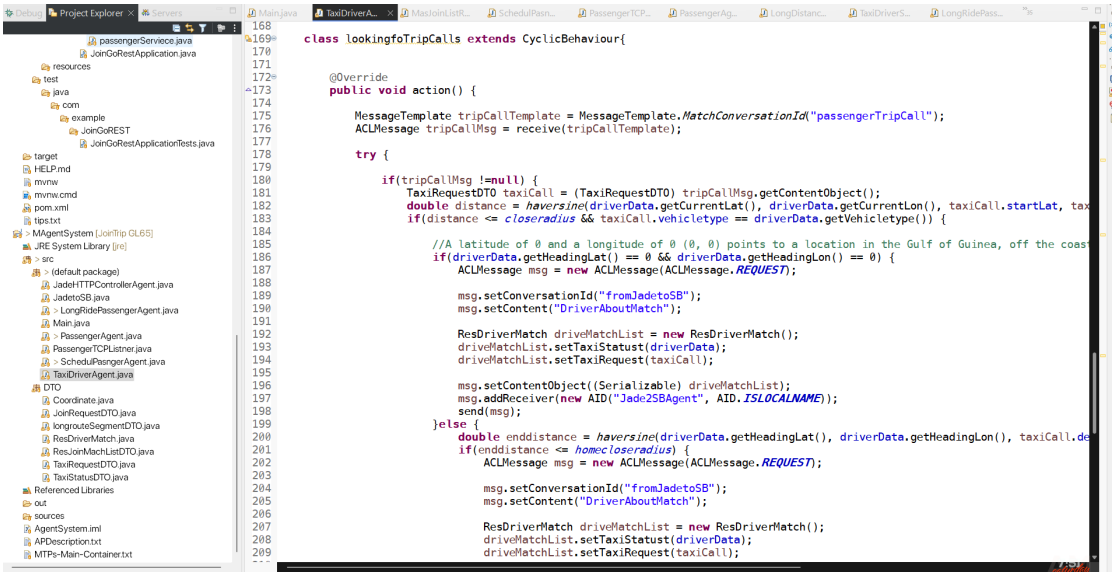
- TaxiDriverAgent class-


Project Explorer
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
HELP.md
mvnw
mvnw.cmd
pom.xml
tips.txt
> MAgentSystem [JoinTrip GL65]
JRE System Library [jre]
> src
> (default package)
JadeHTTPControllerAgent.java
JadetoSB.java
> LongRidePassengerAgent.java
Main.java
> PassengerAgent.java
PassengerTCPListner.java
> SchedulPasngerAgent.java
TaxiDriverAgent.java
DTO
Coordinate.java
JoinRequestDTO.java
longrouteSegmentDTO.java
ResDriverMatch.java
ResJoinMachListDTO.java
TaxiRequestDTO.java
TaxiStatusDTO.java
Referenced Libraries
out
sources
AgentSystem.iml
APDescription.txt
MTPs-Main-Container.txt
Main.java
TaxiDriverA...
MasJoinListR...
SchedulPasn...
PassengerTCP...
PassengerAg...
LongDistanc...
TaxiDriverS...
LongRidePass...
public class TaxiDriverAgent extends Agent{
private TaxiStatusDTO driverData = new TaxiStatusDTO();
private static int closeradius = 5;
private static int homecloseradius = 10;
@Override
protected void setup() {
registerWithDF();
Object[] args = getArguments();
if (args != null && args.length > 0) {
// Assuming the first argument is the PassengerDTO object
driverData = (TaxiStatusDTO) args[0];
} else {
System.out.println("No driver data received.");
}
System.out.println("++Driver Created agent: " +getAID().getLocalName() + ": A Driver.");
addBehaviour(new DriverStatisUpdates());
}
private void registerWithDF() {
ServiceDescription sd = new ServiceDescription();
sd.setType("taxidriver");
sd.setName(getLocalName());
DFAgentDescription dfd = new DFAgentDescription();
dfd.setName(getAID());
dfd.addServices(sd);
try {
DFService.register(this, dfd);
} catch (FIPAException fe) {
fe.printStackTrace();
}


Project Explorer
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
HELP.md
mvnw
mvnw.cmd
pom.xml
tips.txt
> MAgentSystem [JoinTrip GL65]
JRE System Library [jre]
> src
> (default package)
JadeHTTPControllerAgent.java
JadetoSB.java
> LongRidePassengerAgent.java
Main.java
> PassengerAgent.java
PassengerTCPListner.java
> SchedulPasngerAgent.java
TaxiDriverAgent.java
DTO
Coordinate.java
JoinRequestDTO.java
longrouteSegmentDTO.java
ResDriverMatch.java
ResJoinMachListDTO.java
TaxiRequestDTO.java
TaxiStatusDTO.java
Referenced Libraries
out
sources
AgentSystem.iml
APDescription.txt
MTPs-Main-Container.txt
Main.java
TaxiDriverA...
private void registerWithDF() {
ServiceDescription sd = new ServiceDescription();
sd.setType("taxidriver");
sd.setName(getLocalName());
DFAgentDescription dfd = new DFAgentDescription();
dfd.setName(getAID());
dfd.addServices(sd);
try {
DFService.register(this, dfd);
} catch (FIPAException fe) {
fe.printStackTrace();
}
addBehaviour(new newTaxiAvailableBroadcase());
}
class newTaxiAvailableBroadcase extends OneShotBehaviour{
@Override
public void action() {
final String[] passTypes = {"longtrippassenger"};
for(String pasnger:passTypes) {
DFAgentDescription template = new DFAgentDescription();
ServiceDescription fromDriver = new ServiceDescription();
fromDriver.setType(pasnger);
template.addServices(fromDriver);
try {
DFAgentDescription[] result = DFService.search(this.getAgent(), template);
if (result.length > 0) {
for (DFAgentDescription dfAgent : result) {
ACLMessage msgnewalert = new ACLMessage(ACLMessage.REQUEST);

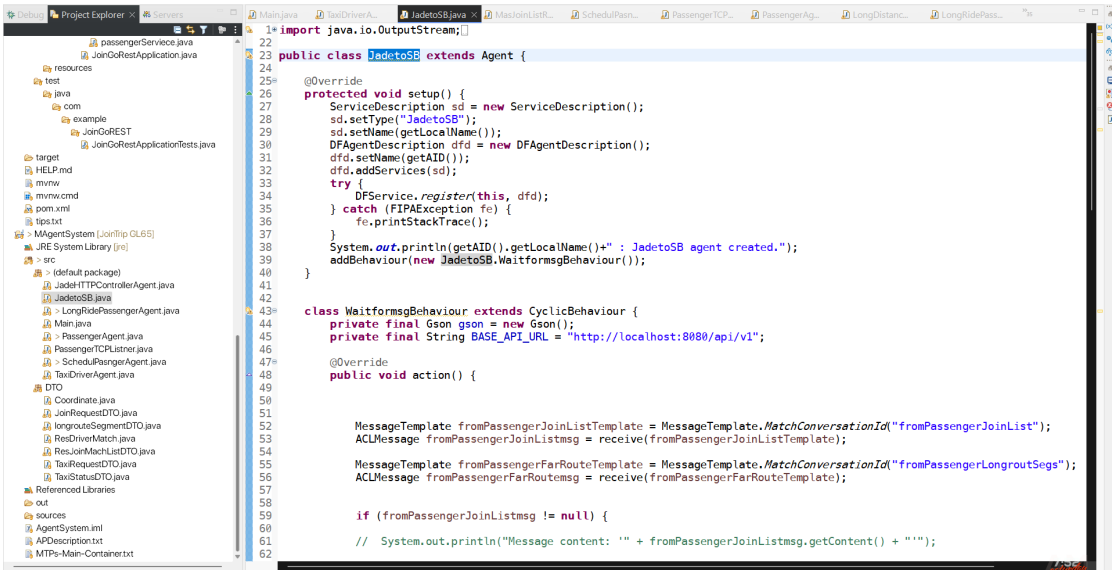

Project Explorer
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
HELP.md
mvnw
mvnw.cmd
pom.xml
tips.txt
MAgentSystem [JoinTrip GL65]
JRE System Library [jre]
src
(default package)
JadeHTTPControllerAgent.java
JadetoSB.java
LongRidePassengerAgent.java
Main.java
PassengerAgent.java
PassengerTCPListner.java
SchedulPasngerAgent.java
TaxiDriverAgent.java
DTO
Coordinate.java
JoinRequestDTO.java
longrouteSegmentDTO.java
ResDriverMatch.java
ResJoinMachListDTO.java
TaxiRequestDTO.java
TaxiStatusDTO.java
Referenced Libraries
out
sources
AgentSystem.iml
APDescription.txt
MTPs-Main-Container.txt
class newTaxiAvailableBroadcase extends OneShotBehaviour{
@Override
public void action() {
final String[] passTypes = {"longtrippassenger"};
for(String pasnger:passTypes) {
DFAgentDescription template = new DFAgentDescription();
ServiceDescription fromDriver = new ServiceDescription();
fromDriver.setType(pasnger);
template.addServices(fromDriver);
try {
DFAgentDescription[] result = DFService.search(this.getAgent(), template);
if (result.length > 0) {
for (DFAgentDescription dfAgent : result) {
ACLMessage msgnewalert = new ACLMessage(ACLMessage.REQUEST);
msgnewalert.addReceiver(dfAgent.getName());
msgnewalert.setConversationId("newTaxiAvailable");
msgnewalert.setContent("newTaxiHere");
send(msgnewalert);
}
}
} catch (Exception e) {
e.printStackTrace();
}
}
}
class DriverStatisUpdates extends CyclicBehaviour{
@Override


Project Explorer
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
HELP.md
mvnw
mvnw.cmd
pom.xml
tips.txt
MAgentSystem [JoinTrip GL65]
JRE System Library [jre]
src
(default package)
JadeHTTPControllerAgent.java
JadetoSB.java
LongRidePassengerAgent.java
Main.java
PassengerAgent.java
PassengerTCPListner.java
SchedulPasngerAgent.java
TaxiDriverAgent.java
DTO
Coordinate.java
JoinRequestDTO.java
longrouteSegmentDTO.java
ResDriverMatch.java
ResJoinMachListDTO.java
TaxiRequestDTO.java
TaxiStatusDTO.java
Referenced Libraries
out
sources
AgentSystem.iml
APDescription.txt
MTPs-Main-Container.txt
class DriverStatisUpdates extends CyclicBehaviour{
@Override
public void action() {
MessageTemplate stausTemplate = MessageTemplate.MatchConversationId("taxistatus");
ACLMessage taxistatusMsg = receive(stausTemplate);
try {
if(taxistatusMsg !=null) {
TaxiStatusDTO driverStatUpdate =(TaxiStatusDTO) taxistatusMsg.getContentObject();
var headdingchange = (driverData.getHeadingLat() != driverStatUpdate.getHeadingLat() || driverData.getHead
if(headdingchange) {
driverData.setHeadingLat(driverStatUpdate.getHeadingLat());
driverData.setHeadingLon(driverStatUpdate.getHeadingLon());
}
var currentlocationchange = (driverData.getCurrentLat() != driverStatUpdate.getCurrentLat() || driverData.
if(currentlocationchange) {
driverData.setCurrentLat(driverStatUpdate.getCurrentLat());
driverData.setCurrentLon(driverStatUpdate.getCurrentLon());
}
var taxiStasuschange = (driverData.getTaxiStatus() != driverStatUpdate.getTaxiStatus());
if(taxiStasuschange) {
driverData.setTaxiStatus(driverStatUpdate.getTaxiStatus());
if(driverData.getTaxiStatus() == 0) {
DFService.deregister(myAgent);
}else if(driverData.getTaxiStatus() ==1) {
registerWithDF();
//this.notify();
}
}
var serviceavailabilitychange = (driverData.getOnService() != driverStatUpdate.getOnService());

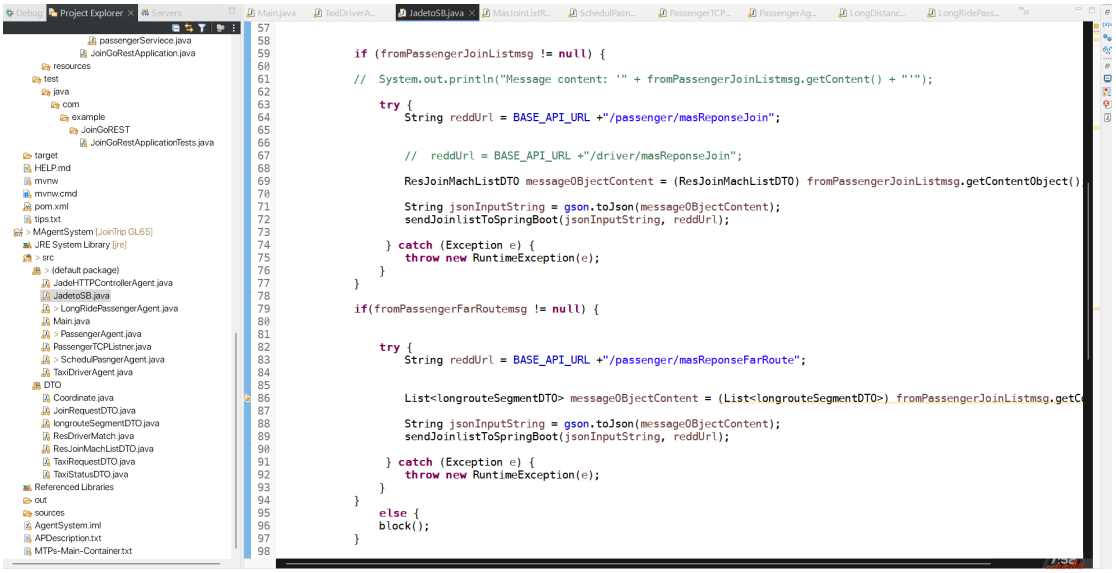


```
class lookingfoTripCalls extends CyclicBehaviour{


    @Override
    public void action() {

        MessageTemplate tripCallTemplate = MessageTemplate.MatchConversationId("passengerTripCall");
        ACLMessage tripCallMsg = receive(tripCallTemplate);

        try {

            if(tripCallMsg !=null) {
                TaxiRequestDTO taxiCall = (TaxiRequestDTO) tripCallMsg.getContentObject();
                double distance = haversine(driverData.getCurrentLat(), driverData.getCurrentLon(), taxiCall.startLat, tax
                if(distance <= closeradius && taxiCall.vehicletype == driverData.getVehicletype()) {

                    //A latitude of 0 and a longitude of 0 (0, 0) points to a location in the Gulf of Guinea, off the coast
                    if(driverData.getHeadingLat() == 0 && driverData.getHeadingLon() == 0) {
                        ACLMessage msg = new ACLMessage(ACLMessage.REQUEST);

                        msg.setConversationId("fromJadetoSB");
                        msg.setContent("DriverAboutMatch");

                        ResDriverMatch driveMatchList = new ResDriverMatch();
                        driveMatchList.setTaxiStatust(driverData);
                        driveMatchList.setTaxiRequest(taxiCall);

                        msg.setContentObject((Serializable) driveMatchList);
                        msg.addReceiver(new AID("Jade2SBAgent", AID.ISLOCALNAME));
                        send(msg);
                    }else {
                        double enddistance = haversine(driverData.getHeadingLat(), driverData.getHeadingLon(), taxiCall.de
                        if(enddistance <= homecloseradius) {
                            ACLMessage msg = new ACLMessage(ACLMessage.REQUEST);

                            msg.setConversationId("fromJadetoSB");
                            msg.setContent("DriverAboutMatch");

                            ResDriverMatch driveMatchList = new ResDriverMatch();
                            driveMatchList.setTaxiStatust(driverData);
                            driveMatchList.setTaxiRequest(taxiCall);
```

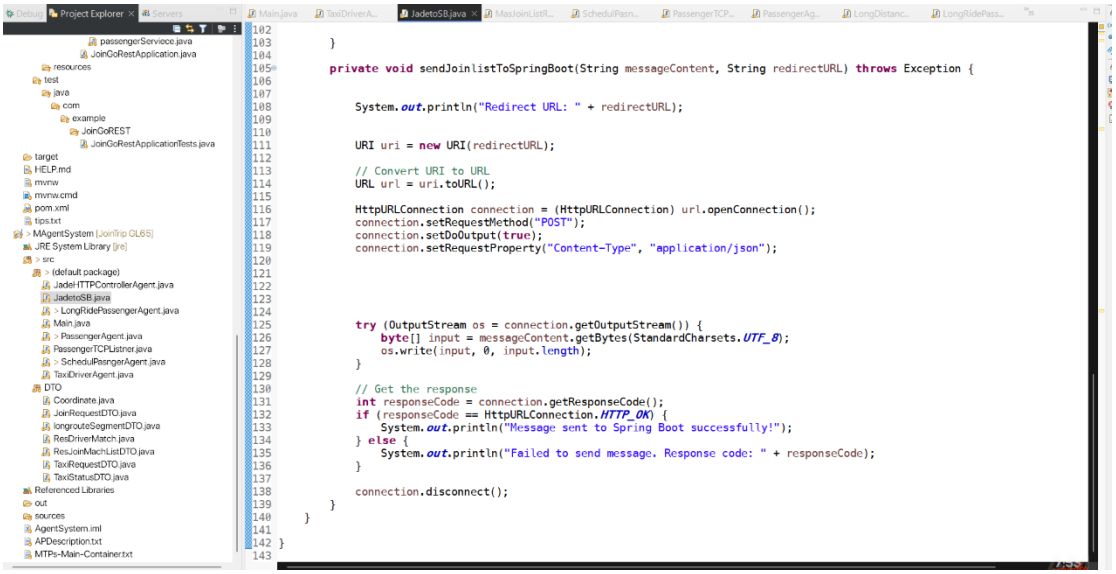
- JadetoSB Agent class-



```
import java.io.OutputStream;

public class JadetoSB extends Agent {

    @Override
    protected void setup() {
        ServiceDescription sd = new ServiceDescription();
        sd.setType("JadetoSB");
        sd.setName(getLocalName());
        DFAgentDescription dfd = new DFAgentDescription();
        dfd.setName(getAID());
        dfd.addServices(sd);
        try {
            DFService.register(this, dfd);
        } catch (FIPAException fe) {
            fe.printStackTrace();
        }
        System.out.println(getAID().getLocalName()+" : JadetoSB agent created.");
        addBehaviour(new JadetoSB.WaitformsgBehaviour());
    }


    class WaitformsgBehaviour extends CyclicBehaviour {
        private final Gson gson = new Gson();
        private final String BASE_API_URL = "http://localhost:8080/api/v1";

        @Override
        public void action() {



            MessageTemplate fromPassengerJoinListTemplate = MessageTemplate.MatchConversationId("fromPassengerJoinList");
            ACLMessage fromPassengerJoinListmsg = receive(fromPassengerJoinListTemplate);

            MessageTemplate fromPassengerFarRouteTemplate = MessageTemplate.MatchConversationId("fromPassengerLongroutSegs");
            ACLMessage fromPassengerFarRoutemsg = receive(fromPassengerFarRouteTemplate);


            if (fromPassengerJoinListmsg != null) {

            //  System.out.println("Message content: '" + fromPassengerJoinListmsg.getContent() + "'");
```

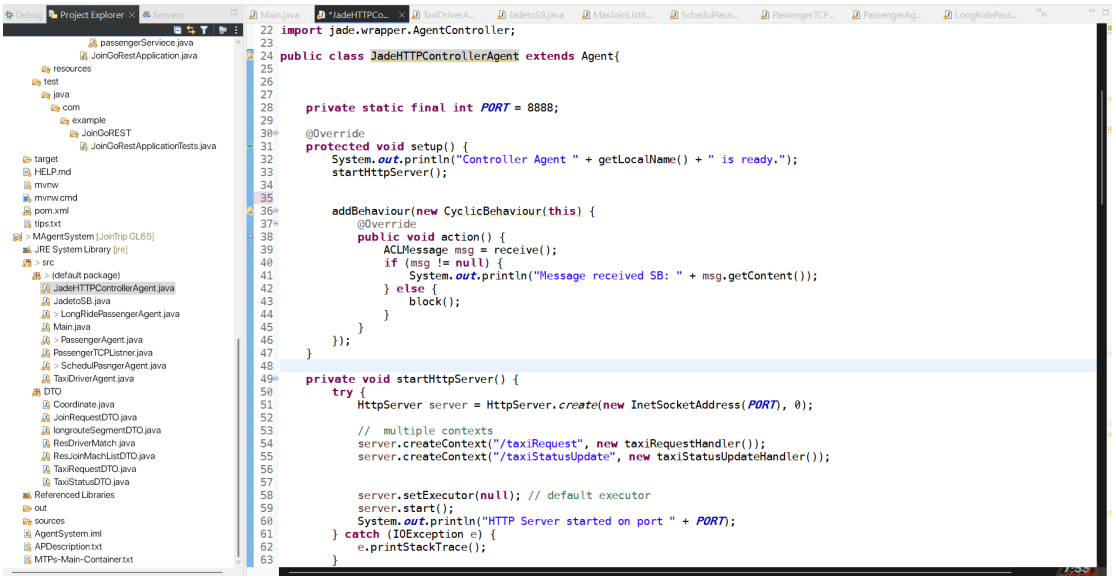


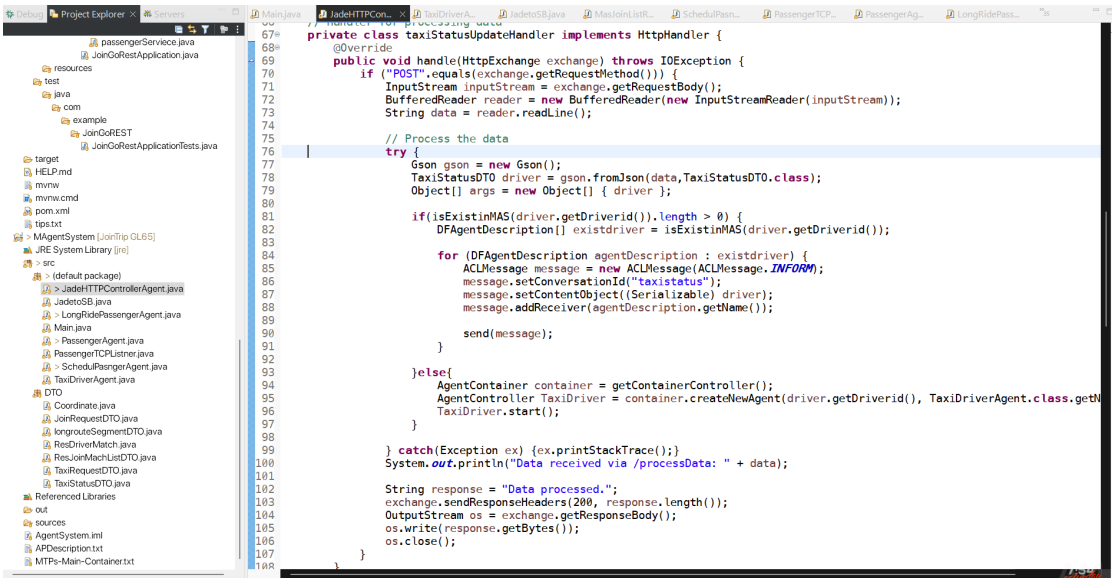
```
57
58
59          if (fromPassengerJoinListmsg != null) {
60
61          //  System.out.println("Message content: '" + fromPassengerJoinListmsg.getContent() + "'");
62
63              try {
64                  String reddUrl = BASE_API_URL +"/passenger/masReponseJoin";
65
66
67                  //  reddUrl = BASE_API_URL +"/driver/masReponseJoin";
68
69                  ResJoinMachListDTO messageOBjectContent = (ResJoinMachListDTO) fromPassengerJoinListmsg.getContentObject();
70
71                  String jsonInputString = gson.toJson(messageOBjectContent);
72                  sendJoinlistToSpringBoot(jsonInputString, reddUrl);
73
74               } catch (Exception e) {
75                  throw new RuntimeException(e);
76              }
77          }
78
79          if(fromPassengerFarRoutemsg != null) {
80
81
82              try {
83                  String reddUrl = BASE_API_URL +"/passenger/masReponseFarRoute";
84
85
86                  List<longrouteSegmentDTO> messageOBjectContent = (List<longrouteSegmentDTO>) fromPassengerJoinListmsg.getC
87
88                  String jsonInputString = gson.toJson(messageOBjectContent);
89                  sendJoinlistToSpringBoot(jsonInputString, reddUrl);
90
91               } catch (Exception e) {
92                  throw new RuntimeException(e);
93              }
94          }
95              else {
96              block();
97          }
98
```

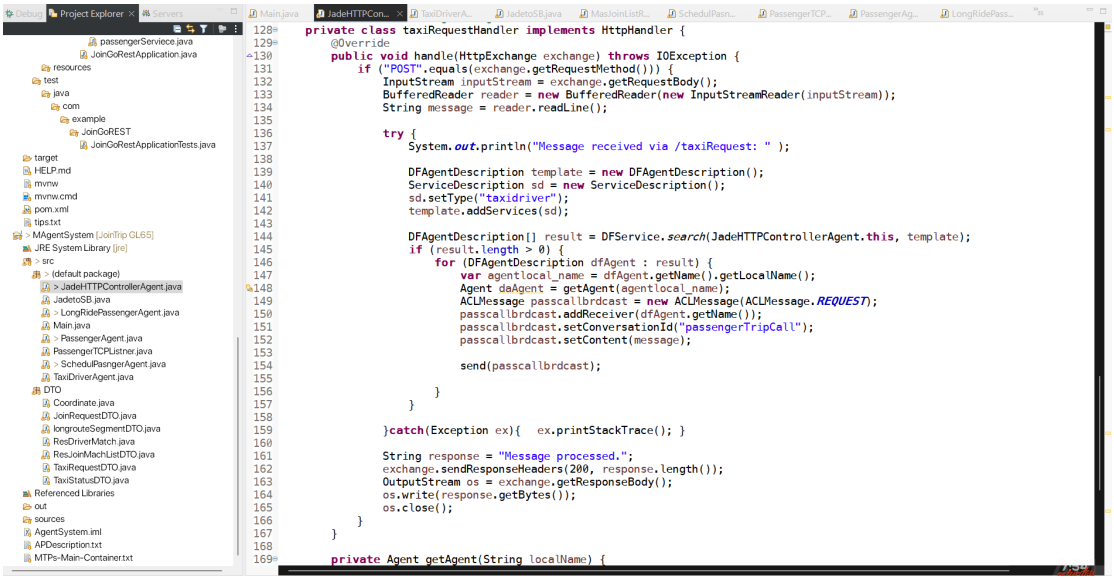


```
102
103         }
104
105         private void sendJoinlistToSpringBoot(String messageContent, String redirectURL) throws Exception {
106
107
108             System.out.println("Redirect URL: " + redirectURL);
109
110
111             URI uri = new URI(redirectURL);
112
113             // Convert URI to URL
114             URL url = uri.toURL();
115
116             HttpURLConnection connection = (HttpURLConnection) url.openConnection();
117             connection.setRequestMethod("POST");
118             connection.setDoOutput(true);
119             connection.setRequestProperty("Content-Type", "application/json");
120
121
122
123
124
125             try (OutputStream os = connection.getOutputStream()) {
126                 byte[] input = messageContent.getBytes(StandardCharsets.UTF_8);
127                 os.write(input, 0, input.length);
128             }
129
130             // Get the response
131             int responseCode = connection.getResponseCode();
132             if (responseCode == HttpURLConnection.HTTP_OK) {
133                 System.out.println("Message sent to Spring Boot successfully!");
134             } else {
135                 System.out.println("Failed to send message. Response code: " + responseCode);
136             }
137
138             connection.disconnect();
139         }
140     }
141
142 }
143
```

- JadeHTTPControllerAgent class-


Project Explorer
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
HELP.md
mvnw
mvnw.cmd
pom.xml
tips.txt
> MAgentSystem [JoinTrip GL65]
JRE System Library [jre]
> src
> (default package)
JadeHTTPControllerAgent.java
JadetoSB.java
> LongRidePassengerAgent.java
Main.java
> PassengerAgent.java
PassengerTCPListner.java
> SchedulPasngerAgent.java
TaxiDriverAgent.java
DTO
Coordinate.java
JoinRequestDTO.java
longrouteSegmentDTO.java
ResDriverMatch.java
ResJoinMachListDTO.java
TaxiRequestDTO.java
TaxiStatusDTO.java
Referenced Libraries
out
sources
AgentSystem.iml
APDescription.txt
MTPs-Main-Container.txt
Main.java
*JadeHTTPCo...
TaxiDriverA...
JadetoSB.java
MasJoinListR...
SchedulPasn...
PassengerTCP...
PassengerAg...
LongRidePass...
import jade.wrapper.AgentController;
public class JadeHTTPControllerAgent extends Agent{
private static final int PORT = 8888;
@Override
protected void setup() {
System.out.println("Controller Agent " + getLocalName() + " is ready.");
startHttpServer();
addBehaviour(new CyclicBehaviour(this) {
@Override
public void action() {
ACLMessage msg = receive();
if (msg != null) {
System.out.println("Message received SB: " + msg.getContent());
} else {
block();
}
}
});
}
private void startHttpServer() {
try {
HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
// multiple contexts
server.createContext("/taxiRequest", new taxiRequestHandler());
server.createContext("/taxiStatusUpdate", new taxiStatusUpdateHandler());
server.setExecutor(null); // default executor
server.start();
System.out.println("HTTP Server started on port " + PORT);
} catch (IOException e) {
e.printStackTrace();
}


Project Explorer
passengerServiece.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
HELP.md
mvnw
mvnw.cmd
pom.xml
tips.txt
> MAgentSystem [JoinTrip GL65]
JRE System Library [jre]
> src
> (default package)
> JadeHTTPControllerAgent.java
JadetoSB.java
> LongRidePassengerAgent.java
Main.java
> PassengerAgent.java
PassengerTCPListner.java
> SchedulPasngerAgent.java
TaxiDriverAgent.java
DTO
Coordinate.java
JoinRequestDTO.java
longrouteSegmentDTO.java
ResDriverMatch.java
ResJoinMachListDTO.java
TaxiRequestDTO.java
TaxiStatusDTO.java
Referenced Libraries
out
sources
AgentSystem.iml
APDescription.txt
MTPs-Main-Container.txt
Main.java
JadeHTTPCon...
TaxiDriverA...
JadetoSB.java
MasJoinListR...
SchedulPasn...
PassengerTCP...
PassengerAg...
LongRidePass...
private class taxiStatusUpdateHandler implements HttpHandler {
@Override
public void handle(HttpExchange exchange) throws IOException {
if ("POST".equals(exchange.getRequestMethod())) {
InputStream inputStream = exchange.getRequestBody();
BufferedReader reader = new BufferedReader(new InputStreamReader(inputStream));
String data = reader.readLine();
// Process the data
try {
Gson gson = new Gson();
TaxiStatusDTO driver = gson.fromJson(data,TaxiStatusDTO.class);
Object[] args = new Object[] { driver };
if(isExistinMAS(driver.getDriverid()).length > 0) {
DFAgentDescription[] existdriver = isExistinMAS(driver.getDriverid());
for (DFAgentDescription agentDescription : existdriver) {
ACLMessage message = new ACLMessage(ACLMessage.INFORM);
message.setConversationId("taxistatus");
message.setContentObject((Serializable) driver);
message.addReceiver(agentDescription.getName());
send(message);
}
}else{
AgentContainer container = getContainerController();
AgentController TaxiDriver = container.createNewAgent(driver.getDriverid(), TaxiDriverAgent.class.getN
TaxiDriver.start();
}
} catch(Exception ex) {ex.printStackTrace();}
System.out.println("Data received via /processData: " + data);
String response = "Data processed.";
exchange.sendResponseHeaders(200, response.length());
OutputStream os = exchange.getResponseBody();
os.write(response.getBytes());
os.close();
}


Project Explorer
passengerService.java
JoinGoRestApplication.java
resources
test
java
com
example
JoinGoREST
JoinGoRestApplicationTests.java
target
HELP.md
mvnw
mvnw.cmd
pom.xml
tips.txt
> MAgentSystem [JoinTrip GL65]
JRE System Library [jre]
> src
> (default package)
> JadeHTTPControllerAgent.java
JadetoSB.java
> LongRidePassengerAgent.java
Main.java
> PassengerAgent.java
PassengerTCPListner.java
> SchedulPasngerAgent.java
TaxiDriverAgent.java
DTO
Coordinate.java
JoinRequestDTO.java
longrouteSegmentDTO.java
ResDriverMatch.java
ResJoinMachListDTO.java
TaxiRequestDTO.java
TaxiStatusDTO.java
Referenced Libraries
out
sources
AgentSystem.iml
APDescription.txt
MTPs-Main-Container.txt
Main.java
JadeHTTPCon...
TaxiDriverA...
JadetoSB.java
MasJoinListR...
SchedulPasn...
PassengerTCP...
PassengerAg...
LongRidePass...
private class taxiRequestHandler implements HttpHandler {
@Override
public void handle(HttpExchange exchange) throws IOException {
if ("POST".equals(exchange.getRequestMethod())) {
InputStream inputStream = exchange.getRequestBody();
BufferedReader reader = new BufferedReader(new InputStreamReader(inputStream));
String message = reader.readLine();
try {
System.out.println("Message received via /taxiRequest: ");
DFAgentDescription template = new DFAgentDescription();
ServiceDescription sd = new ServiceDescription();
sd.setType("taxidriver");
template.addServices(sd);
DFAgentDescription[] result = DFService.search(JadeHTTPControllerAgent.this, template);
if (result.length > 0) {
for (DFAgentDescription dfAgent : result) {
var agentlocal_name = dfAgent.getName().getLocalName();
Agent daAgent = getAgent(agentlocal_name);
ACLMessage passcallbrdcast = new ACLMessage(ACLMessage.REQUEST);
passcallbrdcast.addReceiver(dfAgent.getName());
passcallbrdcast.setConversationId("passengerTripCall");
passcallbrdcast.setContent(message);
send(passcallbrdcast);
}
}
}catch(Exception ex){ ex.printStackTrace(); }
String response = "Message processed.";
exchange.sendResponseHeaders(200, response.length());
OutputStream os = exchange.getResponseBody();
os.write(response.getBytes());
os.close();
}
}
private Agent getAgent(String localName) {